ML / Math Audit — Marketing AI Platform

Документ для математического аудита логических ошибок

> *Каждый файл содержит: назначение, математическую логику, входные/выходные данные, взаимодействие с другими файлами, потенциальные риски. Полный исходный код — после описания.*

СЕКЦИЯ 1 — Thompson Sampling и Байесовские эксперименты

1.1 `ml-service/app/services/experiment_service.py`

Назначение: Управление жизненным циклом A/B-экспериментов и принятие решения о завершении/паузе.

Математическая логика:
- **ESS (Effective Sample Size)** = Σ(α + β − 2) по всем рукавам. Вычитание 2 удаляет плоский приор Beta(1,1).
- **P(best)** — Монте-Карло симуляция: n=5000 семплов из Beta(α_i, β_i) для каждого рукава; доля итераций, где рукав i выиграл у всех остальных.
- **Авто-завершение:** P(best) > 0.95 И ESS ≥ 200.
- **Авто-пауза:** P(хуже контроля) > 0.95 И ESS ≥ 50. Контрольный рукав НИКОГДА не паузируется.
- **Информированные приоры при перезапуске:** победитель → Beta(5,1), проигравший → Beta(1,5), сомнительный → Beta(2,2).
- **Деградация к приору:** α = 1 + 0.99·(α−1); β = 1 + 0.99·(β−1) (не наивное умножение α×0.99).

Входные данные: список рукавов с (α, β, variant_id, is_control), min_ess, auto_stop_p.
Выходные данные: {should_conclude, winner_variant_id, reason}; список рукавов для паузы.

Взаимодействие: вызывается из `ExperimentService.java` (Spring Boot). Единственный писатель thompson_arms.

Потенциальные риски:
- При n=5000 оценка P(best) имеет стандартную ошибку ~0.7% → при граничном P=0.95 можно ошибиться. Рекомендуется n=10000+.
- ESS = Σ(α+β−2) корректен только при Beta(1,1) приоре. Если приоры информированы (Beta(5,1)), ESS занижается.

Полный код

```
"""
A/B Experiment Lifecycle — Bayesian stopping rules (vision §3; arch §16.9).
Pure logic — zero FastAPI/Pydantic imports.

Stopping criteria:
- Auto-conclude: P(best arm > all others) > auto_stop_p AND ESS >= min_ess
- Auto-pause: P(arm worse than control) > 0.95 AND ESS >= 50
ESS approximated as sum(alpha + beta - 2) across arms.
"""
from __future__ import annotations
import random


def estimate_ess(
    arms: list[dict],
    prior_alpha: float = 1.0,
    prior_beta: float = 1.0,
) -> float:
    """
    ESS approximation: sum of (alpha + beta - prior_alpha - prior_beta) across arms.

    Subtracts the actual prior parameters so informed priors (alpha > 1 or beta > 1)
    are not counted as observed data.  For flat Beta(1,1) priors the result is
    identical to the old ``- 2`` formula.
    """
    return sum(
        max(float(a.get("alpha", 1.0)) + float(a.get("beta", 1.0)) - prior_alpha - prior_beta, 0)
        for a in arms
    )


def thompson_best_arm(arms: list[dict], n_samples: int = 10_000) -> list[dict]:
    """
    Returns list of {variant_id, win_probability, sample_mean} for all arms.
    win_probability = fraction of MC trials this arm was best.
    """
    if not arms:
        return []

    wins: dict[str, int] = {a["variant_id"]: 0 for a in arms}
    for _ in range(n_samples):
        samples = {
            a["variant_id"]: random.betavariate(
                max(float(a.get("alpha", 1.0)), 1.0),
                max(float(a.get("beta",  1.0)), 1.0),
            )
            for a in arms
        }
        best = max(samples, key=lambda k: samples[k])
        wins[best] += 1

    results = []
    for a in arms:
        vid   = a["variant_id"]
        alpha = max(float(a.get("alpha", 1.0)), 1.0)
        beta  = max(float(a.get("beta",  1.0)), 1.0)
        results.append({
            "variant_id":      vid,
            "win_probability": round(wins[vid] / n_samples, 4),
            "sample_mean":     round(alpha / (alpha + beta), 4),
            "alpha":           alpha,
            "beta":            beta,
        })
```

```
    results.sort(key=lambda x: x["win_probability"], reverse=True)
    return results


def should_conclude(
    arms: list[dict],
    min_ess: int = 200,
    auto_stop_p: float = 0.95,
) -> dict:
    """
    Checks whether the experiment should be concluded.
    Returns {should_conclude, winner_variant_id, reason, ess, best_win_prob}.
    """
    ess = estimate_ess(arms)
    if ess < min_ess:
        return {
            "should_conclude":   False,
            "reason":            f"ESS {ess:.0f} < min_ess {min_ess}",
            "ess":               ess,
            "winner_variant_id": None,
            "best_win_prob":     None,
        }

    ranked = thompson_best_arm(arms)
    if not ranked:
        return {"should_conclude": False, "reason": "no arms", "ess": ess}

    best = ranked[0]
    if best["win_probability"] >= auto_stop_p:
        return {
            "should_conclude":   True,
            "winner_variant_id": best["variant_id"],
            "best_win_prob":     best["win_probability"],
            "reason":            f"P(best)={best['win_probability']:.3f} >= {auto_stop_p}",
            "ess":               ess,
            "ranked":            ranked,
        }

    return {
        "should_conclude":   False,
        "reason":            f"P(best)={best['win_probability']:.3f} < {auto_stop_p}",
        "ess":               ess,
        "best_win_prob":     best["win_probability"],
        "ranked":            ranked,
    }


def arm_worse_than_control(
    arm_alpha: float,
    arm_beta:  float,
    ctrl_alpha: float,
    ctrl_beta:  float,
    n_samples: int = 2000,
) -> float:
    """Monte Carlo P(arm < control). Used for auto-pause."""
    worse = 0
    for _ in range(n_samples):
        a = random.betavariate(max(arm_alpha, 1.0), max(arm_beta, 1.0))
        c = random.betavariate(max(ctrl_alpha, 1.0), max(ctrl_beta, 1.0))
        if a < c:
            worse += 1
    return worse / n_samples


def check_auto_pause(
```

```
    arms: list[dict],
    min_ess_pause: int = 50,
    p_threshold: float = 0.95,
) -> list[dict]:
    """
    Returns list of {variant_id, p_worse, should_pause} for non-control arms.
    Flags arm for pause if P(arm < control) > p_threshold AND ESS >= min_ess_pause.
    """
    control = next((a for a in arms if a.get("is_control")), None)
    if control is None:
        return []

    ctrl_alpha = max(float(control.get("alpha", 1.0)), 1.0)
    ctrl_beta  = max(float(control.get("beta",  1.0)), 1.0)

    results = []
    for arm in arms:
        if arm.get("is_control"):
            continue
        arm_alpha = max(float(arm.get("alpha", 1.0)), 1.0)
        arm_beta  = max(float(arm.get("beta",  1.0)), 1.0)
        arm_ess   = arm_alpha + arm_beta - 2

        if arm_ess < min_ess_pause:
            results.append({
                "variant_id":   arm["variant_id"],
                "p_worse":      None,
                "should_pause": False,
                "reason":       f"ESS {arm_ess:.0f} < {min_ess_pause}",
            })
            continue

        p_worse = arm_worse_than_control(arm_alpha, arm_beta, ctrl_alpha, ctrl_beta)
        results.append({
            "variant_id":   arm["variant_id"],
            "p_worse":      round(p_worse, 4),
            "should_pause": p_worse > p_threshold,
            "reason":       f"P(worse)={p_worse:.3f}",
        })

    return results
```

1.2 `ml-service/app/services/thompson_service.py`

Назначение: Полный асинхронный Thompson Sampling — выбор рукава, логирование решений, применение наград.

Математическая логика:
- **Семплинг:** sample(α, β) из Beta-распределения для каждого рукава; выбирается максимум.
- **Пропенсити (propensity):** N=200 Монте-Карло семплов; доля, в которой выбранный рукав победил = P(arm selected).
- **Reward Map:** QUALITY_GATE_PASS → +1.0 к α (победа); только бинарные инкременты. Дробные дельты запрещены.
- **Churn penalty:** cap=30% от (α+β). Штраф не может уменьшить α+β более чем на 30%.
- **Benchmark:** P(treatment > holdout) via Монте-Карло N=10000.
- **Redis кеш:** TTL 15 минут на состояние рукавов.

Входные данные: tenant_id, variant_id, reward (1.0), event_type.
Выходные данные: выбранный variant_id, propensity, решение залогировано в decision_log.

Взаимодействие: вызывается FastAPI эндпоинтами `/thompson/select`, `/thompson/reward`; результат используется в `funnel_path_service.py`.

Потенциальные риски:
- N=200 для пропенсити — низкое. Грубая оценка вероятности выбора; для OPE нужна точность выше.
- Кеш 15 мин: в быстрых экспериментах (много трафика) posterior устаревает.

Полный код

```
"""Thompson Sampling service — Beta(α,β) arm selection with Redis-cached posteriors.

Reward schedule (arch v1.1 §4.4, vision v1.9.9 §24):
  ONE binary event per arm. All deltas = 1.0 (integer counts only).
  CTA_CLICK / FORM_START   → α += 1  (primary micro-conversion, hundreds/week)
  FORM_SUBMIT              → α += 1  (secondary, tens/week; skip when reward_on_form_submit=false)
  CALL_ANSWERED            → α += 1  (sparse CRM signal, used as corrector)
  MEETING_ATTENDED         → α += 1  (key quality signal)
  CONVERSION / PURCHASE    → α += 1
  NO_RESPONSE_7D           → β += 1  (lead went cold)

Fractional increments (e.g. α+=0.3) are FORBIDDEN — arch v1.1 §4.4.

Decay toward prior (arch v1.1 §9, vision v1.9.9 §6.2):
  α = 1 + 0.99·(α − 1),  β = 1 + 0.99·(β − 1)
  Naive α×=0.99 drives α below 1 → Beta(0,0) is improper. Decay toward prior is correct.

Propensity (arch v1.1 §4.6):
  Thompson's propensity has no closed form; estimated at decision time by
  Monte Carlo (N=200 draws from the cached posteriors, <1ms) and stored in candidates.

Cache: Redis key posterior:{tenant_id}:{context_key}, TTL 15 min.
"""

import hashlib
import json
import math
from datetime import datetime, timezone
from typing import Optional

import numpy as np

from app.config import settings
from app.services.db_client import system_pool, tenant_pool  # ML-26: removed unused get_pool
from app.services.glm_prior_service import prior_for_variant
from app.services.expert_prior_service import prior_for_arm
from app.services.job_store import get_redis

CACHE_PREFIX = "posterior"
CACHE_TTL = 900  # 15 min
DECISION_TTL = 3600  # 1 h
DECAY_FACTOR = 0.99
DECAY_INTERVAL_SECS = 86_400  # 24 h
N_PROPENSITY = 2_000  # Monte Carlo draws for propensity estimation

_UPDATE_ALPHA = ("UPDATE thompson_arms SET alpha=alpha+$1, total_samples=total_samples+1, updated_at=$2
"
                 "WHERE id=$3::uuid AND tenant_id=$4::uuid")
_UPDATE_BETA  = ("UPDATE thompson_arms SET beta=beta+$1,  total_samples=total_samples+1, updated_at=$2 "
                 "WHERE id=$3::uuid AND tenant_id=$4::uuid")
_SET_TENANT   = "SELECT set_config('app.current_tenant', $1, true)"  # SET LOCAL cannot take a bind
parameter

NO_RESPONSE_WINDOW_DAYS = 7   # attribution window before a decision counts as a negative (beta += 1)

# arch v1.1 §4.4: all deltas are 1.0 — binary counts only, no fractional increments
REWARD_MAP: dict[str, tuple[str, float]] = {
    "CTA_CLICK":        ("alpha", 1.0),  # primary micro-conversion (fast loop)
    "FORM_START":       ("alpha", 1.0),  # micro-conversion
    "FORM_SUBMIT":      ("alpha", 1.0),  # secondary (skip via reward_on_form_submit flag for B2B)
    "CALL_ANSWERED":    ("alpha", 1.0),  # sparse CRM signal
    "MEETING_ATTENDED": ("alpha", 1.0),  # key quality signal
    "CONVERSION":       ("alpha", 1.0),
    "PURCHASE":         ("alpha", 1.0),
```

```
    "NO_RESPONSE_7D":   ("beta",  1.0),  # lead went cold
}


# ── Pure helpers (no I/O) ──────────────────────────────────────────

def holdout_bucket(session_id: Optional[str]) -> int:
    """Deterministic 0-9999 bucket. Session ids are cookie strings, not necessarily hex/UUID."""
    if not session_id:
        return int(np.random.randint(0, 10_000))
    return int(hashlib.sha1(session_id.encode("utf-8")).hexdigest()[:8], 16) % 10_000


def decay_toward_prior(value: float, days: int) -> float:
    """arch v1.1 §9: alpha = 1 + 0.99^days * (alpha - 1). Never crosses below the Beta(1,1) prior."""
    if days <= 0:
        return value
    return 1.0 + (DECAY_FACTOR ** days) * (value - 1.0)


def churn_penalty_delta(weighted_sum: float, alpha: float, beta: float) -> tuple[int, bool]:
    """Integer beta penalty for a churn wave (vision §6.7 cap, arch §4.4 binary counts).

    Returns (applied_delta, capped). The cap is 30 % of (alpha+beta); the result is floored
    because fractional increments are forbidden - a penalty below 1 is skipped.
    """
    cap = _CHURN_CAP * (alpha + beta)
    capped = weighted_sum > cap
    return int(math.floor(min(weighted_sum, cap))), capped


def arm_for_decision(chosen_variant_id: str, candidates) -> Optional[str]:
    """Recover the chosen arm id from a decision_log row (candidates JSON)."""
    if isinstance(candidates, str):
        try:
            candidates = json.loads(candidates)
        except ValueError:
            return None
    for c in candidates or []:
        if str(c.get("variant_id")) == str(chosen_variant_id):
            return c.get("arm_id")
    return None


def _as_dict(value) -> dict:
    if isinstance(value, str):
        try:
            return json.loads(value) or {}
        except ValueError:
            return {}
    return value or {}


# ── Arms ──────────────────────────────────────────────────────────

async def ensure_arms(tenant_id: str, context_key: str) -> int:
    """Create Beta(1,1) arms for every ACTIVE variant under context_key (idempotent).

    Audit 2.9: this used to run inside load_arms for *any* requested key, which created a
    cold arm set for every cluster/channel and defeated the documented fallback to the
    channel-global arms. It is now an explicit call made only for the canonical key.
    """
    pool = await tenant_pool(tenant_id)
    async with pool.acquire() as conn:
        status = await conn.execute(
```

```
            """
            INSERT INTO thompson_arms (tenant_id, variant_id, context_key)
            SELECT $1::uuid, lv.id, $2::varchar
            FROM   landing_variants lv
            WHERE  lv.tenant_id = $1::uuid
              AND  lv.status    = 'ACTIVE'
              AND  NOT EXISTS (
                  SELECT 1 FROM thompson_arms ta
                  WHERE  ta.variant_id  = lv.id
                    AND  ta.context_key = $2::varchar
              )
            ON CONFLICT DO NOTHING
            """,
            tenant_id,
            context_key,
        )
        try:
            created = int(status.split()[-1])
        except (ValueError, IndexError):
            created = 0
        if created:
            await _apply_glm_priors(conn, tenant_id, context_key)   # roadmap 9-10
    await get_redis().delete(f"{CACHE_PREFIX}:{tenant_id}:{context_key}")
    return created


GLM_PRIOR_BASE_RATE = 0.03   # fallback CVR when the tenant has no closed sessions yet
GLM_PRIOR_BASE_SD   = 0.02


async def _apply_glm_priors(conn, tenant_id: str, context_key: str) -> int:
    """Roadmap 9-10 (vision §15.6): new arms start from the GLM posterior predictive of their atoms
    instead of Beta(1,1). Only untouched arms (total_samples = 0, alpha = beta = 1) are changed, so a
    re-run never overwrites learned counts. Without GLM posteriors nothing happens."""
    try:
        posteriors = await conn.fetch(
            "SELECT atom_type, atom_value, mean, std AS sd FROM glm_posteriors "
            "WHERE tenant_id = $1::uuid AND model_stage = 'CVR'",
            tenant_id,
        )
        base = await conn.fetchrow(
            """
            SELECT COUNT(*) FILTER (WHERE converted) AS conv, COUNT(*) AS n
            FROM   sessions WHERE tenant_id = $1::uuid AND started_at > now() - INTERVAL '90 days'
            """,
            tenant_id,
        )
        n = int(base["n"] or 0) if base else 0
        base_rate = (int(base["conv"] or 0) + 1) / (n + 2) if n >= 50 else GLM_PRIOR_BASE_RATE
        base_sd = GLM_PRIOR_BASE_SD
        fresh = await conn.fetch(
            """
            SELECT ta.id::text AS id, lv.creative_id::text AS creative_id,
                   c.exploration_prior_alpha AS exp_alpha, c.exploration_prior_beta AS exp_beta
            FROM   thompson_arms ta JOIN landing_variants lv ON lv.id = ta.variant_id
            LEFT JOIN creatives c ON c.id = lv.creative_id
            WHERE  ta.tenant_id = $1::uuid AND ta.context_key = $2
              AND  ta.total_samples = 0 AND ta.alpha = 1 AND ta.beta = 1
            """,
            tenant_id, context_key,
        )
        atom_posts = [dict(r) for r in posteriors]
        updated = 0
        for arm in fresh:
            if not arm["creative_id"]:
```

```
                continue
            atoms = await conn.fetch(
                "SELECT atom_type, atom_value FROM creative_atoms WHERE creative_id = $1::uuid AND
tenant_id = $2::uuid",
                arm["creative_id"], tenant_id,
            )
            prior = prior_for_variant(atom_posts, [dict(a) for a in atoms], base_rate, base_sd)
            # roadmap 9-9 / arch §15.13: expert rating prior (creatives.exploration_prior_*) combined
with the GLM one
            stored = (arm["exp_alpha"], arm["exp_beta"]) if arm["exp_alpha"] is not None else None
            resolved = prior_for_arm(prior, None, stored=stored)
            if resolved["source"] == "FLAT":
                continue
            await conn.execute(
                "UPDATE thompson_arms SET alpha = $1, beta = $2, updated_at = now() "
                "WHERE id = $3::uuid AND total_samples = 0",
                float(resolved["alpha0"]), float(resolved["beta0"]), arm["id"],
            )
            updated += 1
        return updated
    except Exception as exc:  # priors are an optimisation, never a failure of /decide
        import logging
        logging.getLogger(__name__).warning("GLM priors skipped: %s", exc)
        return 0


async def load_arms(tenant_id: str, context_key: str = "global") -> list[dict]:
    """Return ACTIVE arms from Redis cache or DB (read-only apart from decay).

    Decay (arch v1.1 §9) is applied atomically in SQL for arms idle > 24 h, using
    0.99^days so a 10-day idle arm decays 10 steps, not one (audit 2.6, 6.22).
    Concurrent alpha/beta increments are never overwritten because the UPDATE is relative.
    """
    redis = get_redis()
    cache_key = f"{CACHE_PREFIX}:{tenant_id}:{context_key}"
    cached = await redis.get(cache_key)
    if cached:
        return json.loads(cached)

    pool = await tenant_pool(tenant_id)
    async with pool.acquire() as conn:
        await conn.execute(
            """
            UPDATE thompson_arms ta
            SET    alpha           = 1 + power($3::float8, d.days) * (ta.alpha - 1),
                   beta            = 1 + power($3::float8, d.days) * (ta.beta  - 1),
                   last_decayed_at = now(),
                   updated_at      = now()
            FROM (
                SELECT id,
                       floor(extract(epoch FROM (now() - last_decayed_at)) / 86400)::int AS days
                FROM   thompson_arms
                WHERE  tenant_id   = $1::uuid
                  AND  context_key = $2
                  AND  last_decayed_at < now() - INTERVAL '24 hours'
            ) d
            WHERE ta.id = d.id
            """,
            tenant_id,
            context_key,
            DECAY_FACTOR,
        )
        rows = await conn.fetch(
            """
            SELECT ta.id::text            AS id,
```

```
                    ta.variant_id::text    AS variant_id,
                    ta.alpha,
                    ta.beta,
                    ta.total_samples,
                    ta.last_decayed_at,
                    lv.block_order
            FROM    thompson_arms ta
            JOIN    landing_variants lv ON lv.id = ta.variant_id
            WHERE   ta.tenant_id  = $1::uuid
              AND   ta.context_key = $2
              AND   lv.status      = 'ACTIVE'
              -- roadmap 9-9 / arch §15.6: margin floor — variants without an offer row stay eligible
              -- ML-25: safe-cast min_margin; non-numeric value → NULL → COALESCE fallback avoids 500
              AND   COALESCE((SELECT MIN(vo.estimated_margin) FROM variant_offers vo WHERE vo.variant_id
= lv.id), 1e9)
                    >= COALESCE((SELECT CASE WHEN t.settings->>'min_margin' ~ '^-?[0-9]+(\\.[0-9]+)?$'
                                             THEN (t.settings->>'min_margin')::float8 END
                                 FROM tenants t WHERE t.id = $1::uuid), -1e9)
            """,
            tenant_id,
            context_key,
        )
        excluded = await conn.fetchval(
            """
            SELECT COUNT(*) FROM thompson_arms ta JOIN landing_variants lv ON lv.id = ta.variant_id
            WHERE ta.tenant_id = $1::uuid AND ta.context_key = $2 AND lv.status = 'ACTIVE'
            """,
            tenant_id, context_key,
        )
        if excluded is not None and int(excluded) > len(rows):
            import logging
            logging.getLogger(__name__).info(
                "load_arms: %d variant(s) excluded by min_margin for tenant %s", int(excluded) -
len(rows), tenant_id)

    if not rows:
        return []

    arms: list[dict] = [
        {
            "id": row["id"],
            "variant_id": row["variant_id"],
            "alpha": float(row["alpha"]),
            "beta": float(row["beta"]),
            "total_samples": row["total_samples"],
            "block_order": list(row["block_order"]) if row["block_order"] else [],
        }
        for row in rows
    ]

    await redis.setex(cache_key, CACHE_TTL, json.dumps(arms))
    return arms


def sample_arms(arms: list[dict]) -> tuple[dict, list[dict]]:
    """Beta(α,β) sampling — pure, no I/O. Returns (best_arm, all_candidates).

    candidates format (arch v1.1 §4.6):
      [{"variant_id":..., "arm_id":..., "propensity":0.63, "sample_value":0.42,
        "alpha":12, "beta":88}, ...]

    propensity = P(arm is chosen by Thompson), estimated via N=200 Monte Carlo draws.
    Required for off-policy evaluation; stored in decision_log before /decide returns.
    """
    # 1. One real sample — determines the winner this request
```

```
    sample_values: dict[str, float] = {}
    best: dict | None = None
    best_val = -1.0

    for arm in arms:
        val = float(np.random.beta(arm["alpha"], arm["beta"]))
        sample_values[arm["id"]] = val
        if val > best_val:
            best_val = val
            best = {**arm, "sample_value": val}

    # 2. Monte Carlo propensity (N=200, <1ms) — arch v1.1 §4.6
    win_counts: dict[str, int] = {arm["id"]: 0 for arm in arms}
    for _ in range(N_PROPENSITY):
        mc_best_val = -1.0
        mc_best_id: str | None = None
        for arm in arms:
            v = float(np.random.beta(arm["alpha"], arm["beta"]))
            if v > mc_best_val:
                mc_best_val = v
                mc_best_id = arm["id"]
        if mc_best_id is not None:
            win_counts[mc_best_id] += 1

    # 3. Build candidates with propensity field
    candidates: list[dict] = [
        {
            "variant_id":   arm["variant_id"],
            "arm_id":       arm["id"],
            "propensity":   round(win_counts[arm["id"]] / N_PROPENSITY, 4),
            "sample_value": round(sample_values[arm["id"]], 6),
            "alpha":        arm["alpha"],
            "beta":         arm["beta"],
        }
        for arm in arms
    ]

    return best, candidates  # type: ignore[return-value]


async def log_decision(
    tenant_id: str,
    session_id: str,
    creative_id: Optional[str],
    chosen: dict,
    candidates: list[dict],
    context: dict,
    is_holdout: bool = False,
) -> str:
    """Insert into decision_log and cache for reward lookups. Returns decision_id."""
    pool = await tenant_pool(tenant_id)
    async with pool.acquire() as conn:
        row = await conn.fetchrow(
            """
            INSERT INTO decision_log
                (tenant_id, session_id, creative_id, chosen_variant_id,
                 candidates, context, model_version, is_holdout)
            VALUES ($1, $2, $3, $4, $5::jsonb, $6::jsonb, 'beta-v1', $7)
            RETURNING id::text AS id
            """,
            tenant_id,
            session_id,
            creative_id,
            chosen["variant_id"],
            json.dumps(candidates),
```

```
            json.dumps(context),
            is_holdout,
        )

    decision_id: str = row["id"]
    redis = get_redis()
    await redis.setex(
        f"decision:{decision_id}",
        DECISION_TTL,
        json.dumps(
            {
                "tenant_id": tenant_id,
                "variant_id": chosen["variant_id"],
                "arm_id": chosen["id"],
                "context_key": context.get("context_key", "global"),
            }
        ),
    )
    return decision_id


_CHURN_CAP = 0.30  # vision §6.7: max 30% of (α+β) per retroactive recalc


async def apply_churn_penalty(
    tenant_id: str,
    arm_id: str,
    context_key: str,
    weighted_sum: float,
    cohort_size: int,
    churn_count: int,
) -> dict:
    """Retroactively penalise a Thompson arm for post-conversion churn (vision §6.7).

    weighted_sum: sum of exp(-0.02 * days_since_decision) over churned sessions.
    Cap: penalty ≤ 30% of current (α+β) so a single churn wave can't kill a good arm.
    """
    pool = await tenant_pool(tenant_id)
    async with pool.acquire() as conn:
        row = await conn.fetchrow(
            """
            SELECT id::text, variant_id::text, alpha, beta, context_key
            FROM   thompson_arms
            WHERE  id = $1::uuid AND tenant_id = $2::uuid
            """,
            arm_id,
            tenant_id,
        )
        if not row:
            raise ValueError(f"Arm not found: {arm_id}")

        current_alpha = float(row["alpha"])
        current_beta  = float(row["beta"])
        applied_delta, capped = churn_penalty_delta(weighted_sum, current_alpha, current_beta)
        if applied_delta == 0:
            return {"arm_id": arm_id, "beta_delta": 0, "capped": capped,
                    "new_beta": current_beta, "skipped": True,
                    "reason": "penalty below 1 - fractional increments forbidden (arch §4.4)"}

        now = datetime.now(timezone.utc)
        await conn.execute(
            "UPDATE thompson_arms SET beta = beta + $1, total_samples = total_samples + $1, "
            "updated_at = $2 WHERE id = $3::uuid AND tenant_id = $4::uuid",
            applied_delta,
            now,
```

```
            arm_id,
            tenant_id,
        )
        await conn.execute(
            """
            INSERT INTO churn_penalty_log
                (tenant_id, arm_id, variant_id, context_key,
                 beta_delta, cohort_size, churn_count, weighted_sum, capped)
            VALUES ($1::uuid, $2::uuid, $3::uuid, $4,
                    $5, $6, $7, $8, $9)
            """,
            tenant_id,
            arm_id,
            row["variant_id"],
            context_key,
            applied_delta,
            cohort_size,
            churn_count,
            weighted_sum,
            capped,
        )

    redis = get_redis()
    await redis.delete(f"{CACHE_PREFIX}:{tenant_id}:{context_key}")

    return {
        "arm_id": arm_id,
        "beta_delta": applied_delta,
        "capped": capped,
        "new_beta": current_beta + applied_delta,
    }


async def _load_decision(decision_id: str) -> Optional[dict]:
    """Audit 2.2: the Redis decision key lives 1 h, CRM rewards arrive days later."""
    pool = await system_pool()
    async with pool.acquire() as conn:
        row = await conn.fetchrow(
            """
            SELECT tenant_id::text AS tenant_id, chosen_variant_id::text AS variant_id,
                   session_id::text AS session_id, candidates, context
            FROM   decision_log
            WHERE  id = $1::uuid
            """,
            decision_id,
        )
    if not row:
        return None
    return {
        "tenant_id":   row["tenant_id"],
        "variant_id":  row["variant_id"],
        "session_id":  row["session_id"],  # ML-5: include session_id for reward validation
        "arm_id":      arm_for_decision(row["variant_id"], row["candidates"]),
        "context_key": _as_dict(row["context"]).get("context_key", "global"),
    }


async def apply_reward(
    decision_id: str,
    reward_type: str,
    tenant_configured: bool = False,
    session_id: Optional[str] = None,  # ML-5: optional caller-supplied session for validation
) -> dict:
    """Update the alpha or beta of the arm that was chosen in the given decision.
```

```
    - Idempotent per (decision_id, reward_type) via thompson_reward_log (audit 2.8).
    - Falls back to decision_log when the Redis key has expired (audit 2.2).
    - A positive reward reverses an earlier NO_RESPONSE_7D sweep for the same decision,
      so every decision stays one Bernoulli observation.
    - vision §16.1: FORM_SUBMIT reward is disabled when reward_on_form_submit=false — unless the
      tenant defines tenants.settings.reward_levels (roadmap 8-3, Audit F1): then the caller has
      already resolved the stage through that config and passes tenant_configured=True.
    """
    if reward_type not in REWARD_MAP:
        raise ValueError(f"Unknown reward_type '{reward_type}'. Valid: {list(REWARD_MAP)}")

    if reward_type == "FORM_SUBMIT" and not settings.reward_on_form_submit and not tenant_configured:
        return {"arm_id": None, "tenant_id": None, "variant_id": None,
                "field_updated": None, "delta": 0.0, "skipped": True,
                "reason": "reward_on_form_submit=false (§16.1 B2B mode)"}

    field, delta = REWARD_MAP[reward_type]

    redis = get_redis()
    raw = await redis.get(f"decision:{decision_id}")
    dec = json.loads(raw) if raw else await _load_decision(decision_id)
    if not dec:
        raise ValueError(f"Decision not found: {decision_id}")
    if not dec.get("arm_id"):
        raise ValueError(f"Decision {decision_id} has no resolvable arm")

    # ML-5: when caller supplies session_id, verify it matches the decision —
    # prevents a browser script from replaying any decision_id it finds.
    if session_id is not None and dec.get("session_id") and dec["session_id"] != session_id:
        raise ValueError(f"session_id mismatch for decision {decision_id}")

    pool = await system_pool()
    now = datetime.now(timezone.utc)
    async with pool.acquire() as conn:
        async with conn.transaction():
            await conn.execute(_SET_TENANT, dec["tenant_id"])
            inserted = await conn.fetchval(
                """
                INSERT INTO thompson_reward_log (decision_id, reward_type, tenant_id, arm_id,
field_updated)
                VALUES ($1::uuid, $2, $3::uuid, $4::uuid, $5)
                ON CONFLICT DO NOTHING
                RETURNING 1
                """,
                decision_id, reward_type, dec["tenant_id"], dec["arm_id"], field,
            )
            if not inserted:
                return {"arm_id": dec["arm_id"], "tenant_id": dec["tenant_id"],
                        "variant_id": dec["variant_id"], "field_updated": None,
                        "delta": 0.0, "skipped": True,
                        "reason": f"{reward_type} already applied for this decision"}

            if field == "alpha":
                # ML-3: ONE alpha per decision (arch §4.4 "one Bernoulli observation").
                # If a prior alpha reward already exists for this decision, skip — don't
                # let CTA_CLICK + FORM_SUBMIT + CONVERSION each increment α.
                prior_alpha = await conn.fetchval(
                    "SELECT 1 FROM thompson_reward_log "
                    "WHERE decision_id = $1::uuid AND field_updated = 'alpha' "
                    "AND reward_type != $2 LIMIT 1",
                    decision_id, reward_type,
                )
                if prior_alpha:
                    # Roll back the just-inserted row and skip the arm update
                    await conn.execute(
```

```
                        "DELETE FROM thompson_reward_log "
                        "WHERE decision_id = $1::uuid AND reward_type = $2",
                        decision_id, reward_type,
                    )
                    return {"arm_id": dec["arm_id"], "tenant_id": dec["tenant_id"],
                            "variant_id": dec["variant_id"], "field_updated": None,
                            "delta": 0.0, "skipped": True,
                            "reason": "one alpha per decision (arch §4.4 Bernoulli model)"}

                reversed_sweep = await conn.fetchval(
                    "DELETE FROM thompson_reward_log WHERE decision_id = $1::uuid "
                    "AND reward_type = 'NO_RESPONSE_7D' AND field_updated = 'beta' RETURNING 1",
                    decision_id,
                )
                if reversed_sweep:
                    await conn.execute(
                        "UPDATE thompson_arms SET beta = GREATEST(1.0, beta - 1), "
                        "total_samples = GREATEST(0, total_samples - 1), updated_at = $1 "
                        "WHERE id = $2::uuid AND tenant_id = $3::uuid",
                        now, dec["arm_id"], dec["tenant_id"],
                    )

            sql = _UPDATE_ALPHA if field == "alpha" else _UPDATE_BETA
            await conn.execute(sql, delta, now, dec["arm_id"], dec["tenant_id"])

    context_key = dec.get("context_key", "global")
    await redis.delete(f"{CACHE_PREFIX}:{dec['tenant_id']}:{context_key}")

    return {
        "arm_id": dec["arm_id"],
        "tenant_id": dec["tenant_id"],
        "variant_id": dec["variant_id"],
        "field_updated": field,
        "delta": delta,
    }


async def sweep_no_response(
    window_days: int = NO_RESPONSE_WINDOW_DAYS,
    max_age_days: int = 30,
    limit: int = 1000,
    tenant_id: Optional[str] = None,
) -> dict:
    """Audit 2.3: beta += 1 for every decision older than the attribution window with no reward.

    Without this, beta never moves and Beta(alpha,beta) degenerates into a raw success count.
    Each swept decision is logged as NO_RESPONSE_7D, so the sweep is idempotent and a
    late positive reward can reverse it (see apply_reward).
    """
    pool = await system_pool()
    async with pool.acquire() as conn:
        rows = await conn.fetch(
            """
            SELECT dl.id::text AS id, dl.tenant_id::text AS tenant_id,
                   dl.chosen_variant_id::text AS variant_id, dl.candidates, dl.context
            FROM   decision_log dl
            WHERE  dl.decided_at < now() - ($1 * INTERVAL '1 day')
              AND  dl.decided_at > now() - ($2 * INTERVAL '1 day')
              AND  ($4::uuid IS NULL OR dl.tenant_id = $4::uuid)
              AND  NOT EXISTS (SELECT 1 FROM thompson_reward_log r WHERE r.decision_id = dl.id)
            ORDER BY dl.decided_at
            LIMIT  $3
            """,
            window_days, max_age_days, limit, tenant_id,
        )
```

```
        swept = 0
        unresolved = 0
        touched: set[tuple[str, str]] = set()
        now = datetime.now(timezone.utc)
        for row in rows:
            arm_id = arm_for_decision(row["variant_id"], row["candidates"])
            async with conn.transaction():
                await conn.execute(_SET_TENANT, row["tenant_id"])
                inserted = await conn.fetchval(
                    """
                    INSERT INTO thompson_reward_log (decision_id, reward_type, tenant_id, arm_id,
field_updated)
                    VALUES ($1::uuid, 'NO_RESPONSE_7D', $2::uuid, $3::uuid, $4)
                    ON CONFLICT DO NOTHING
                    RETURNING 1
                    """,
                    row["id"], row["tenant_id"], arm_id, "beta" if arm_id else "none",
                )
                if not inserted:
                    continue
                if not arm_id:
                    unresolved += 1
                    continue
                await conn.execute(_UPDATE_BETA, 1.0, now, arm_id, row["tenant_id"])
            swept += 1
            touched.add((row["tenant_id"], _as_dict(row["context"]).get("context_key", "global")))

    redis = get_redis()
    for t, ck in touched:
        await redis.delete(f"{CACHE_PREFIX}:{t}:{ck}")
    return {"swept": swept, "unresolved": unresolved, "scanned": len(rows),
            "window_days": window_days, "max_age_days": max_age_days}


async def apply_followup_outcome(tenant_id: str, appointment_id: str, outcome: str) -> dict:
    """Audit 2.10: the follow-up sequence bandit never learned - nothing wrote
followup_variants.alpha/beta.

    SHOWED -> alpha += 1, NO_SHOW -> beta += 1 on the assigned variant. Idempotent: the
    assignment row transitions from UNKNOWN/NULL exactly once.
    """
    if outcome not in ("SHOWED", "NO_SHOW"):
        raise ValueError("outcome must be SHOWED or NO_SHOW")
    field = "alpha" if outcome == "SHOWED" else "beta"
    pool = await tenant_pool(tenant_id)
    async with pool.acquire() as conn:
        async with conn.transaction():
            await conn.execute(_SET_TENANT, tenant_id)
            variant_id = await conn.fetchval(
                """
                UPDATE followup_assignments
                SET    outcome = $1, resolved_at = now()
                WHERE  appointment_id = $2::uuid AND tenant_id = $3::uuid
                  AND  (outcome IS NULL OR outcome = 'UNKNOWN')
                RETURNING variant_id::text
                """,
                outcome, appointment_id, tenant_id,
            )
            if not variant_id:
                return {"updated": False, "variant_id": None, "field_updated": None,
                        "reason": "assignment not found or already resolved"}
            await conn.execute(
                f"UPDATE followup_variants SET {field} = {field} + 1 WHERE id = $1::uuid AND tenant_id =
$2::uuid",
```

```
                variant_id, tenant_id,
            )
    return {"updated": True, "variant_id": variant_id, "field_updated": field}


async def get_benchmark(tenant_id: str, days: int = 30) -> dict:
    """Bayesian lift: P(system CVR > holdout CVR) over the last N days.

    Treatment = Thompson-selected sessions (is_holdout=false).
    Holdout   = randomly-selected sessions (is_holdout=true).
    Lift      = (treatment_cvr - holdout_cvr) / holdout_cvr * 100.
    P(better) = Monte Carlo integral over Beta posteriors (N=10 000).
    """
    pool = await tenant_pool(tenant_id)
    async with pool.acquire() as conn:
        counts = await conn.fetchrow(
            """
            SELECT
                COUNT(*) FILTER (WHERE NOT is_holdout) AS treatment_n,
                COUNT(*) FILTER (WHERE     is_holdout) AS holdout_n
            FROM decision_log
            WHERE tenant_id = $1::uuid
              AND decided_at > now() - ($2 * INTERVAL '1 day')
            """,
            tenant_id,
            days,
        )
        convs = await conn.fetchrow(
            """
            SELECT
                COUNT(DISTINCT dl.id) FILTER (WHERE NOT dl.is_holdout AND e.id IS NOT NULL)
                    AS treatment_conv,
                COUNT(DISTINCT dl.id) FILTER (WHERE dl.is_holdout AND e.id IS NOT NULL)
                    AS holdout_conv
            FROM decision_log dl
            LEFT JOIN events e
                ON  e.session_id  = dl.session_id
                AND e.tenant_id   = dl.tenant_id
                AND e.event_type  = 'FORM_SUBMIT'
                AND e.event_time  BETWEEN dl.decided_at
                                      AND dl.decided_at + INTERVAL '7 days'
            WHERE dl.tenant_id = $1::uuid
              AND dl.decided_at > now() - ($2 * INTERVAL '1 day')
            """,
            tenant_id,
            days,
        )

    treatment_n    = int(counts["treatment_n"]  or 0)
    holdout_n      = int(counts["holdout_n"]    or 0)
    treatment_conv = int(convs["treatment_conv"] or 0)
    holdout_conv   = int(convs["holdout_conv"]   or 0)

    treatment_cvr = treatment_conv / treatment_n if treatment_n else 0.0
    holdout_cvr   = holdout_conv   / holdout_n   if holdout_n   else 0.0
    lift_pct      = ((treatment_cvr - holdout_cvr) / holdout_cvr * 100) if holdout_cvr else None

    # Bayesian P(treatment > holdout) via Monte Carlo (Beta posteriors, N=10 000)
    t_a = 1 + treatment_conv
    t_b = 1 + (treatment_n - treatment_conv)
    h_a = 1 + holdout_conv
    h_b = 1 + (holdout_n   - holdout_conv)
    t_samples = np.random.beta(t_a, t_b, 10_000)
    h_samples = np.random.beta(h_a, h_b, 10_000)
    prob_better = float(np.mean(t_samples > h_samples))
```

```
    return {
        "tenant_id":      tenant_id,
        "period_days":    days,
        "treatment_n":    treatment_n,
        "treatment_conv": treatment_conv,
        "treatment_cvr":  round(treatment_cvr, 4),
        "holdout_n":      holdout_n,
        "holdout_conv":   holdout_conv,
        "holdout_cvr":    round(holdout_cvr, 4),
        "lift_pct":       round(lift_pct, 2) if lift_pct is not None else None,
        "prob_system_better": round(prob_better, 4),
        "sufficient_data": holdout_n >= 100 and treatment_n >= 100,
    }
```

1.3 `ml-service/app/services/followup_service.py`

Назначение: Thompson Sampling для выбора рукава follow-up последовательности после звонка.

Математическая логика:
- **Контекстуализация:** domestic/intl × high/low ticket → 4 контекстных сегмента.
- **Exploration rate:** 0.15 (15% вероятность случайного выбора вместо Thompson).
- **Авто-пауза:** ESS = α+β−2; n_simulations=2000; P(хуже) > 0.95 при ESS ≥ 50.
- **Show-rate posterior:** Beta posterior на данных о show/no-show; 90% CI via n=2000 семплов.

Входные данные: tenant_id, context (domestic/intl, high/low ticket), список рукавов.
Выходные данные: выбранный рукав follow-up, флаг паузы.

Взаимодействие: вызывается `FollowupController.java`; данные о show-rate берутся из noshow_scores.

Потенциальные риски:
- ESS = α+β−2 при информированных приорах (Beta(5,1)) даст ESS=4 сразу при создании → риск преждевременной паузы при ESS ≥ 50 фактически нейтрализован, но формально некорректно.

Полный код

```
"""
Follow-up sequence optimisation — Thompson Sampling bandit (vision §19.6).
No FastAPI/Pydantic imports — pure logic only.
"""
from __future__ import annotations

import random
from typing import Optional


# ── Context classification ────────────────────────────────────────────────

_INTL_COUNTRY_CODES = frozenset([
    "CA", "GB", "AU", "DE", "FR", "SG", "AE", "CH", "NL", "SE", "NO",
    "DK", "FI", "NZ", "HK", "JP", "KR", "IL",
])

_HIGH_TICKET_BUCKETS = frozenset(["50_100K", "100_250K", "GT250K"])


def classify_context(country_code: Optional[str], ticket_bucket: Optional[str]) -> str:
    """
    Returns one of four context keys for follow-up bandit partitioning:
      domestic:high_ticket  |  domestic:low_ticket
      intl:high_ticket      |  intl:low_ticket

    Matches arch §16.6 context keys: {intl/domestic} × {high/low ticket}.
    """
    geo = "intl" if (country_code or "").upper() in _INTL_COUNTRY_CODES else "domestic"
    ticket = "high_ticket" if (ticket_bucket or "") in _HIGH_TICKET_BUCKETS else "low_ticket"
    return f"{geo}:{ticket}"


# ── Thompson bandit ───────────────────────────────────────────────────────

def thompson_select_variant(
    variants: list[dict],
    context_key: str,
    exploration_rate: float = 0.15,
) -> dict:
    """
    Beta-Thompson sampling over followup_variants for a given context_key.

    Rules (vision §19.6):
    - Control arm is NEVER paused; always included.
    - exploration_rate (10-20%) forces random choice; otherwise Beta sampling.
    - Only APPROVED + is_control variants are eligible; DRAFT/PAUSED excluded.

    Returns the selected variant dict (must include id, name, alpha, beta, is_control).
    Raises ValueError if no eligible variant exists.
    """
    eligible = [
        v for v in variants
        if v.get("lifecycle_state") == "APPROVED" or v.get("is_control", False)
    ]
    if not eligible:
        raise ValueError(f"No eligible followup variants for context_key={context_key}")

    if random.random() < exploration_rate:
        return random.choice(eligible)

    best = max(eligible, key=lambda v: random.betavariate(
        max(v.get("alpha", 1.0), 0.001),
        max(v.get("beta", 1.0), 0.001),
```

```
    ))
    return best


# ── Auto-pause guard ──────────────────────────────────────────────

_N_SIMULATIONS = 2000


def should_auto_pause(
    variant_alpha: float,
    variant_beta: float,
    control_alpha: float,
    control_beta: float,
    prior_alpha: float = 1.0,
    prior_beta: float = 1.0,
    min_ess: int = 100,
) -> tuple[bool, float]:
    """
    Monte Carlo estimate of P(variant is WORSE than control).
    Auto-pause if P(worse) > 0.95 AND ESS >= min_ess.

    ESS approximation: α + β - prior_alpha - prior_beta (informed priors supported).
    Returns (should_pause, p_worse).
    """
    ess = (variant_alpha + variant_beta - prior_alpha - prior_beta)
    if ess < min_ess:
        return False, 0.0

    worse_count = 0
    for _ in range(_N_SIMULATIONS):
        v_sample = random.betavariate(
            max(variant_alpha, 0.001), max(variant_beta, 0.001))
        c_sample = random.betavariate(
            max(control_alpha, 0.001), max(control_beta, 0.001))
        if v_sample < c_sample:
            worse_count += 1

    p_worse = worse_count / _N_SIMULATIONS
    return p_worse > 0.95, round(p_worse, 4)


# ── Show-rate posterior ──────────────────────────────────────────

def compute_show_rate_posterior(
    showed: int,
    total: int,
    prior_alpha: float = 1.0,
    prior_beta: float = 1.0,
) -> dict:
    """
    Beta posterior for show-rate given binary outcome counts.
    Returns mean, credible interval, and raw posterior params.
    """
    if total < 0 or showed < 0 or showed > total:
        raise ValueError("Invalid showed/total counts")

    alpha_post = prior_alpha + showed
    beta_post = prior_beta + (total - showed)
    mean = alpha_post / (alpha_post + beta_post)

    # 90% credible interval via random.betavariate sampling
    samples = sorted(
        random.betavariate(max(alpha_post, 0.001), max(beta_post, 0.001))
        for _ in range(2000)
```

```
    )
    lo = samples[int(0.05 * 2000)]
    hi = samples[int(0.95 * 2000)]

    return {
        "alpha": alpha_post,
        "beta": beta_post,
        "mean": round(mean, 4),
        "ci_90_lo": round(lo, 4),
        "ci_90_hi": round(hi, 4),
        "ess": showed + (total - showed),
    }
```

1.4 `ml-service/app/services/funnel_path_service.py`

Назначение: Thompson Sampling для выбора пути воронки (Funnel Path A/B).

Математическая логика:
- **TIERED override:** если lead_score ≥ high_tier_threshold → принудительный путь B независимо от Thompson.
- **Gate B:** MATERIAL_VIEW + DATAROOM_OPEN/VIDEO_75 — дополнительные условия для входа в путь B.
- **Метрика качества:** held_calls / 1000 LPV с 80% доверительным интервалом; z=1.282.

Входные данные: tenant_id, session_id, lead_score.
Выходные данные: выбранный funnel_path_id, вариант лендинга.

Взаимодействие: вызывается из `FunnelPathController.java`; использует thompson_arms для пути.

Полный код

```
"""
Funnel Path Thompson Sampling (vision §19.18.1; arch §16.14).
Pure logic — zero FastAPI/Pydantic imports.

Paths are Thompson arms, rewarded on CALL_HELD normalised per 1000 PAGE_VIEW.
Single writer rule: FastAPI is sole writer of alpha/beta.
"""
from __future__ import annotations
import random
import math


def select_path(
    paths: list[dict],
    ticket_bucket: str | None = None,
) -> dict | None:
    """
    Thompson Sampling over funnel paths.
    TIERED_BY_TICKET overrides when ticket_bucket is known.
    """
    active = [p for p in paths if p.get("status") == "ACTIVE"]
    if not active:
        return None

    tiered = next((p for p in active if p.get("path_key") == "TIERED_BY_TICKET"), None)
    if tiered and ticket_bucket:
        return tiered

    samples = {
        p["path_id"]: random.betavariate(
            max(float(p.get("alpha", 1.0)), 1.0),
            max(float(p.get("beta",  1.0)), 1.0),
        )
        for p in active
        if p.get("path_key") != "TIERED_BY_TICKET"
    }
    if not samples:
        return active[0]

    best_id = max(samples, key=lambda k: samples[k])
    return next(p for p in active if p["path_id"] == best_id)


def check_path_b_engagement_gate(
    events: list[dict],
    gate_threshold: dict | None = None,
) -> dict:
    """
    Path B (MATERIALS_FIRST) engagement gate.
    Pass: MATERIAL_VIEW + (DATAROOM_OPEN OR VIDEO_PROGRESS >= 75).
    """
    defaults = {
        "require_material_view": True,
        "require_dataroom_or_video": True,
        "video_progress_threshold": 75,
    }
    gate = gate_threshold or defaults

    event_types = {e.get("event_type") for e in events}
    has_material_view = "MATERIAL_VIEW" in event_types
    has_dataroom      = "DATAROOM_OPEN" in event_types
    has_video_75      = any(
        e.get("event_type") == "VIDEO_PROGRESS" and
        int(e.get("payload", {}).get("progress_pct")
```

```
            or e.get("payload", {}).get("progress")
            or e.get("payload", {}).get("percent")
            or 0) >= gate["video_progress_threshold"]
        for e in events
    )

    material_ok   = has_material_view if gate["require_material_view"]         else True
    engagement_ok = (has_dataroom or has_video_75) if gate["require_dataroom_or_video"] else True
    gate_passed   = material_ok and engagement_ok

    return {
        "gate_passed":       gate_passed,
        "has_material_view": has_material_view,
        "has_dataroom":      has_dataroom,
        "has_video_75":      has_video_75,
        "missing": [
            k for k, v in {
                "MATERIAL_VIEW":        has_material_view,
                "DATAROOM_OR_VIDEO_75": (has_dataroom or has_video_75),
            }.items() if not v
        ],
    }


def resolve_tiered_path_variant(
    ticket_bucket: str,
    tiered_config: dict,
) -> dict:
    """Path C (TIERED_BY_TICKET): sub-variant by ticket bucket."""
    if ticket_bucket in ("GT250K", "100_250K"):
        return tiered_config.get("GTE250K", {"ghl_tag": "path_c_founder_call", "variant_key":
"1on1_founder"})
    elif ticket_bucket in ("50_100K", "25_50K"):
        return tiered_config.get("GTE50K",  {"ghl_tag": "path_c_1on1_call",    "variant_key":
"1on1_standard"})
    else:
        return tiered_config.get("LT50K",   {"ghl_tag": "path_c_webinar",      "variant_key":
"webinar"})


def compute_path_metrics(
    reward_events: list[dict],
    page_views: dict,
    per_1000_lpv: bool = True,
) -> list[dict]:
    """CALL_HELD per 1000 LPV per path with 80% credible interval."""
    from collections import defaultdict
    path_held: dict = defaultdict(int)

    for ev in reward_events:
        pid = ev.get("path_id") or "null"
        if ev.get("is_quality_stage"):
            path_held[pid] += 1

    results = []
    for path_id, lpv in page_views.items():
        held  = path_held.get(path_id, 0)
        rate  = (held / max(lpv, 1)) * (1000 if per_1000_lpv else 1)
        alpha = held + 1
        beta  = (lpv - held) + 1
        mean  = alpha / (alpha + beta)
        z     = 1.282
        se    = math.sqrt(mean * (1 - mean) / max(lpv, 1))
        results.append({
            "path_id":         path_id,
```

```
            "held_calls":      held,
            "page_views":      lpv,
            "held_per_1k_lpv": round(rate, 2),
            "posterior_mean":  round(mean, 4),
            "ci_80_lower":     round(max(0.0, mean - z * se), 4),
            "ci_80_upper":     round(min(1.0, mean + z * se), 4),
        })
    results.sort(key=lambda x: -x["held_per_1k_lpv"])
    return results
```

СЕКЦИЯ 2 — Предиктивные модели

2.1 `ml-service/app/services/noshow_service.py`

Назначение: Предсказание вероятности no-show для каждого бронирования.

Математическая логика:
- **Rule-based v1:** базовый rate = 0.30; корректировки по признакам:
- booking_distance_h > 72ч → +0.10
- timezone: EST/PST/CST → +0; другие → +0.05
- ticket_bucket: LT25K → +0.15; GT250K → −0.10
- accredited = False → +0.10
- device = mobile → +0.05
- is_international → +0.08
- Сумма, ограничение [0.05, 0.95]
- **Tier:** HIGH ≥ 0.5, MED ≥ 0.3, LOW < 0.3.
- **Заполнение слота:** P(никто не придёт) = П p_noshow по всем бронированиям в слоте.

Входные данные: детали бронирования (время, тикет, аккредитация, устройство, таймзона).
Выходные данные: {risk_score, risk_tier} → сохраняется в noshow_scores.

Взаимодействие: результат используется `lead_value_service.py` (p_show = 1 − risk_score), `followup_service.py`, `BookingWindowService.java`.

Потенциальные риски:
- Произвольные пороги (+0.10, +0.05 и т.д.) без эмпирической калибровки. До накопления ≥200 реализованных лидов — только экспертная оценка.
- P(никто не придёт) = произведение — предполагает независимость бронирований в слоте, что может быть нереалистично (групповые бронирования).

Полный код

```
"""
No-show Propensity Model (vision §19.18.2; arch §16.15).
Pure logic — zero FastAPI/Pydantic imports.

Rule-based model until holdout AUC >= 0.70.
"""
from __future__ import annotations

BOOKING_DISTANCE_COEF = 0.012
# Seed constants — no empirical basis yet (Phase 12-13).
# Replace with fitted LogisticRegression coefficients once ≥200 labelled leads available.
# Source: expert judgment from noshow literature (Gupta et al. 2018, B2B webinar context).
TZ_UNKNOWN_PENALTY    = 0.10
INTERNATIONAL_PENALTY = 0.08
LOW_TICKET_PENALTY    = 0.06
UNACCREDITED_PENALTY  = 0.05
MOBILE_PENALTY        = 0.03
BASE_NOSHOW_RATE      = 0.30


def predict_noshow(
    booking_distance_h: float | None,
    lead_tz_known:       bool,
    ticket_bucket:       str | None,
    self_accredited:     bool | None,
    device_type:         str | None,
    is_international:    bool,
    model_version:       str = "rule_based_v1",
) -> dict:
    """Rule-based no-show probability, clipped [0.05, 0.90]."""
    score = BASE_NOSHOW_RATE

    if booking_distance_h is not None:
        if booking_distance_h < 2:
            score += 0.08
        else:
            score += min(BOOKING_DISTANCE_COEF * booking_distance_h, 0.20)

    if not lead_tz_known:
        score += TZ_UNKNOWN_PENALTY
    if is_international:
        score += INTERNATIONAL_PENALTY
    if ticket_bucket in ("LT25K", None):
        score += LOW_TICKET_PENALTY
    if self_accredited is False:
        score += UNACCREDITED_PENALTY
    if device_type == "MOBILE":
        score += MOBILE_PENALTY

    noshow_prob = max(0.05, min(0.90, score))
    risk_tier   = "HIGH" if noshow_prob >= 0.45 else ("MED" if noshow_prob >= 0.32 else "LOW")

    return {
        "noshow_prob":   round(noshow_prob, 4),
        "risk_tier":     risk_tier,
        "model_version": model_version,
        "auc_note":      "Rule-based; ML activates when holdout AUC >= 0.70",
        "features_used": {
            "booking_distance_h": booking_distance_h,
            "lead_tz_known":      lead_tz_known,
            "ticket_bucket":      ticket_bucket,
            "self_accredited":    self_accredited,
            "device_type":        device_type,
            "is_international":   is_international,
```

```
        },
    }


def get_followup_audience_override(risk_tier: str) -> dict:
    """HIGH-risk: coordinator call ≤ 15 min + nearer slot."""
    overrides = {
        "HIGH": {
            "audience_tag":       "HIGH_RISK_NOSHOW",
            "ghl_tag":            "high_risk_noshow",
            "max_touches":        6,
            "cadence_note":       "Coordinator call within 15 min + offer nearer slot",
            "preferred_channels": ["CALL", "SMS"],
        },
        "MED": {
            "audience_tag":       "MED_RISK_NOSHOW",
            "ghl_tag":            "med_risk_noshow",
            "max_touches":        4,
            "preferred_channels": ["SMS", "EMAIL"],
        },
        "LOW": {
            "audience_tag":       None,
            "ghl_tag":            None,
            "max_touches":        3,
            "preferred_channels": ["EMAIL"],
        },
    }
    return overrides.get(risk_tier, overrides["MED"])


def compute_slot_fill_probability(
    booked_count: int,
    noshow_probs: list[float],
) -> dict:
    """P(at least one shows) = 1 − Π(noshow_prob_i)."""
    if not noshow_probs:
        return {"p_no_one_shows": 0.0, "p_at_least_one_shows": 1.0, "booked": booked_count}

    # Independence assumption (Phase 12-14): each booking's no-show probability is treated
    # as independent. Valid for solo bookings; underestimates P(no_one) for paired/group slots
    # where correlated no-show behaviour is expected. Track group_booking flag when available.
    p_no_one = 1.0
    for p in noshow_probs:
        p_no_one *= p

    return {
        "p_no_one_shows":       round(p_no_one, 4),
        "p_at_least_one_shows": round(1 - p_no_one, 4),
        "booked":               booked_count,
        "expected_attendees":   round(sum(1 - p for p in noshow_probs), 2),
    }
```

2.2 `ml-service/app/services/lead_value_service.py`

Назначение: Предсказание ожидаемой ценности лида (Predicted Lead Value, PLV).

Математическая логика:

```
PLV = p_show × p_verified|show × p_invest|verified × E[ticket|bucket]
```

- **p_show** = 1 − noshow_risk_score из noshow_scores

- **p_invest** = seed 0.21 (до 200 реализованных лидов)
- **p_verified** = seed 0.65 (до 200 реализованных лидов)
- **E[ticket|bucket]:** LT25K=0.3, 25_50K=0.6, 50_100K=1.0 (base=50K), 100_250K=1.8, GT250K=3.0 × k_ticket
- **OLS калибровка:** на ≥200 реализованных лидов; slope принимается если ∈[0.8, 1.2]. При отклонении slope остаётся 1.0.

Входные данные: noshow_scores.risk_score, ticket_bucket, аккредитация.
Выходные данные: predicted_value_usd → сохраняется в lead_value_predictions.

Взаимодействие: вызывается `LeadValueService.java` на событие BOOKED; realised value записывается при INVEST_COMMITTED.

Потенциальные риски:
- Произведение четырёх вероятностей — ошибка в любом компоненте умножается. При p_show=0.7, p_verified=0.65, p_invest=0.21 → PLV = 0.70×0.65×0.21×k ≈ 9.5% от k_ticket.
- OLS slope [0.8,1.2] — хороший диапазон, но если истинный slope = 1.5 (систематическое занижение), калибровка будет отклонена и модель выдаёт неверные значения бесконечно.

Полный код

```
"""
Predicted Lead Value at Day 0 (vision §19.18.3; arch §16.16).
Pure logic — zero FastAPI/Pydantic imports.

Formula: p_show × p_verified|show × p_invest|verified × E[ticket|bucket]
Acceptance: calibration slope 0.8–1.2 on >=200 realised leads.
"""
from __future__ import annotations
import logging
from collections import defaultdict

_logger = logging.getLogger(__name__)

# ── Phase 12-4: PLV slope auto-recalibration state ──────────────────────────
# Counts consecutive out-of-range OLS slope results; resets on an in-range slope.
_ols_out_of_range_streak: int = 0
# Dynamically adjusted seed coefficient (soft correction when slope drifts).
_seed_p_invest_adjusted: float | None = None  # None = use SEED_P_INVEST_GIVEN_VERIFIED

E_TICKET = {
    "LT25K":    12_500,
    "25_50K":   37_500,
    "50_100K":  75_000,
    "100_250K": 175_000,
    "GT250K":   350_000,
}
DEFAULT_E_TICKET              = 75_000
SEED_P_INVEST_GIVEN_VERIFIED  = 0.21
SEED_P_VERIFIED_GIVEN_SHOW    = 0.65


def predict_lead_value(
    p_show:                  float,
    ticket_bucket:           str | None,
    p_verified_given_show:   float = SEED_P_VERIFIED_GIVEN_SHOW,
    p_invest_given_verified: float = SEED_P_INVEST_GIVEN_VERIFIED,
    e_ticket_override:       float | None = None,
    model_version:           str = "seed_v1",
) -> dict:
    """Day-0 predicted lead value."""
    e_ticket  = e_ticket_override or E_TICKET.get(ticket_bucket or "", DEFAULT_E_TICKET)
    predicted = p_show * p_verified_given_show * p_invest_given_verified * e_ticket

    return {
        "predicted_value": round(predicted, 2),
        "components": {
            "p_show":                  round(p_show, 4),
            "p_verified_given_show":   round(p_verified_given_show, 4),
            "p_invest_given_verified": round(p_invest_given_verified, 4),
            "e_ticket":                e_ticket,
            "ticket_bucket":           ticket_bucket,
        },
        "model_version":    model_version,
        "calibration_note": "Seed model — live when slope 0.8–1.2 on >=200 realised leads",
    }


def calibrate_value_model(
    predictions: list[dict],
    min_samples: int = 200,
) -> dict:
    """OLS calibration: predicted vs realised. Acceptance: slope ∈ [0.8, 1.2]."""
    valid = [
        p for p in predictions
```

```
        if p.get("predicted_value") is not None and p.get("realised_value") is not None
        and p["predicted_value"] > 0
    ]

    if len(valid) < min_samples:
        return {
            "calibrated": False,
            "n":          len(valid),
            "reason":     f"Need {min_samples} realised leads, have {len(valid)}",
            "slope":      None,
        }

    n      = len(valid)
    x_vals = [v["predicted_value"] for v in valid]
    y_vals = [v["realised_value"]   for v in valid]
    x_mean = sum(x_vals) / n
    y_mean = sum(y_vals) / n
    cov    = sum((x - x_mean) * (y - y_mean) for x, y in zip(x_vals, y_vals))
    var    = sum((x - x_mean) ** 2 for x in x_vals)
    slope  = cov / var if var > 0 else 0.0
    intercept = y_mean - slope * x_mean

    by_bucket: dict = defaultdict(lambda: {"n": 0, "sum_pred": 0.0, "sum_real": 0.0})
    for v in valid:
        bk = v.get("ticket_bucket", "unknown")
        by_bucket[bk]["n"]        += 1
        by_bucket[bk]["sum_pred"] += v["predicted_value"]
        by_bucket[bk]["sum_real"] += v["realised_value"]

    bucket_stats = {
        bk: {
            "n":         d["n"],
            "mean_pred": round(d["sum_pred"] / d["n"], 2),
            "mean_real": round(d["sum_real"] / d["n"], 2),
            "ratio":     round(d["sum_real"] / max(d["sum_pred"], 1), 3),
        }
        for bk, d in by_bucket.items()
    }

    calibrated = 0.8 <= slope <= 1.2

    # ── Phase 12-4: auto-recalibration ──────────────────────────────────
    global _ols_out_of_range_streak, _seed_p_invest_adjusted

    if not calibrated:
        _ols_out_of_range_streak += 1
        _logger.warning(
            "PLV calibration slope=%.3f outside [0.8,1.2] — adjusting seed coefficients",
            slope,
        )
        base = SEED_P_INVEST_GIVEN_VERIFIED
        if slope > 1.2:
            _seed_p_invest_adjusted = base * min(slope / 1.0, 1.3)
        else:  # slope < 0.8
            _seed_p_invest_adjusted = base * max(slope / 1.0, 0.7)

        if _ols_out_of_range_streak >= 3:
            _logger.error(
                "ALERT PLV calibration slope out of range %d consecutive times",
                _ols_out_of_range_streak,
            )
    else:
        _ols_out_of_range_streak = 0

    return {
```

```
        "calibrated":              calibrated,
        "n":                       n,
        "slope":                   round(slope, 4),
        "intercept":               round(intercept, 2),
        "by_bucket":               bucket_stats,
        "acceptance":              "slope ∈ [0.8, 1.2]",
        "adjusted_p_invest":       round(_seed_p_invest_adjusted or SEED_P_INVEST_GIVEN_VERIFIED, 6),
        "ols_out_of_range_streak": _ols_out_of_range_streak,
    }
```

2.3 `ml-service/app/services/reward_service.py`

Назначение: Вычисление наград за прохождение стадий воронки + Show-Rate Corrector.

Математическая логика (основная часть):
- Thompson increment = 1.0 ТОЛЬКО для quality gate stage (CALL_HELD). Дробные дельты запрещены.
- Funnel progress score = deepest_ordinal / max_ordinal ∈ [0,1] — для аналитики, НЕ для Thompson.
- Quality rate posterior: Beta(quality_hits+1, noshow+1); 80% CI via z=1.282.

Show-Rate Corrector (vision §19.10):

```
post_α = prior_α + held_count
post_β = prior_β + (booked - held)
post_mean = post_α / (post_α + post_β)
blended = (1 - 0.3) × post_mean + 0.3 × prior_mean
multiplier = blended / prior_mean ∈ [0.5, 1.5]
```

- prior_α=3.0, prior_β=2.0 → prior_mean = 0.6 (60% ожидаемый show rate)
- multiplier < 1.0 → плохой show rate → ужесточение L1/L2 gate порогов (threshold / multiplier)

Взаимодействие: `ShowRateCorrectorScheduler.java` запускает еженедельно (пн 06:00); результат пишется в tenants.settings.show_rate_multiplier.

Потенциальные риски:
- Shrinkage = 0.3 фиксирован. При малой выборке (booked_count < 10) posterior почти равен приору — корректор не чувствителен к реальным данным.
- apply_show_rate_to_gates: threshold / multiplier при multiplier=0.5 удваивает порог. Если L2=0.9 → после коррекции L2=1.8, что никогда не достигается → все лиды блокируются. Граница [0.5, 1.5] ограничивает умножение, но граница порогов [0,1] не проверяется.

Полный код

```
"""
Reward Ladder for landing variant Thompson Sampling (vision §19.3).
Pure logic — zero FastAPI/Pydantic imports.

CRITICAL: thompson_arms uses BINARY increments (1.0 only).
reward_weight in funnel_stages is for ANALYTICS CALIBRATION only.
The quality gate stage triggers the actual thompson_arms reward.
"""
from __future__ import annotations
import math


def compute_stage_reward(
    stage_key: str,
    is_quality_stage: bool,
    funnel_stages: list[dict],
) -> dict:
    """
    Returns thompson_increment (0.0 or 1.0), micro_reward, should_update_thompson.
    thompson_increment is 1.0 ONLY when stage is the quality gate stage.
    """
    stage = next((s for s in funnel_stages if s["stage_key"] == stage_key), None)
    if stage is None:
        return {
            "thompson_increment":     0.0,
            "micro_reward":           0.0,
            "should_update_thompson": False,
        }

    is_quality = stage.get("is_quality_key", False) or is_quality_stage
    micro      = float(stage.get("micro_reward_weight", 0.0))

    return {
        "thompson_increment":     1.0 if is_quality else 0.0,
        "micro_reward":           micro,
        "should_update_thompson": is_quality,
    }


def compute_funnel_progress_score(
    reached_stages: list[str],
    funnel_stages: list[dict],
) -> dict:
    """
    Normalized progress score [0,1] — how far a session got through the funnel.
    NOT used for thompson_arms.
    """
    if not funnel_stages:
        return {"score": 0.0, "deepest_stage": None, "stages_reached": 0}

    ordered     = sorted(funnel_stages, key=lambda s: s.get("ordinal", 0))
    max_ordinal = ordered[-1].get("ordinal", 1)

    deepest_ordinal = 0
    deepest_key     = None
    for stage in ordered:
        if stage["stage_key"] in reached_stages:
            if stage.get("ordinal", 0) > deepest_ordinal:
                deepest_ordinal = stage.get("ordinal", 0)
                deepest_key     = stage["stage_key"]

    score = deepest_ordinal / max(max_ordinal, 1)
    return {
        "score":           round(score, 4),
```

```
        "deepest_stage":   deepest_key,
        "stages_reached":  len(reached_stages),
        "max_ordinal":     max_ordinal,
        "deepest_ordinal": deepest_ordinal,
    }


def estimate_quality_rate(
    reward_events: list[dict],
    variant_id: str,
    days: int = 30,
) -> dict:
    """
    Beta(quality_hits+1, noshow+1) posterior for show-rate of a landing variant.
    """
    from datetime import datetime, timezone, timedelta
    cutoff = datetime.now(timezone.utc) - timedelta(days=days)

    recent = []
    for e in reward_events:
        if e.get("variant_id") != variant_id:
            continue
        rat = e.get("rewarded_at")
        if rat is None:
            recent.append(e)
        else:
            try:
                dt = datetime.fromisoformat(str(rat).replace("Z", "+00:00"))
                if dt >= cutoff:
                    recent.append(e)
            except Exception:
                recent.append(e)

    quality_hits = sum(1 for e in recent if e.get("is_quality_stage"))
    total        = len(recent)
    alpha        = quality_hits + 1
    beta         = (total - quality_hits) + 1
    mean         = alpha / (alpha + beta)
    z            = 1.282
    se           = math.sqrt(mean * (1 - mean) / max(total, 1))

    return {
        "variant_id":   variant_id,
        "quality_hits": quality_hits,
        "total":        total,
        "alpha":        alpha,
        "beta":         beta,
        "mean":         round(mean, 4),
        "ci_80_lower":  round(max(0.0, mean - z * se), 4),
        "ci_80_upper":  round(min(1.0, mean + z * se), 4),
    }


# ── Show-Rate Corrector (roadmap 8-3; vision §19.10) ──────────────────

def compute_show_rate_corrector(
    booked_count:     int,
    held_count:       int,
    prior_alpha:      float = 3.0,
    prior_beta:       float = 2.0,
    shrinkage_weight: float = 0.3,
) -> dict:
    """
    Weekly show-rate corrector with Beta shrinkage (roadmap 8-3; vision §19.10).
    Returns multiplier for L1/L2 quality gate thresholds.
```

```
    multiplier < 1.0 → penalty (high no-show rate)
    multiplier = 1.0 → neutral (at prior expectation)
    multiplier > 1.0 → bonus (better show rate than prior)
    Capped [0.5, 1.5] — never halve or more-than-double the gate.
    """
    if booked_count <= 0:
        return {
            "multiplier":     1.0,
            "show_rate_obs":  None,
            "show_rate_post": round(prior_alpha / (prior_alpha + prior_beta), 4),
            "note":           "No bookings — using pure prior",
        }

    post_alpha = prior_alpha + held_count
    post_beta  = prior_beta  + (booked_count - held_count)
    post_mean  = post_alpha / (post_alpha + post_beta)
    obs_rate   = held_count / booked_count
    prior_mean = prior_alpha / (prior_alpha + prior_beta)

    blended    = (1 - shrinkage_weight) * post_mean + shrinkage_weight * prior_mean
    multiplier = blended / prior_mean if prior_mean > 0 else 1.0
    multiplier = max(0.5, min(1.5, multiplier))

    return {
        "multiplier":      round(multiplier, 4),
        "show_rate_obs":   round(obs_rate, 4),
        "show_rate_post":  round(post_mean, 4),
        "show_rate_blend": round(blended, 4),
        "booked":          booked_count,
        "held":            held_count,
        "post_alpha":      round(post_alpha, 2),
        "post_beta":       round(post_beta, 2),
        "prior_mean":      round(prior_mean, 4),
    }


def apply_show_rate_to_gates(
    l1_threshold: float,
    l2_threshold: float,
    multiplier: float,
) -> dict:
    """
    Gate threshold adjustment: bad show rate tightens gates (higher threshold needed).
    """
    adjusted_l1 = l1_threshold / multiplier if multiplier > 0 else l1_threshold
    adjusted_l2 = l2_threshold / multiplier if multiplier > 0 else l2_threshold
    return {
        "l1_original":  l1_threshold,
        "l2_original":  l2_threshold,
        "l1_adjusted":  round(adjusted_l1, 4),
        "l2_adjusted":  round(adjusted_l2, 4),
        "multiplier":   round(multiplier, 4),
    }
```

2.4 `ml-service/app/services/propensity_service.py`

Назначение: Оценка P(конверсии) для сессии — правило-базированный фолбек + Gradient Boosting.

Математическая логика:
- **Rule-based:** base=0.05; +0.50 если form_submitted; +0.20 если form_started; +0.10 repeat visit; +0.10 cta_time>5s; −0.05 short session; ограничение [0,1].

- **Gradient Boosting:** sklearn Pipeline(StandardScaler, GradientBoostingClassifier); n_estimators=200, max_depth=4, learning_rate=0.05, random_state=42.
- **Uplift:** p_convert_personalised − p_convert_control ∈ [−1, 1].
- **Objection:** определяется по порогам времени на блоках (price>20s, proof>25s, session<15s).

Входные данные: feature vector из cluster_service (11 признаков) + cluster_id.
Выходные данные: {p_convert, main_objection}.

Взаимодействие: использует признаки из `cluster_service.py`; вызывается `decide_service.py`.

Потенциальные риски:
- Model deserialisation via joblib из базы данных — риск безопасности (pickle injection) при ненадёжном источнике.
- Минимум 30 сессий для обучения — недостаточно для GBM. Рекомендуется ≥1000.

Полный код

```
"""
Propensity Model Service (Phase 5-2, §7.3 Этап 2).

Pure-logic functions: feature extraction, gradient boosting scoring, objection detection.
No FastAPI imports — safe to unit-test without Pydantic.

Model:
  - GradientBoostingClassifier — P(conversion)
  - Threshold-based objection: price | trust | urgency | understanding

Model serialisation (Phase 12-1):
  - Preferred: ONNX bytes (no arbitrary code execution on load)
    Format: b"ONNX\\x00" + onnx_protobuf_bytes
  - Fallback: SHA-256 hash-validated joblib pickle
    Format: b"SAFE\\x00" + 64-byte hex hash + b"\\x00" + pickle_bytes
  - Legacy (read-only, WARN on load): raw pickle starting with b"\\x80"
"""
import logging
from typing import Any

import numpy as np

_log = logging.getLogger(__name__)


# ─────────────────────────────────────────────────────────────────────────────
# Feature set for propensity model
# ─────────────────────────────────────────────────────────────────────────────

PROPENSITY_FEATURE_NAMES = [
    "session_duration_sec",
    "scroll_depth_max",
    "block_price_time",
    "block_proof_time",
    "block_cta_time",
    "form_started",
    "form_submitted",
    "is_mobile",
    "is_repeat_visit",
    "event_count",
    "cluster_id",            # 0-based cluster index (-1 = unknown → 0)
    "scroll_velocity",       # from cluster_service features
]
N_PROPENSITY_FEATURES = len(PROPENSITY_FEATURE_NAMES)

# Objection thresholds (rule-based, used without trained model)
OBJECTION_THRESHOLDS = {
    "price":         ("block_price_time",   20.0),   # >20 s on pricing = price concern
    "trust":         ("block_proof_time",   25.0),   # >25 s on proof = trust concern
    "urgency":       ("session_duration_sec", 15.0), # very short = urgency issue (bounce risk)
    "understanding": ("event_count",         3.0),   # very few events = confused
}


# ─────────────────────────────────────────────────────────────────────────────
# Feature extraction
# ─────────────────────────────────────────────────────────────────────────────

def extract_propensity_features(
    cluster_features: np.ndarray,       # from cluster_service.extract_features()
    cluster_id: int = -1,
) -> np.ndarray:
    """
    Build propensity feature vector by extending cluster features with cluster_id.
```

```
    Args:
        cluster_features: (N_CLUSTER_FEATURES,) array from cluster_service
        cluster_id: assigned cluster index (-1 if not assigned)

    Returns:
        np.ndarray of shape (N_PROPENSITY_FEATURES,) — float32
    """
    from app.services.cluster_service import FEATURE_NAMES

    cluster_feat_map = dict(zip(FEATURE_NAMES, cluster_features.tolist()))

    feat = [
        cluster_feat_map.get("session_duration_sec", 0.0),
        cluster_feat_map.get("scroll_depth_max",     0.0),
        cluster_feat_map.get("block_price_time",     0.0),
        cluster_feat_map.get("block_proof_time",     0.0),
        cluster_feat_map.get("block_cta_time",       0.0),
        cluster_feat_map.get("form_started",         0.0),
        cluster_feat_map.get("form_submitted",       0.0),
        cluster_feat_map.get("is_mobile",            0.0),
        cluster_feat_map.get("is_repeat_visit",      0.0),
        cluster_feat_map.get("event_count",          0.0),
        float(max(cluster_id, 0)),                        # -1 → 0
        cluster_feat_map.get("scroll_velocity",      0.0),
    ]

    return np.array(feat, dtype=np.float32)


# ─────────────────────────────────────────────────────────────────────────────
# Rule-based propensity score (fallback without trained model)
# ─────────────────────────────────────────────────────────────────────────────

def rule_based_score(features: np.ndarray) -> float:
    """
    Estimate P(conversion) from simple rules. Returns float in [0, 1].

    Used as fallback before a trained model exists.
    """
    f = dict(zip(PROPENSITY_FEATURE_NAMES, features.tolist()))

    score = 0.05  # base rate

    # Strong positive signals
    if f["form_submitted"] > 0:
        score += 0.50
    elif f["form_started"] > 0:
        score += 0.20

    if f["is_repeat_visit"] > 0:
        score += 0.10

    if f["block_cta_time"] > 5:
        score += 0.10

    # Negative signals
    dur = f["session_duration_sec"]
    if dur < 10:
        score -= 0.05
    if f["scroll_depth_max"] < 0.1:
        score -= 0.05

    return float(min(max(score, 0.0), 1.0))
```

```
# ─────────────────────────────────────────────────────────────────────────────
# Objection detection (rule-based, no LLM required in tests)
# ─────────────────────────────────────────────────────────────────────────────

def determine_objection(features: np.ndarray) -> str | None:
    """
    Identify primary objection from session behaviour.

    Returns one of: 'price', 'trust', 'urgency', 'understanding', or None.
    """
    f = dict(zip(PROPENSITY_FEATURE_NAMES, features.tolist()))

    # Understanding: very low event count means the visitor is confused
    if f["event_count"] <= OBJECTION_THRESHOLDS["understanding"][1]:
        return "understanding"

    # Urgency: extremely short session — didn't engage at all
    if f["session_duration_sec"] < 15 and f["form_started"] == 0:
        return "urgency"

    # Price vs trust — pick whichever has more time spent
    price_t = f["block_price_time"]
    proof_t = f["block_proof_time"]

    price_threshold = OBJECTION_THRESHOLDS["price"][1]
    trust_threshold = OBJECTION_THRESHOLDS["trust"][1]

    if price_t > price_threshold and price_t >= proof_t:
        return "price"
    if proof_t > trust_threshold and proof_t > price_t:
        return "trust"

    return None


# ─────────────────────────────────────────────────────────────────────────────
# Internal: safe model loading (Phase 12-1)
# ─────────────────────────────────────────────────────────────────────────────

def _load_and_score(model_bytes: bytes, features: np.ndarray) -> float:
    """
    Deserialise model bytes and return P(convert) for a single feature vector.

    Supported formats:
      b"ONNX\\x00"  — ONNX protobuf; loaded with onnxruntime (no pickle exec).
      b"SAFE\\x00"  — SHA-256-validated joblib pickle.
      b"\\x80..."   — Legacy raw pickle; WARN logged; rotate after next fit.
    """
    import io

    if model_bytes[:5] == b"ONNX\x00":
        import onnxruntime as rt
        sess = rt.InferenceSession(model_bytes[5:])
        input_name = sess.get_inputs()[0].name
        X_input = features.reshape(1, -1).astype(np.float32)
        outputs = sess.run(None, {input_name: X_input})
        prob_output = outputs[1]  # list[dict] or 2-D array
        if isinstance(prob_output, list):
            # list of {class: prob} dicts
            p = float(prob_output[0][1])
        else:
            p = float(prob_output[0][1])
        return p
```

```
    elif model_bytes[:5] == b"SAFE\x00":
        import hashlib
        import joblib
        # Format: b"SAFE\x00" + 64 ASCII hex chars + b"\x00" + pickle_bytes
        sha256_hex = model_bytes[5:69].decode("ascii")
        pickle_bytes = model_bytes[70:]
        actual_hex = hashlib.sha256(pickle_bytes).hexdigest()
        if actual_hex != sha256_hex:
            raise ValueError(
                f"Propensity model hash mismatch — possible tampering "
                f"(stored={sha256_hex[:8]}…, actual={actual_hex[:8]}…)"
            )
        model = joblib.load(io.BytesIO(pickle_bytes))
        X = features.reshape(1, -1)
        return float(model.predict_proba(X)[0][1])

    elif model_bytes[:1] == b"\x80":
        import joblib
        _log.warning(
            "Propensity model: legacy pickle model detected — "
            "arbitrary code execution risk; rotate after next fit"
        )
        model = joblib.load(io.BytesIO(model_bytes))
        X = features.reshape(1, -1)
        return float(model.predict_proba(X)[0][1])

    else:
        raise ValueError(
            f"Unknown propensity model format (magic={model_bytes[:5]!r})"
        )


# ─────────────────────────────────────────────────────────────────────────────
# Gradient Boosting scoring (uses trained model bytes from caller)
# ─────────────────────────────────────────────────────────────────────────────

def score_session(
    features: np.ndarray,
    model_bytes: bytes | None = None,
) -> dict[str, Any]:
    """
    Score a single session.

    If model_bytes is None → falls back to rule-based score.
    If model_bytes provided → deserialise joblib model and predict.

    Returns:
        {p_convert: float, main_objection: str|None}
    """
    p_convert: float

    if model_bytes is not None:
        import io
        p_convert = _load_and_score(model_bytes, features)
    else:
        p_convert = rule_based_score(features)

    objection = determine_objection(features)

    return {
        "p_convert":       p_convert,
        "main_objection":  objection,
    }
```

```
# ──────────────────────────────────────────────────────────────────────────
# Gradient Boosting model fitting
# ──────────────────────────────────────────────────────────────────────────

def fit_propensity_model(
    X: np.ndarray,
    y: np.ndarray,
    n_estimators: int = 200,
    max_depth: int = 4,
    learning_rate: float = 0.05,
    random_state: int = 42,
) -> bytes:
    """
    Fit GradientBoostingClassifier and return serialised model bytes.

    Args:
        X: (n_samples, N_PROPENSITY_FEATURES) feature matrix
        y: (n_samples,) binary conversion labels (0/1)
        n_estimators / max_depth / learning_rate: model hyper-parameters

    Returns:
        joblib-serialised model bytes for storage
    """
    import io
    import joblib
    from sklearn.ensemble import GradientBoostingClassifier
    from sklearn.preprocessing import StandardScaler
    from sklearn.pipeline import Pipeline

    if len(X) < 30:
        raise ValueError(f"Need at least 30 labelled sessions, got {len(X)}")
    if len(X) != len(y):
        raise ValueError("X and y must have the same length")
    if len(np.unique(y)) < 2:
        raise ValueError("y must contain both classes (0 and 1)")

    pipe = Pipeline([
        ("scaler", StandardScaler()),
        ("gb", GradientBoostingClassifier(
            n_estimators=n_estimators,
            max_depth=max_depth,
            learning_rate=learning_rate,
            random_state=random_state,
        )),
    ])
    pipe.fit(X, y)

    # ── Phase 12-1: prefer ONNX (no pickle exec on load) ──────────────────
    try:
        from skl2onnx import convert_sklearn
        from skl2onnx.common.data_types import FloatTensorType
        initial_type = [("float_input", FloatTensorType([None, N_PROPENSITY_FEATURES]))]
        onnx_proto = convert_sklearn(pipe, initial_types=initial_type)
        return b"ONNX\x00" + onnx_proto.SerializeToString()
    except Exception as exc:
        _log.warning(
            "ONNX conversion failed (%s) — falling back to SHA-256-validated joblib pickle",
            exc,
        )

    # ── Fallback: SHA-256 hash-validated joblib pickle ────────────────────
    import hashlib
    buf = io.BytesIO()
    joblib.dump(pipe, buf)
    pickle_bytes = buf.getvalue()
```

```
    sha256_hex = hashlib.sha256(pickle_bytes).hexdigest().encode("ascii")  # 64 bytes
    return b"SAFE\x00" + sha256_hex + b"\x00" + pickle_bytes


# ─────────────────────────────────────────────────────────────────────────────
# Uplift score (Phase 5-3)
# ─────────────────────────────────────────────────────────────────────────────

def compute_uplift_score(
    p_convert_personalised: float,
    p_convert_control: float,
) -> float:
    """
    Uplift = P(convert | personalised) - P(convert | control).
    Clamped to [-1, 1].
    """
    return float(min(max(p_convert_personalised - p_convert_control, -1.0), 1.0))
```

2.5 `ml-service/app/services/burnout_survival_service.py`

Назначение: Предсказание выгорания (burnout) креатива через модель выживаемости Cox PH + Weibull.

Математическая логика:
- **Cox Proportional Hazard:** h(t) = h0(t) × exp(β₁×log_frequency + β₂×log_initial_roas)
- DEFAULT_BETAS: log_frequency=0.8, log_initial_roas=−0.5
- **Weibull функция выживаемости:** S(t) = exp(−(t/λ_eff)^k), scale=14, shape=1.5
- λ_eff = scale / exp(β×X)^(1/k)
- **Бинарный поиск:** день t, *где S(t)* < 0.5 (медиана выживаемости) → burnout_day.
- **Фолбек для N<30:** эвристика: 90/(1 + frequency) дней.

Входные данные: frequency (частота показа), initial_roas.
Выходные данные: {burnout_day, survival_at_30d, hazard_ratio}.

Взаимодействие: используется `BurnoutController.java`; результат влияет на решение о ротации креативов.

Потенциальные риски:
- DEFAULT_BETAS — эмпирически не откалиброваны под инвестиционный нейш (получены из общих данных).
- При log_initial_roas=0 (ROAS не известен) hazard ratio не меняется, но это корректно только если ROAS не коррелирует с выгоранием.
- Weibull scale=14 дней — агрессивно короткий базовый срок жизни для инвестиционного контента.

Полный код

```
"""
Burnout survival model — pure logic, no FastAPI/Pydantic imports.
Cox PH simplified implementation. Heuristic kept as fallback below n_min.
vision §15.8 task 2: survival model on frequency, initial_roas, audience_type, season.
"""
from __future__ import annotations
import math
from typing import Optional

N_MIN_FOR_SURVIVAL = 30

EXPERT_PRIOR_BETAS = {  # expert priors, NOT fitted coefficients — no training loop in this service
    "log_frequency":     0.8,   # higher frequency → faster burnout
    "log_initial_roas": -0.5,   # stronger ROAS start → longer survival
    "audience_broad":   -0.2,
    "audience_retarget": 0.4,
    "season_peak":       0.3,
}
DEFAULT_BETAS = EXPERT_PRIOR_BETAS  # backward-compat alias

AUDIENCE_TYPE_ENCODING = {
    "BROAD":       "audience_broad",
    "LOOKALIKE":    0.0,
    "RETARGETING": "audience_retarget",
    "CUSTOM":       0.1,
}

SEASON_ENCODING = {
    "PEAK":   "season_peak",
    "NORMAL":  0.0,
    "LOW":    -0.2,
}


def encode_covariates(
    frequency: float,
    initial_roas: float,
    audience_type: str = "BROAD",
    season: str = "NORMAL",
) -> dict:
    cov = {
        "log_frequency":    math.log(max(frequency, 0.01)),
        "log_initial_roas": math.log(max(initial_roas, 0.01)),
    }
    aud_key = AUDIENCE_TYPE_ENCODING.get(audience_type.upper(), 0.0)
    cov["audience_effect"] = EXPERT_PRIOR_BETAS.get(aud_key, 0.0) if isinstance(aud_key, str) else
float(aud_key)
    sea_key = SEASON_ENCODING.get(season.upper(), 0.0)
    cov["season_effect"] = EXPERT_PRIOR_BETAS.get(sea_key, 0.0) if isinstance(sea_key, str) else
float(sea_key)
    return cov


_WEIBULL_SCALE_DAYS = 21.0  # baseline creative half-life; 14d for DTC, 21d+ for B2B/investment funnels


def cox_hazard_ratio(covariates: dict) -> float:
    linear = (
        EXPERT_PRIOR_BETAS["log_frequency"]    * covariates["log_frequency"]
      + EXPERT_PRIOR_BETAS["log_initial_roas"] * covariates["log_initial_roas"]
      + covariates.get("audience_effect", 0.0)
      + covariates.get("season_effect", 0.0)
    )
    return math.exp(linear)
```

```
def weibull_survival(t: float, scale: float = _WEIBULL_SCALE_DAYS, shape: float = 1.5, hazard_ratio:
float = 1.0) -> float:
    """S(t) = exp(-(t/λ_eff)^k), λ_eff = scale / HR^(1/k)."""
    if t <= 0:
        return 1.0
    effective_scale = scale / (hazard_ratio ** (1.0 / shape))
    return math.exp(-((t / effective_scale) ** shape))


def predict_burnout_days(
    frequency: float,
    initial_roas: float,
    current_days_running: int,
    audience_type: str = "BROAD",
    season: str = "NORMAL",
    survival_threshold: float = 0.5,
    n_obs: int = 0,
) -> dict:
    """
    Predict days until burnout (S(t) < survival_threshold).
    Falls back to heuristic when n_obs < N_MIN_FOR_SURVIVAL.
    """
    if n_obs < N_MIN_FOR_SURVIVAL:
        heuristic_days = max(7, int(21 / max(frequency, 0.5)))
        return {
            "predicted_burnout_day": heuristic_days,
            "days_remaining":        max(0, heuristic_days - current_days_running),
            "survival_at_current":   None,
            "method":                "HEURISTIC",
            "confidence":            "LOW",
            "note":                  f"n_obs={n_obs} < {N_MIN_FOR_SURVIVAL} — using heuristic",
        }

    cov = encode_covariates(frequency, initial_roas, audience_type, season)
    hr  = cox_hazard_ratio(cov)
    s_current = weibull_survival(current_days_running, hazard_ratio=hr)

    # Binary search for day when S(t) < threshold
    lo, hi = float(current_days_running), float(current_days_running + 120)
    for _ in range(40):
        mid = (lo + hi) / 2
        if weibull_survival(mid, hazard_ratio=hr) < survival_threshold:
            hi = mid
        else:
            lo = mid
    burnout_day = int(hi)

    confidence = "HIGH" if n_obs >= 100 else ("MEDIUM" if n_obs >= 50 else "LOW")
    return {
        "predicted_burnout_day": burnout_day,
        "days_remaining":        max(0, burnout_day - current_days_running),
        "survival_at_current":   round(s_current, 4),
        "hazard_ratio":          round(hr, 4),
        "method":                "SURVIVAL",
        "confidence":            confidence,
        "n_obs":                 n_obs,
    }


def should_suppress_burnout_alert(anchor_verdict: Optional[str]) -> bool:
    """Suppress burnout alert when anchor verdict is MARKET — vision §8.1."""
    return anchor_verdict in ("MARKET", "MARKET_SHIFT")
```

СЕКЦИЯ 3 — Атрибуция

3.1 `ml-service/app/services/attribution_service.py`

Назначение: Многомодельная атрибуция конверсий по каналам.

Математическая логика:

Markov Attribution:
- Строится матрица переходов P(состояние j | состояние i) по всем путям.
- Removal effects: для каждого канала k убирается строка/столбец → вычисляется P(конверсии) без k.
- Вес канала k = (P_полная − P_без_k) / Σ_j(P_полная − P_без_j).

Shapley Value (Monte Carlo):
- n=200 случайных коалиций, seed=42.
- Для каждой коалиции: marginal contribution канала k = v(S∪{k}) − v(S).
- Вес k = среднее marginal contribution по всем коалициям.
- n=200 — точность ~±5% для 5 каналов.

Time-Decay:
- Вес касания в момент t_i: w_i = 2^(−(T−t_i)/half_life), half_life=7 дней.
- Нормализованные веса → атрибуция.

Data-Driven:
- Предобученные channel_weights из external source (фолбек = равномерное распределение).

Last-Touch, Linear: стандартные алгоритмы.

Входные данные: список touchpoints с channel_id и timestamp.
Выходные данные: {channel_id: attribution_value} для каждой модели.

Потенциальные риски:
- n=200 для Shapley — очень мало. При 6+ каналах возможны высокие отклонения. Рекомендуется n≥1000.
- Markov предполагает стационарность цепи Маркова — нарушается при сезонности.
- seed=42 фиксирован → один и тот же порядок коалиций каждый раз. Следует проверить на чувствительность к seed.

Полный код

```
"""
Cross-channel attribution — Shapley + Markov MTA. Phase 7, §17.3.
No FastAPI/Pydantic imports.
"""
from __future__ import annotations

import random
from itertools import combinations


def compute_transition_matrix(
    touchpoint_paths: list[list[str]],
    channels: list[str],
) -> dict[str, dict[str, float]]:
    """Build Markov transition matrix from touch paths."""
    states = channels + ["convert", "null"]
    counts: dict[str, dict[str, int]] = {s: {t: 0 for t in states} for s in states}

    for path in touchpoint_paths:
        if not path:
            continue
        full = path[:]
        if full[-1] != "convert":
            full.append("null")
        for i in range(len(full) - 1):
            frm = full[i]
            to_ = full[i + 1]
            if frm in counts and to_ in counts[frm]:
                counts[frm][to_] += 1

    matrix: dict[str, dict[str, float]] = {}
    for frm, targets in counts.items():
        total = sum(targets.values())
        if total == 0:
            matrix[frm] = {t: 0.0 for t in targets}
        else:
            matrix[frm] = {t: v / total for t, v in targets.items()}

    return matrix


def _conversion_rate_from_matrix(
    matrix: dict[str, dict[str, float]],
    channels: list[str],
    max_steps: int = 100,
) -> float:
    """Estimate overall conversion probability via matrix power iteration."""
    # Start with equal probability on each channel
    if not channels:
        return 0.0
    state_prob = {ch: 1.0 / len(channels) for ch in channels}
    convert_prob = 0.0

    for _ in range(max_steps):
        next_state: dict[str, float] = {ch: 0.0 for ch in channels}
        for state, prob in state_prob.items():
            if prob < 1e-9 or state not in matrix:
                continue
            for target, t_prob in matrix[state].items():
                if target == "convert":
                    convert_prob += prob * t_prob
                elif target in next_state:
                    next_state[target] += prob * t_prob
        state_prob = next_state
```

```
        if sum(state_prob.values()) < 1e-9:
            break

    return convert_prob


def compute_removal_effects(
    transition_matrix: dict[str, dict[str, float]],
    channels: list[str],
) -> dict[str, float]:
    """Removal effect for each channel: how much overall CVR drops when it's removed."""
    baseline = _conversion_rate_from_matrix(transition_matrix, channels)
    effects: dict[str, float] = {}
    for ch in channels:
        remaining = [c for c in channels if c != ch]
        removed_rate = _conversion_rate_from_matrix(transition_matrix, remaining)
        effects[ch] = max(0.0, baseline - removed_rate)
    return effects


def compute_markov_attribution(touchpoint_paths: list[list[str]]) -> dict[str, float]:
    """Full Markov chain attribution. Returns fraction per channel."""
    channels: list[str] = []
    seen: set[str] = set()
    for path in touchpoint_paths:
        for ch in path:
            if ch != "convert" and ch not in seen:
                channels.append(ch)
                seen.add(ch)

    if not channels:
        return {}

    converting = [p for p in touchpoint_paths if "convert" in p]
    if not converting:
        return {ch: 0.0 for ch in channels}

    matrix = compute_transition_matrix(touchpoint_paths, channels)
    effects = compute_removal_effects(matrix, channels)
    total_effect = sum(effects.values())
    if total_effect == 0.0:
        equal = 1.0 / len(channels)
        return {ch: round(equal, 6) for ch in channels}

    return {ch: round(e / total_effect, 6) for ch, e in effects.items()}


def compute_shapley_attribution(touchpoint_paths: list[list[str]]) -> dict[str, float]:
    """
    Shapley value attribution via Monte Carlo coalition sampling (n_samples=200, seed=42).
    Returns fraction per channel.
    """
    channels: list[str] = []
    seen: set[str] = set()
    for path in touchpoint_paths:
        for ch in path:
            if ch != "convert" and ch not in seen:
                channels.append(ch)
                seen.add(ch)

    if not channels:
        return {}

    converting = [p for p in touchpoint_paths if "convert" in p]
    if not converting:
```

```
        return {ch: 0.0 for ch in channels}

    # Build a simple characteristic function: CVR of paths containing the coalition
    def cvr_coalition(coalition: frozenset[str]) -> float:
        if not coalition:
            return 0.0
        relevant = [p for p in touchpoint_paths if coalition.issubset(set(p))]
        if not relevant:
            return 0.0
        conv = sum(1 for p in relevant if "convert" in p)
        return conv / len(relevant)

    n_ch = len(channels)
    shapley: dict[str, float] = {ch: 0.0 for ch in channels}

    rng = random.Random(42)
    # SE ≈ sqrt(p*(1-p)/n_samples). At n=1000, SE≈1.6% for p≈0.5
    n_samples = 1_000

    for _ in range(n_samples):
        perm = channels[:]
        rng.shuffle(perm)
        coalition: set[str] = set()
        prev_v = cvr_coalition(frozenset())
        for ch in perm:
            coalition.add(ch)
            new_v = cvr_coalition(frozenset(coalition))
            shapley[ch] += new_v - prev_v
            prev_v = new_v

    total = sum(shapley.values())
    if total == 0.0:
        equal = 1.0 / n_ch
        return {ch: round(equal, 6) for ch in channels}

    return {ch: round(v / total, 6) for ch, v in shapley.items()}


def _last_touch(touchpoint_paths: list[list[str]]) -> dict[str, float]:
    channels: set[str] = set()
    for path in touchpoint_paths:
        for ch in path:
            if ch != "convert":
                channels.add(ch)

    counts: dict[str, int] = {ch: 0 for ch in channels}
    for path in touchpoint_paths:
        if "convert" not in path:
            continue
        # last channel before "convert"
        for ch in reversed(path):
            if ch != "convert":
                counts[ch] = counts.get(ch, 0) + 1
                break

    total = sum(counts.values())
    if total == 0:
        return {ch: 0.0 for ch in channels}
    return {ch: round(v / total, 6) for ch, v in counts.items()}


def _linear_touch(touchpoint_paths: list[list[str]]) -> dict[str, float]:
    channels: set[str] = set()
    for path in touchpoint_paths:
        for ch in path:
```

```
            if ch != "convert":
                channels.add(ch)

    counts: dict[str, float] = {ch: 0.0 for ch in channels}
    for path in touchpoint_paths:
        if "convert" not in path:
            continue
        touch_channels = [ch for ch in path if ch != "convert"]
        if not touch_channels:
            continue
        credit = 1.0 / len(touch_channels)
        for ch in touch_channels:
            counts[ch] = counts.get(ch, 0.0) + credit

    total = sum(counts.values())
    if total == 0:
        return {ch: 0.0 for ch in channels}
    return {ch: round(v / total, 6) for ch, v in counts.items()}


def compute_mta_summary(
    touchpoint_paths: list[list[str]],
    method: str = "markov",
) -> dict:
    """Returns attribution summary dict."""
    total_paths = len(touchpoint_paths)
    converting_paths = sum(1 for p in touchpoint_paths if "convert" in p)

    if method == "shapley":
        attribution = compute_shapley_attribution(touchpoint_paths)
    elif method == "last_touch":
        attribution = _last_touch(touchpoint_paths)
    elif method == "linear":
        attribution = _linear_touch(touchpoint_paths)
    else:
        attribution = compute_markov_attribution(touchpoint_paths)

    return {
        "method": method,
        "attribution": attribution,
        "total_paths": total_paths,
        "converting_paths": converting_paths,
    }


# ── Per-session touchpoint attribution (8-10) ──────────────────────────

import math
from datetime import datetime, timezone


def _parse_dt(val) -> datetime | None:
    if val is None:
        return None
    if isinstance(val, datetime):
        return val
    try:
        return datetime.fromisoformat(str(val).replace("Z", "+00:00"))
    except Exception:
        return None


def last_touch_attribution(touchpoints: list[dict]) -> list[dict]:
    """100% credit to last touchpoint."""
    if not touchpoints:
```

```
        return []
    sorted_tps = sorted(
        touchpoints,
        key=lambda t: _parse_dt(t.get("touched_at")) or datetime.min.replace(tzinfo=timezone.utc),
    )
    result = [{"touchpoint_id": t["id"], "credit": 0.0, "model": "last_touch"} for t in sorted_tps]
    result[-1]["credit"] = 1.0
    return result


def linear_attribution(touchpoints: list[dict]) -> list[dict]:
    """Equal credit across all touchpoints."""
    if not touchpoints:
        return []
    credit = 1.0 / len(touchpoints)
    return [{"touchpoint_id": t["id"], "credit": round(credit, 6), "model": "linear"} for t in
touchpoints]


_TIME_DECAY_HALF_LIFE_DAYS = 21.0  # 7d too aggressive for B2B/investment funnels where 30d-old touches
matter

def time_decay_attribution(touchpoints: list[dict], half_life_days: float = _TIME_DECAY_HALF_LIFE_DAYS)
-> list[dict]:
    """Exponential time decay: closer to conversion gets more credit."""
    if not touchpoints:
        return []
    sorted_tps = sorted(
        touchpoints,
        key=lambda t: _parse_dt(t.get("touched_at")) or datetime.min.replace(tzinfo=timezone.utc),
    )
    conversion_time = _parse_dt(sorted_tps[-1].get("touched_at")) or datetime.now(timezone.utc)
    weights = []
    for tp in sorted_tps:
        tp_time = _parse_dt(tp.get("touched_at")) or conversion_time
        try:
            days_before = (conversion_time - tp_time).total_seconds() / 86400
        except Exception:
            days_before = 0
        weights.append(math.pow(2, -days_before / half_life_days))
    total = sum(weights) or 1.0
    return [
        {"touchpoint_id": tp["id"], "credit": round(w / total, 6), "model": "time_decay"}
        for tp, w in zip(sorted_tps, weights)
    ]


_DEFAULT_CHANNEL_WEIGHTS = {
    "PAID_SOCIAL": 1.4,
    "PAID_SEARCH":  1.2,
    "EMAIL":        1.1,
    "ORGANIC":      0.9,
    "REFERRAL":     0.8,
    "DIRECT":       0.6,
}


def data_driven_attribution(
    touchpoints: list[dict],
    channel_weights: dict | None = None,
) -> list[dict]:
    """Channel-weight based attribution. Falls back to linear if no weights."""
    if not touchpoints:
        return []
    weights_map = channel_weights or _DEFAULT_CHANNEL_WEIGHTS
```

```
    raw = [weights_map.get(tp.get("channel", "DIRECT"), 1.0) for tp in touchpoints]
    total = sum(raw) or 1.0
    return [
        {"touchpoint_id": tp["id"], "credit": round(w / total, 6), "model": "data_driven"}
        for tp, w in zip(touchpoints, raw)
    ]


def run_all_models(touchpoints: list[dict], channel_weights: dict | None = None) -> dict:
    """Runs all 4 per-session attribution models."""
    return {
        "last_touch":  last_touch_attribution(touchpoints),
        "linear":      linear_attribution(touchpoints),
        "time_decay":  time_decay_attribution(touchpoints),
        "data_driven": data_driven_attribution(touchpoints, channel_weights),
    }


def aggregate_channel_credit(model_credits: list[dict], touchpoints: list[dict]) -> dict:
    """Aggregates credits per channel for a given model run."""
    tp_map = {t["id"]: t for t in touchpoints}
    result: dict[str, float] = {}
    for row in model_credits:
        tp      = tp_map.get(row["touchpoint_id"], {})
        channel = tp.get("channel", "UNKNOWN")
        result[channel] = result.get(channel, 0.0) + row["credit"]
    return {k: round(v, 4) for k, v in sorted(result.items(), key=lambda x: -x[1])}
```

СЕКЦИЯ 4 — Статистическая аналитика

4.1 `ml-service/app/services/calibration_service.py`

Назначение: ECE (Expected Calibration Error) для оценки калиброванности вероятностных моделей.

Математическая логика:

```
ECE = Σ_{b=1}^{B} (n_b / N) × |mean_pred_b − mean_actual_b|
```

- B=10 бинов; n_b — количество семплов в бине b; N — всего.
- gap < 0.05 → модель считается калиброванной.

Взаимодействие: `model_quality_service.py` использует аналогичную формулу для оценки Thompson моделей.

Потенциальные риски:
- ECE чувствителен к выбору B. B=10 стандарт, но при малой выборке (<200) ряд бинов будет пустым.

Полный код

```
"""
Match-score calibration — pure logic, zero FastAPI/Pydantic imports.
Monthly: match_score buckets vs realised CVR (vision §6.3a).
"""
from __future__ import annotations


def compute_calibration_report(
    predictions_and_actuals: list[dict],
    n_buckets: int = 10,
) -> dict:
    """
    Calibration: are predicted scores aligned with actual conversion rates?
    Groups into n_buckets, computes mean predicted vs mean actual per bucket.
    Returns ECE (Expected Calibration Error) + per-bucket breakdown.
    """
    if not predictions_and_actuals:
        return {"ece": None, "buckets": [], "n_total": 0, "note": "No data"}

    bucket_size = 1.0 / n_buckets
    buckets = [
        {"min": i * bucket_size, "max": (i + 1) * bucket_size, "predictions": [], "actuals": []}
        for i in range(n_buckets)
    ]

    for item in predictions_and_actuals:
        score  = float(item.get("score", 0.0))
        actual = 1.0 if item.get("converted", False) else 0.0
        idx    = min(int(score / bucket_size), n_buckets - 1)
        buckets[idx]["predictions"].append(score)
        buckets[idx]["actuals"].append(actual)

    results = []
    ece_sum = 0.0
    n_total = len(predictions_and_actuals)

    for b in buckets:
        n = len(b["predictions"])
        if n == 0:
            continue
        mean_pred   = sum(b["predictions"]) / n
        mean_actual = sum(b["actuals"]) / n
        gap         = abs(mean_pred - mean_actual)
        ece_sum    += gap * (n / n_total)
        results.append({
            "bucket_min":     round(b["min"], 2),
            "bucket_max":     round(b["max"], 2),
            "n":              n,
            "mean_predicted": round(mean_pred, 4),
            "mean_actual":    round(mean_actual, 4),
            "gap":            round(gap, 4),
            "calibrated":     gap < 0.05,
        })

    return {
        "ece":     round(ece_sum, 4),
        "buckets": results,
        "n_total": n_total,
        "note":    "ECE < 0.05 = well-calibrated (vision §6.3a)",
    }
```

4.2 `ml-service/app/services/ope_service.py`

Назначение: Off-Policy Evaluation для валидации политики перед деплоем с реальным бюджетом.

Математическая логика:
- **IPS:** Σ(w_i × r_i) / n; w_i = π_новая(a_i|x_i) / π_старая(a_i|x_i).
- **SNIPS (self-normalized):** Σ(w_i × r_i) / Σw_i — уменьшает дисперсию.
- **Doubly-Robust:** IPS − w×mean_r + mean_r — комбинирует direct model и IPS.
- **ESS:** $(\Sigma w)^2 / \Sigma w^2$ — эффективный размер выборки.
- **Bootstrap CI:** n=500 бутстреп выборок; w_max=10.0 (кеппинг весов).
- **Условие PASS:** SNIPS ≥ 0.95 AND ESS ≥ 200.

Входные данные: decisions (action, propensity, reward) из decision_log.
Выходные данные: {ips, snips, dr, ess, ci_lower, ci_upper, pass}.

Взаимодействие: `OpeController.java`; propensity берётся из `thompson_service.py` (n=200 симуляций).

Потенциальные риски:
- w_max=10.0 — агрессивный кеппинг. При неравномерной политике (одни рукавы выбираются >90% времени) веса для редких рукавов обрезаются → IPS занижен.
- n=500 бутстрепов для CI — минимально достаточно. Рекомендуется n≥1000.
- Propensity из n=200 симуляций (thompson_service) — с погрешностью. Неточная propensity → смещённый IPS/SNIPS.

Полный код

```
"""
Off-Policy Evaluation (IPS / SNIPS / Doubly-Robust) — no FastAPI/Pydantic imports.
Vision §12.5, §23 Phase 3; arch §15.11.

Estimates value of the CURRENT policy (today's posteriors) vs the LOGGED policy
on the decision_log rows, using propensity weights.
"""
from __future__ import annotations

import math
import random
from typing import Optional


def _clip_weight(w: float, w_max: float) -> float:
    return min(w, w_max)


def _bootstrap_snips_ci(weights: list[float], rewards: list[float], n_boot: int = 500,
                        alpha: float = 0.05, seed: int = 42) -> tuple[float, float]:
    """Percentile bootstrap of the self-normalised estimator Σwr/Σw over resampled decisions.

    Audit 6.2: the previous version bootstrapped per-decision values in which the weights cancelled,
    i.e. a CI for the logged reward rate rather than for SNIPS.
    """
    rng = random.Random(seed)
    n = len(weights)
    stats = []
    for _ in range(n_boot):
        sw = swr = 0.0
        for _k in range(n):
            i = rng.randint(0, n - 1)
            sw += weights[i]
            swr += weights[i] * rewards[i]
        stats.append(swr / sw if sw > 1e-9 else 0.0)
    stats.sort()
    lo_idx = int(alpha / 2 * n_boot)
    hi_idx = int((1 - alpha / 2) * n_boot)
    return stats[lo_idx], stats[min(hi_idx, n_boot - 1)]


def compute_ope(
    decisions: list[dict],
    current_policy_probs: dict,  # {arm_id: probability under current policy}
    w_max: float = 10.0,
    min_ess: float = 200.0,
) -> dict:
    """
    decisions: list of {arm_id, logged_prob, reward}
    current_policy_probs: {arm_id: new_prob}

    Returns {ips, snips, dr, ci_low, ci_high, ess, clipped_mass, verdict}.
    """
    if len(decisions) < 10:
        return {
            "ips": None, "snips": None, "dr": None,
            "ci_low": None, "ci_high": None,
            "ess": 0, "clipped_mass": 0,
            "verdict": "INSUFFICIENT_DATA"
        }

    weights = []
    clipped = 0
    ips_terms = []
```

```
    weight_sum = 0.0

    for d in decisions:
        arm_id = d.get("arm_id") or d.get("variant_id", "")
        logged_prob = float(d.get("logged_prob", d.get("propensity", 1e-6)) or 1e-6)
        new_prob = float(current_policy_probs.get(str(arm_id), logged_prob))
        reward = float(d.get("reward", 0) or 0)

        raw_w = new_prob / max(logged_prob, 1e-9)
        w = _clip_weight(raw_w, w_max)
        if raw_w > w_max:
            clipped += 1

        weights.append(w)
        ips_terms.append(w * reward)
        weight_sum += w

    n = len(decisions)
    clipped_mass = clipped / n

    # IPS
    ips = sum(ips_terms) / n

    # SNIPS (self-normalised)
    snips = sum(ips_terms) / weight_sum if weight_sum > 1e-9 else 0.0

    # DR (doubly-robust) — use mean reward as outcome model (simple baseline)
    mean_reward = sum(d.get("reward", 0) or 0 for d in decisions) / n
    dr_terms = [ips_terms[i] - weights[i] * mean_reward + mean_reward
                for i in range(n)]
    dr = sum(dr_terms) / n

    # ESS
    w2 = sum(w ** 2 for w in weights)
    ess = (weight_sum ** 2) / max(w2, 1e-9)

    # Bootstrap CI on SNIPS
    rewards = [float(d.get("reward", 0) or 0) for d in decisions]
    if n >= 10:
        ci_low, ci_high = _bootstrap_snips_ci(weights, rewards)
    else:
        ci_low, ci_high = snips * 0.8, snips * 1.2

    # Verdict (audit 6.1): SNIPS is an absolute reward *rate* of the current policy (typically
    # 0.03-0.05), so it is compared with the logged policy's mean reward, not with 0.95.
    # PASS when the current policy keeps >= 95 % of the logged reward rate AND ESS is sufficient.
    baseline = mean_reward
    verdict = "PASS" if snips >= 0.95 * baseline and ess >= min_ess else "FAIL"
    if len(decisions) < 30:
        verdict = "INSUFFICIENT_DATA"

    return {
        "ips": round(ips, 4),
        "snips": round(snips, 4),
        "dr": round(dr, 4),
        "ci_low": round(ci_low, 4),
        "ci_high": round(ci_high, 4),
        "ess": round(ess, 1),
        "clipped_mass": round(clipped_mass, 4),
        "logged_mean_reward": round(mean_reward, 4),
        "verdict": verdict,
    }


def passes_ope_gate(result: dict, min_ess: float = 200.0) -> bool:
```

```
    """True when an OPE result clears the promotion gate (verdict PASS and ESS >= min_ess).

    Used by /analyze/ope/ips; INSUFFICIENT_DATA and FAIL never pass.
    """
    return result.get("verdict") == "PASS" and float(result.get("ess", 0.0)) >= min_ess
```

4.3 `ml-service/app/services/cb_bayesian_service.py`

Назначение: Байесовский circuit breaker для мониторинга CPA.

Математическая логика:
- **Beta posterior на CVR:** α = conversions + 1; β = clicks − conversions + 1.
- **Семплинг:** n Монте-Карло семплов из Beta(α, β).
- **CPA из семпла:** cpa_sample = spend_per_click / cvr_sample.
- **P(CPA > 2×target):** доля семплов, где cpa_sample > threshold.
- **Триггер:** P > 0.90.

Взаимодействие: `CbBayesianController.java`; вызывается автоматически при обновлении рекламных метрик.

Потенциальные риски:
- CPA = spend_per_click / CVR_sample — только если CVR уже включает все кликнувших. Если clicks ≠ покупатели/лиды, знаменатель некорректен.
- Порог 2×target фиксирован. Для инвестиционного нейша с высоким ticket нормальное отклонение CPA может быть >2×.

Полный код

```
"""
Circuit Breaker Bayesian L1 trigger — pure logic, no FastAPI/Pydantic imports.
P(CPA > k * target_cpa | data) via Monte Carlo Beta posterior on CVR.
vision §12.5a L1 row: P(CPA > 2× target) > 0.90 triggers pause.
"""
from __future__ import annotations
import math
from typing import Optional


def compute_cpa_exceedance_probability(
    clicks: int,
    conversions: int,
    total_spend: float,
    target_cpa: float,
    cpa_multiplier: float = 2.0,
    n_samples: int = 2_000,
) -> dict:
    """
    P(CPA > cpa_multiplier * target_cpa | observed data).

    Model: CVR ~ Beta(α=conversions+1, β=clicks-conversions+1)
           CPA  = spend_per_click / CVR
    Returns p_exceed, should_trigger (p > 0.90), observed_cpa.
    """
    threshold = target_cpa * cpa_multiplier

    if clicks < 1 or total_spend <= 0:
        return {
            "p_exceed": 0.0,
            "should_trigger": False,
            "observed_cpa": None,
            "reason": "Insufficient data (no clicks or spend)",
        }

    # Audit 6.5: no special case for zero conversions — Beta(1, clicks+1) already encodes it.
    # (The old branch fired on 1 click / $101 and stayed silent on 500 clicks / $99.)
    observed_cpa = total_spend / conversions if conversions > 0 else None

    alpha_cvr = float(conversions) + 1.0
    beta_cvr  = float(clicks - conversions) + 1.0

    try:
        import random
        rng = random.Random()  # unseeded: deterministic seed hid MC variance
        spend_per_click = total_spend / clicks
        exceed_count = 0
        for _ in range(n_samples):
            cvr_s = rng.betavariate(alpha_cvr, beta_cvr)
            if cvr_s < 1e-9:
                exceed_count += 1
                continue
            cpa_s = spend_per_click / cvr_s
            if cpa_s > threshold:
                exceed_count += 1
        p_exceed = exceed_count / n_samples

    except Exception:
        p_exceed = 1.0 if (observed_cpa is None or observed_cpa > threshold) else 0.0

    return {
        "p_exceed": round(p_exceed, 4),
        "should_trigger": p_exceed > 0.90,
        "observed_cpa": round(observed_cpa, 2) if observed_cpa is not None else None,
```

```
        "threshold_cpa": round(threshold, 2),
        "target_cpa": target_cpa,
        "cpa_multiplier": cpa_multiplier,
        "clicks": clicks,
        "conversions": conversions,
        "reason": (
            f"P(CPA > {threshold:.2f}) = {p_exceed:.3f} "
            f"{'→ TRIGGER' if p_exceed > 0.90 else '→ OK'}"
        ),
    }


def l1_combined_check(
    spend: float,
    spend_limit: float,
    clicks: int,
    conversions: int,
    target_cpa: float,
    cpa_multiplier: float = 2.0,
) -> dict:
    """
    Combined L1: spend-based OR Bayesian CPA-based trigger.
    Either condition fires the circuit breaker.
    """
    spend_trigger = spend >= spend_limit
    bayesian = compute_cpa_exceedance_probability(
        clicks, conversions, spend, target_cpa, cpa_multiplier
    )
    return {
        "spend_trigger":    spend_trigger,
        "bayesian_trigger": bayesian["should_trigger"],
        "should_pause":     spend_trigger or bayesian["should_trigger"],
        "spend":            round(spend, 2),
        "spend_limit":      spend_limit,
        "p_cpa_exceed":     bayesian["p_exceed"],
        "observed_cpa":     bayesian.get("observed_cpa"),
        "threshold_cpa":    bayesian.get("threshold_cpa"),
        "reason": (
            f"Spend {'≥' if spend_trigger else '<'} limit={spend_limit:.2f}; "
            f"P(CPA>2×target)={bayesian['p_exceed']:.3f}"
        ),
    }
```

4.4 `ml-service/app/services/glm_interactions_service.py`

Назначение: BIC-based отбор взаимодействий в GLM модели.

Математическая логика:
- ΔBIC = BIC_restricted − BIC_full
- BF ≈ exp(0.5 × ΔBIC) (приближение Байесовского фактора через BIC)
- BF > 3 → INCLUDE взаимодействие
- BF < 1/3 → EXCLUDE
- 1/3 ≤ BF ≤ 3 → INCONCLUSIVE

Взаимодействие: `GlmController.java`; используется для оценки взаимодействий atom×segment.

Потенциальные риски:
- BIC-based BF — это приближение, корректное только асимптотически (большая выборка). При n<100 BF занижен.
- exp(0.5 × ΔBIC) может overflow при большом ΔBIC. Нужна проверка на NaN/Inf.

Полный код

```
"""
GLM interaction terms via Bayes factor approximation — pure logic, no FastAPI/Pydantic.
vision §15.8 task 1: interaction terms by Bayes factor or BART when BF > 3.
"""
from __future__ import annotations
import math


def compute_bayes_factor_approx(
    n: int,
    r_squared_full: float,
    r_squared_restricted: float,
    k_extra_params: int = 1,
) -> dict:
    """
    BIC-based Bayes factor approximation for interaction term inclusion.
    BF ≈ exp(-0.5 * ΔBIC).
    BF > 3 → INCLUDE; BF < 1/3 → EXCLUDE; otherwise AMBIGUOUS.
    """
    if n < k_extra_params + 2:
        return {
            "bayes_factor": 1.0,
            "decision": "AMBIGUOUS",
            "delta_bic": 0.0,
            "r_squared_full": r_squared_full,
            "r_squared_restricted": r_squared_restricted,
            "threshold": 3.0,
            "reason": "Insufficient data",
        }

    rss_full       = max(1 - r_squared_full,       1e-10)
    rss_restricted = max(1 - r_squared_restricted, 1e-10)

    k_full       = k_extra_params + 1
    k_restricted = 1

    bic_full       = n * math.log(rss_full       / n) + k_full       * math.log(n)
    bic_restricted = n * math.log(rss_restricted / n) + k_restricted * math.log(n)

    delta_bic = bic_restricted - bic_full
    bf = math.exp(min(0.5 * delta_bic, 700.0))  # clamp: math.exp(>710) raises OverflowError

    if bf > 3.0:
        decision = "INCLUDE"
    elif bf < 1.0 / 3.0:
        decision = "EXCLUDE"
    else:
        decision = "AMBIGUOUS"

    return {
        "bayes_factor":          round(bf, 4),
        "decision":              decision,
        "delta_bic":             round(delta_bic, 4),
        "r_squared_full":        r_squared_full,
        "r_squared_restricted":  r_squared_restricted,
        "threshold":             3.0,
        "reason":                f"BF={bf:.2f} → {decision} (BF > 3 to include, vision §15.8 task 1)",
    }


def rank_interactions_by_bf(
    interaction_candidates: list[dict],
) -> list[dict]:
    """Rank candidate interaction terms by BF descending (INCLUDE first)."""
```

```
    results = []
    for c in interaction_candidates:
        bf_result = compute_bayes_factor_approx(
            c.get("n", 100),
            c.get("r2_full", 0.5),
            c.get("r2_restricted", 0.4),
        )
        results.append({**c, **bf_result})
    return sorted(results, key=lambda x: x["bayes_factor"], reverse=True)
```

4.5 `ml-service/app/services/diagnostics_service.py`

Назначение: Диагностика симптомов с коррекцией множественного тестирования (Benjamini-Hochberg).

Математическая логика:
- **Z-score:** z = (observed − baseline) / (baseline_std / √n).
- **P-value:** аппроксимация A&S полиномиального ряда для Φ(z).
- **BH процедура:** сортировка p-значений; порог для ранга k: FDR × k / n; первое нарушение — все выше не значимы.
- **FDR = 0.10** (10% ожидаемая доля ложноположительных).
- **MIN_N = 100** для подавления симптомов с малой выборкой.

Взаимодействие: `DiagnosticsController.java`; симптомы передаются в `signals_service.py`.

Потенциальные риски:
- BH процедура предполагает независимость тестов. CTR и CVR на одних данных — коррелированы → BH консервативнее, чем нужно.
- A&S приближение для p-value менее точно при |z| > 3.5.

Полный код

```
"""
Diagnostics baseline + Benjamini-Hochberg correction — pure logic.
Compares symptoms against tenant's own baseline per placement×device.
No FastAPI/Pydantic imports.
"""
from __future__ import annotations
import math
from typing import Optional

MIN_N_FOR_SYMPTOM = 100

SYMPTOMS = [
    "CTR_DROP", "CVR_DROP", "BOUNCE_SPIKE", "SCROLL_DEPTH_LOW",
    "LPV_CLICK_GAP", "MARKET_SHIFT", "CREATIVE_FATIGUE",
]

# Audit 6.7: for these symptoms the bad direction is a RISE of the observed metric
# (bounce rate, fatigue index, LPV/click gap); every other symptom is a drop.
RISE_IS_BAD = {"BOUNCE_SPIKE", "CREATIVE_FATIGUE", "LPV_CLICK_GAP"}


def symptom_p_value(symptom: str, z: float) -> float:
    """One-tailed p-value in the direction that is bad for this symptom."""
    return z_to_p_value(-z) if symptom in RISE_IS_BAD else z_to_p_value(z)


def compute_z_score(observed: float, baseline: float, baseline_std: float, n: int) -> float:
    if baseline_std <= 0 or n <= 0:
        return 0.0
    se = baseline_std / math.sqrt(n)
    return (observed - baseline) / se if se > 0 else 0.0


def z_to_p_value(z: float) -> float:
    """One-tailed p-value from z-score (lower tail for drops). A&S approximation."""
    abs_z = abs(z)
    t = 1.0 / (1.0 + 0.2316419 * abs_z)
    poly = t * (0.319381530
                + t * (-0.356563782
                       + t * (1.781477937
                              + t * (-1.821255978
                                     + t * 1.330274429))))
    p_upper = (1.0 / math.sqrt(2 * math.pi)) * math.exp(-0.5 * abs_z * abs_z) * poly
    if z < 0:
        return p_upper
    return 1.0 - p_upper


def benjamini_hochberg(p_values: list[float], fdr: float = 0.10) -> list[bool]:
    """BH procedure for FDR control. Returns True = significant after correction.

    Audit 6.8: reject every hypothesis up to the LARGEST rank k with p(k) <= k·q/n. The previous
    loop stopped at the first non-significant rank, which under-rejects
    (p = [0.04, 0.05, 0.06], q = 0.10 -> BH rejects all three; the old code rejected none).
    """
    n = len(p_values)
    if n == 0:
        return []
    indexed = sorted(enumerate(p_values), key=lambda x: x[1])
    k_max = 0
    for rank, (_idx, p) in enumerate(indexed, start=1):
        if p <= fdr * rank / n:
            k_max = rank
    rejected = [False] * n
```

```
    for rank, (idx, _p) in enumerate(indexed, start=1):
        if rank <= k_max:
            rejected[idx] = True
    return rejected


def check_minimum_n(sample_size: int, min_n: int = MIN_N_FOR_SYMPTOM) -> bool:
    return sample_size >= min_n


# BH applied separately to CTR and CVR families: anti-correlation between them violates PRDS assumption
_CTR_FAMILY = {"CTR_DROP", "LPV_CLICK_GAP"}
_CVR_FAMILY = {"CVR_DROP", "BOUNCE_SPIKE", "SCROLL_DEPTH_LOW", "CREATIVE_FATIGUE", "MARKET_SHIFT"}


def _apply_bh_by_family(
    eligible: list[dict],
    p_values: list[float],
    fdr: float,
) -> list[bool]:
    """Apply BH separately to CTR and CVR families; Bonferroni for cross-family.

    Returns a boolean mask (len == len(eligible)) where True means significant.
    """
    n = len(eligible)
    # Split indices by family
    ctr_indices = [i for i, o in enumerate(eligible) if o.get("symptom", "") in _CTR_FAMILY]
    cvr_indices = [i for i, o in enumerate(eligible) if o.get("symptom", "") in _CVR_FAMILY]
    other_indices = [i for i in range(n) if i not in ctr_indices and i not in cvr_indices]

    significant = [False] * n

    for group_indices in (ctr_indices, cvr_indices, other_indices):
        if not group_indices:
            continue
        group_p = [p_values[i] for i in group_indices]
        group_flags = benjamini_hochberg(group_p, fdr)
        for local_j, global_i in enumerate(group_indices):
            significant[global_i] = group_flags[local_j]

    # Bonferroni cross-family alert: both CTR and CVR families must flag at least one symptom
    ctr_flagged = any(significant[i] for i in ctr_indices)
    cvr_flagged = any(significant[i] for i in cvr_indices)
    cross_family_alert = ctr_flagged and cvr_flagged

    return significant, cross_family_alert


def run_diagnostics(
    observations: list[dict],
    fdr: float = 0.10,
) -> dict:
    """
    Run full diagnostics pipeline with BH correction.
    Each symptom compared against tenant's own baseline per placement×device
    to avoid Simpson's paradox (vision §3.2).

    BH is applied separately to CTR and CVR families (PRDS assumption violated
    across families due to anti-correlation). Cross-family alert uses Bonferroni.
    """
    eligible = [o for o in observations if check_minimum_n(o.get("n", 0))]

    if not eligible:
        return {
            "symptoms": [],
```

```
            "suppressed_count": len(observations),
            "note": f"All symptoms suppressed: n < {MIN_N_FOR_SYMPTOM}",
        }

    p_values = []
    for obs in eligible:
        z = compute_z_score(
            obs["observed"], obs["baseline"],
            obs.get("baseline_std", obs["baseline"] * 0.1 + 1e-6),
            obs["n"],
        )
        p_values.append(symptom_p_value(obs.get("symptom", ""), z))

    significant_flags, cross_family_alert = _apply_bh_by_family(eligible, p_values, fdr)

    symptoms_out = []
    for i, obs in enumerate(eligible):
        if significant_flags[i]:
            z = compute_z_score(
                obs["observed"], obs["baseline"],
                obs.get("baseline_std", obs["baseline"] * 0.1 + 1e-6),
                obs["n"],
            )
            symptoms_out.append({
                "symptom":   obs["symptom"],
                "placement": obs.get("placement", "UNKNOWN"),
                "device":    obs.get("device", "UNKNOWN"),
                "observed":  obs["observed"],
                "baseline":  obs["baseline"],
                "z_score":   round(z, 3),
                "p_value":   round(p_values[i], 4),
                "n":         obs["n"],
            })

    return {
        "symptoms":            symptoms_out,
        "total_checked":       len(eligible),
        "suppressed_count":    len(observations) - len(eligible),
        "fdr_threshold":       fdr,
        "bh_applied":          True,
        "bh_family_separated": True,
        "cross_family_alert":  cross_family_alert,
    }


MARKET_SHIFT_CTR_DROP_THRESHOLD  = -0.20  # -20% CTR vs baseline
MARKET_SHIFT_CVR_DROP_THRESHOLD  = -0.20  # -20% CVR vs baseline
MARKET_SHIFT_MULTI_PLACEMENT_MIN = 2      # must appear across ≥2 placements


def detect_market_shift(
    symptom_results: list[dict],
    ctr_baseline: float,
    cvr_baseline: float,
    ctr_observed: float,
    cvr_observed: float,
) -> dict:
    """
    Market-shift signal: drops that are systematic across placements and metrics
    rather than creative-specific. Fires when BOTH CTR and CVR fall >20%
    and the MARKET_SHIFT symptom appears across ≥2 placements (vision §3.2).
    """
    if ctr_baseline <= 0 or cvr_baseline <= 0:
        return {"market_shift_detected": False, "reason": "No baseline available"}
```

```
    ctr_delta = (ctr_observed - ctr_baseline) / ctr_baseline
    cvr_delta = (cvr_observed - cvr_baseline) / cvr_baseline

    market_placements = [
        s for s in symptom_results
        if s.get("symptom") == "MARKET_SHIFT"
    ]

    ctr_drop_ok   = ctr_delta <= MARKET_SHIFT_CTR_DROP_THRESHOLD
    cvr_drop_ok   = cvr_delta <= MARKET_SHIFT_CVR_DROP_THRESHOLD
    multi_place   = len(market_placements) >= MARKET_SHIFT_MULTI_PLACEMENT_MIN

    detected = ctr_drop_ok and cvr_drop_ok and multi_place

    return {
        "market_shift_detected": detected,
        "ctr_delta_pct":         round(ctr_delta * 100, 2),
        "cvr_delta_pct":         round(cvr_delta * 100, 2),
        "placements_affected":   len(market_placements),
        "thresholds": {
            "ctr_drop_pct":      MARKET_SHIFT_CTR_DROP_THRESHOLD * 100,
            "cvr_drop_pct":      MARKET_SHIFT_CVR_DROP_THRESHOLD * 100,
            "min_placements":    MARKET_SHIFT_MULTI_PLACEMENT_MIN,
        },
        "reason": (
            "MARKET_SHIFT: systematic drop across placements"
            if detected else
            f"Not detected — ctr_drop={ctr_delta*100:.1f}%, cvr_drop={cvr_delta*100:.1f}%, "
            f"placements={len(market_placements)}"
        ),
    }


def compute_lpv_click_gap(
    page_views: int, meta_clicks: int, threshold_ratio: float = 0.5
) -> dict:
    """LPV_CLICK_GAP: first-party PAGE_VIEW vs Meta reported clicks."""
    if meta_clicks == 0:
        return {"gap_detected": False, "ratio": None, "reason": "No Meta clicks"}
    ratio = page_views / meta_clicks
    gap = ratio < threshold_ratio
    return {
        "gap_detected": gap,
        "ratio": round(ratio, 3),
        "page_views": page_views,
        "meta_clicks": meta_clicks,
        "reason": f"LPV/click ratio {ratio:.2f} {'< ' + str(threshold_ratio) + ' → gap' if gap else '≥
threshold → OK'}",
    }
```

4.6 `ml-service/app/services/quality_service.py`

Назначение: Вычисление quality score сессии + детекция кликбейта + CVR collapse.

Математическая логика:
- **Quality score:** weighted sum с равными весами 0.25 по 4 метрикам: match_score, non_bounce, scroll_depth, form_completion.
- **Кликбейт:** P(CTR_creative > CTR_anchor) > 0.9 AND (P(CVR_creative < CVR_anchor) > 0.9 OR P(bounce_creative > bounce_anchor) > 0.9) — нормальное приближение для пропорций.
- **CVR collapse:** двухвыборочный z-тест; threshold = −30% относительного падения; $p < 0.05$.
- **P(A>B):** нормальное приближение: $z = (rate_A - rate_B) / \sqrt{(se_A^2 + se_B^2)}$; $\Phi(z)$.

Потенциальные риски:
- Нормальное приближение для пропорций некорректно при n < 30 или rate < 0.05.
- Quality score не добавляется в alpha/beta Thompson — это правильно по архитектуре, но должно быть явно задокументировано для ревьюеров.

Полный код

```
"""
Quality Score service — no FastAPI/Pydantic imports.
Computes per-session quality scores and clickbait detection.
Architecture §15.5.
"""
from __future__ import annotations

import math

DEFAULT_WEIGHTS = {
    "match_score": 0.25,
    "non_bounce": 0.25,
    "scroll_depth": 0.25,
    "form_completion": 0.25,
}

# Roadmap 8-3 (2-12 change): when the tenant's quality key is an appointment stage, whether the
# booked call was actually held carries the highest weight.
APPOINTMENT_WEIGHTS = {
    "match_score": 0.15,
    "non_bounce": 0.15,
    "scroll_depth": 0.15,
    "form_completion": 0.15,
    "show_rate": 0.40,
}


def compute_quality_score(
    match_score: float | None,
    bounced: bool,
    scroll_depth: float,
    form_completion: float,
    weights: dict | None = None,
    show_rate: float | None = None,
) -> float:
    """
    Compute quality score for a session.
    quality_score = w1*match_score + w2*(1-bounce) + w3*scroll_depth_norm + w4*form_completion
                    [+ w5*show_rate when an appointment outcome exists — roadmap 8-3]
    All inputs normalized to [0, 1]. show_rate: 1.0 held, 0.0 no-show, None = no outcome yet
    (base weights apply, so unresolved bookings are never penalised).

    Quality Score is NOT added to alpha/beta (one binary event per arm, arch §4.4).
    It acts as a protective metric and a multiplier on the Phase 3 profit estimate.
    """
    base = APPOINTMENT_WEIGHTS if show_rate is not None else DEFAULT_WEIGHTS
    w = {**base, **(weights or {})}

    ms = match_score if match_score is not None else 0.5  # neutral prior if unknown
    nb = 0.0 if bounced else 1.0
    sd = max(0.0, min(1.0, scroll_depth))
    fc = max(0.0, min(1.0, form_completion))

    score = (
        w["match_score"] * ms
        + w["non_bounce"] * nb
        + w["scroll_depth"] * sd
        + w["form_completion"] * fc
    )
    if show_rate is not None:
        score += w.get("show_rate", 0.0) * max(0.0, min(1.0, show_rate))
    return round(max(0.0, min(1.0, score)), 4)
```

```
def _prob_a_greater_b(rate_a: float, n_a: int, rate_b: float, n_b: int) -> float:
    """P(rate_a > rate_b) using normal approximation of proportions."""
    if n_a < 5 or n_b < 5:
        return 0.5
    # Normal approximation requires np≥5 and n(1-p)≥5; fall back for sparse rates (e.g. CVR≈1%)
    if (rate_a * n_a < 5 or (1 - rate_a) * n_a < 5
            or rate_b * n_b < 5 or (1 - rate_b) * n_b < 5):
        return 0.5
    se = math.sqrt(
        rate_a * (1 - rate_a) / n_a + rate_b * (1 - rate_b) / n_b
    )
    if se < 1e-10:
        return 1.0 if rate_a > rate_b else 0.0
    z = (rate_a - rate_b) / se
    return 0.5 * (1 + math.erf(z / math.sqrt(2)))


def compute_clickbait_check(
    creative_ctr: float,
    anchor_ctr: float,
    creative_cvr: float,
    anchor_cvr: float,
    creative_bounce: float,
    anchor_bounce: float,
    ctr_n: int,
    cvr_n: int,
    bounce_n: int,
    noshow_beta: float | None = None,
    noshow_beta_threshold: float = -0.3,
) -> dict:
    """
    Bayesian clickbait detector (vision §15.3 pre-LKJ heuristic).
    Flags CLICKBAIT_SUSPECT if:
      P(creative_ctr > anchor_ctr) > 0.9 AND
      (P(creative_cvr < anchor_cvr) > 0.9 OR P(creative_bounce > anchor_bounce) > 0.9
       OR the creative's hook is a no-show driver: β_noshow <= threshold — roadmap 8-3)
    Uses normal approximation for probabilities.
    """
    p_ctr_higher = _prob_a_greater_b(creative_ctr, ctr_n, anchor_ctr, ctr_n)
    p_cvr_lower = 1.0 - _prob_a_greater_b(creative_cvr, cvr_n, anchor_cvr, cvr_n)
    p_bounce_higher = _prob_a_greater_b(creative_bounce, bounce_n, anchor_bounce, bounce_n)
    noshow_driver = noshow_beta is not None and noshow_beta <= noshow_beta_threshold

    is_clickbait = p_ctr_higher > 0.9 and (p_cvr_lower > 0.9 or p_bounce_higher > 0.9 or noshow_driver)

    return {
        "suspect": is_clickbait,
        "p_ctr_higher_than_anchor": round(p_ctr_higher, 3),
        "p_cvr_lower_than_anchor": round(p_cvr_lower, 3),
        "p_bounce_higher_than_anchor": round(p_bounce_higher, 3),
        "noshow_driver": noshow_driver,
        "noshow_beta": noshow_beta,
        "verdict": "CLICKBAIT_SUSPECT" if is_clickbait else "CLEAN",
    }


def detect_cvr_collapse(
    current_cvr: float,
    baseline_cvr: float,
    n_conversions: int,
    n_sessions: int,
    threshold_drop: float = 0.3,
    baseline_n: int | None = None,
) -> dict:
    """
```

```
    CVR-collapse detector for 3-13 site health full.
    collapsed=True if relative drop >= threshold AND two-proportion z-test p < 0.05.

    Audit 6.6: the z-test used n_conversions as the baseline sample size (ZeroDivisionError on a
    full collapse, huge SE otherwise). `baseline_n` is the number of baseline sessions; when the
    caller does not know it, the current window size is assumed.
    """
    relative_drop = (baseline_cvr - current_cvr) / max(baseline_cvr, 1e-9)
    n1 = n_sessions
    n2 = baseline_n if baseline_n and baseline_n > 0 else n_sessions

    # Two-proportion z-test (normal approximation)
    if n_sessions < 10 or baseline_cvr <= 0:
        return {
            "collapsed": False,
            "relative_drop": round(relative_drop, 4),
            "p_value_approx": 1.0,
            "verdict": "INSUFFICIENT_DATA",
        }

    pooled = (current_cvr * n1 + baseline_cvr * n2) / (n1 + n2)
    se = math.sqrt(max(pooled * (1 - pooled), 0.0) * (1 / n1 + 1 / n2))
    if se < 1e-10:
        z = 0.0
    else:
        z = (baseline_cvr - current_cvr) / se

    # One-tailed p-value (H1: current < baseline)
    p_value = 1.0 - 0.5 * (1.0 + math.erf(z / math.sqrt(2)))
    collapsed = relative_drop >= threshold_drop and p_value < 0.05

    return {
        "collapsed": collapsed,
        "relative_drop": round(relative_drop, 4),
        "p_value_approx": round(p_value, 4),
        "verdict": "CVR_COLLAPSE" if collapsed else "STABLE",
    }


def aggregate_quality_scores(
    sessions: list[dict],
    match_score_by_variant: dict,
    weights: dict | None = None,
) -> dict:
    """
    Aggregate quality scores across sessions grouped by creative_id and variant_id.
    Returns {"by_creative": {cid: avg_score}, "by_variant": {vid: avg_score}}.
    """
    creative_scores: dict[str, list[float]] = {}
    variant_scores: dict[str, list[float]] = {}

    for s in sessions:
        ms = match_score_by_variant.get(str(s.get("variant_id")))
        event_count = int(s.get("event_count", 0))
        has_scroll = bool(s.get("has_scroll", False))
        bounced = event_count <= 1 and not has_scroll
        scroll = float(s.get("max_scroll_depth", 0)) / 100.0

        if s.get("converted"):
            fc = 1.0
        elif s.get("form_started"):
            fc = 0.3
        else:
            fc = 0.0
```

```
        score = compute_quality_score(ms, bounced, scroll, fc, weights)

        cid = str(s.get("creative_id")) if s.get("creative_id") else None
        vid = str(s.get("variant_id")) if s.get("variant_id") else None

        if cid:
            creative_scores.setdefault(cid, []).append(score)
        if vid:
            variant_scores.setdefault(vid, []).append(score)

    return {
        "by_creative": {k: round(sum(v) / len(v), 4) for k, v in creative_scores.items()},
        "by_variant": {k: round(sum(v) / len(v), 4) for k, v in variant_scores.items()},
    }
```

4.7 `ml-service/app/services/red_flags_service.py`

Назначение: Раннее выключение лендингов с плохим bounce rate (early kill).

Математическая логика:
- **Beta posterior:** Beta(bounced + 1, total − bounced + 1).
- **P(bounce > threshold):** Монте-Карло 5000 семплов via Gamma-based Beta sampler (random.Random(0)).
- **Kill condition:** P > 0.90.
- **Порог bounce threshold:** niche_mean + 1.5 × niche_sigma (из niche benchmark); фолбек = 0.75.
- **MIN_VISITS = 50** перед любой оценкой.
- Применяется ТОЛЬКО к лендингам, НЕ к Meta-креативам.

Взаимодействие: `RedFlagController.java`; threshold берётся из `niche_benchmark_service.py`.

Потенциальные риски:
- random.Random(0) — фиксированный seed → одинаковая последовательность для каждого вызова. Не критично для оценки вероятности, но детерминирует любые отладочные запросы.
- При MIN_VISITS=50 и conversion rate 0.03 ожидаем лишь ~1.5 конверсии → порог 50 посещений достаточен для bounce (которых много), но не для конверсий.

Полный код

```
"""
Red Flags Detector — pure logic, no FastAPI/Pydantic imports.
Early kill at 50-100 visits for landing pages ONLY (not Meta creatives).
Threshold = z-score vs niche×placement×device benchmark (vision §15.12.5).
Target FPR < 5%. Labelled 'heuristic' until niche benchmarks have sufficient data.
"""
from __future__ import annotations
from typing import Optional

MIN_VISITS_FOR_FLAG = 50
KILL_PROBABILITY_THRESHOLD = 0.90


def compute_beta_exceedance(
    successes: int,
    trials: int,
    threshold: float,
    n_samples: int = 5000,
) -> float:
    """P(bounce_rate > threshold | data) via Beta posterior Monte Carlo."""
    if trials < 1:
        return 0.0

    alpha = successes + 1.0
    beta_ = (trials - successes) + 1.0

    try:
        import random
        rng = random.Random()  # unseeded: deterministic seed hid MC variance across calls

        def beta_sample(a: float, b: float) -> float:
            # Gamma-based Beta sampler: O(1) per sample regardless of parameters
            x = rng.gammavariate(a, 1.0)
            y = rng.gammavariate(b, 1.0)
            return x / (x + y)

        exceed = sum(1 for _ in range(n_samples) if beta_sample(alpha, beta_) > threshold)
        return exceed / n_samples
    except Exception:
        observed = successes / max(trials, 1)
        return 1.0 if observed > threshold else 0.0


def get_bounce_threshold(
    niche_benchmark: Optional[dict],
    n_std_above: float = 1.5,
) -> dict:
    """
    Compute kill threshold from niche benchmark.
    Falls back to conservative universal (0.75) and labels as 'heuristic'.
    """
    if niche_benchmark is None or not niche_benchmark.get("is_published"):
        return {"threshold": 0.75, "source": "UNIVERSAL_FALLBACK", "label": "heuristic"}

    mu    = niche_benchmark.get("effect_size", 0.55)
    sigma = (niche_benchmark.get("ci_upper", 0.65) - niche_benchmark.get("ci_lower", 0.45)) / (2 * 1.96)
    threshold = min(mu + n_std_above * sigma, 0.95)

    return {"threshold": round(threshold, 4), "source": "NICHE_BENCHMARK", "label": "niche_calibrated"}


def check_red_flag(
    landing_id: str,
    bounced: int,
```

```
    total_visits: int,
    niche_benchmark: Optional[dict] = None,
    is_landing: bool = True,
) -> dict:
    """
    Check if landing should be early-killed.
    ONLY for landings — NOT Meta creatives (different auction mechanism, vision §15.12.5).
    """
    if not is_landing:
        return {
            "should_kill": False,
            "p_exceed": 0.0,
            "reason": "Red flag detection only applies to landings, not Meta creatives",
        }

    if total_visits < MIN_VISITS_FOR_FLAG:
        return {
            "should_kill": False,
            "p_exceed": 0.0,
            "visits": total_visits,
            "reason": f"Insufficient data ({total_visits} < {MIN_VISITS_FOR_FLAG} visits)",
        }

    threshold_info = get_bounce_threshold(niche_benchmark)
    threshold = threshold_info["threshold"]
    p_exceed  = compute_beta_exceedance(bounced, total_visits, threshold)
    should_kill = p_exceed > KILL_PROBABILITY_THRESHOLD

    return {
        "landing_id":   landing_id,
        "should_kill":  should_kill,
        "p_exceed":     round(p_exceed, 4),
        "threshold":    threshold,
        "bounced":      bounced,
        "total_visits": total_visits,
        "label":        threshold_info["label"],
        "source":       threshold_info["source"],
        "reason": (
            f"P(bounce > {threshold:.2f}) = {p_exceed:.3f} "
            f"{'> 0.90 → KILL' if should_kill else '≤ 0.90 → OK'} "
            f"[{threshold_info['label']}]"
        ),
    }
```

4.8 `ml-service/app/services/model_quality_service.py`

Назначение: Оценка качества ML-моделей (log-loss, Brier score, ECE).

Математическая логика:
- **Log-loss:** $-(1/n)\ \Sigma\ [y \times \log(p) + (1-y) \times \log(1-p)]$; p клипируется к [ε, 1−ε] при ε=1e-7.
- **Brier score:** $(1/n)\ \Sigma\ (p_i - y_i)^2$.
- **ECE:** см. calibration_service.py (10 бинов).
- **Пороги GOOD:** log-loss<0.40, Brier<0.15, ECE<0.10.
- **Пороги DEGRADED:** log-loss>0.65, Brier>0.25.
- **RMSE:** для регрессионных моделей (MMM, GLM): $\sqrt{(1/n\ \Sigma(y-\hat{y})^2)}$.
- Статус GOOD/ACCEPTABLE/DEGRADED по пересечению порогов.

Взаимодействие: используется для мониторинга Thompson и Propensity моделей.

Потенциальные риски:
- MIN_SAMPLE_FOR_QUALITY = 50 — очень мало для надёжной log-loss оценки.

Полный код

```
"""
ML model quality metrics — log-loss, Brier score, calibration error (Block F §8).
Pure logic — ZERO FastAPI/Pydantic imports.
"""
from __future__ import annotations
import math
from typing import Optional

MIN_SAMPLE_FOR_QUALITY = 50  # minimum samples to compute reliable metrics

LOG_LOSS_GOOD      = 0.40   # below → good calibration
LOG_LOSS_WARN      = 0.65   # above → degrade warning
BRIER_GOOD         = 0.15
BRIER_WARN         = 0.25
CALIBRATION_WARN   = 0.10   # mean absolute calibration error


def _clip_prob(p: float, eps: float = 1e-7) -> float:
    return max(eps, min(1.0 - eps, p))


def compute_log_loss(y_true: list[int], y_prob: list[float]) -> float:
    """Binary log-loss."""
    if len(y_true) != len(y_prob) or not y_true:
        return float("inf")
    total = 0.0
    for y, p in zip(y_true, y_prob):
        p = _clip_prob(p)
        total += -(y * math.log(p) + (1 - y) * math.log(1 - p))
    return total / len(y_true)


def compute_brier_score(y_true: list[int], y_prob: list[float]) -> float:
    """Brier score (mean squared probability error)."""
    if len(y_true) != len(y_prob) or not y_true:
        return float("inf")
    return sum((p - y) ** 2 for y, p in zip(y_true, y_prob)) / len(y_true)


def compute_calibration_error(
    y_true: list[int],
    y_prob: list[float],
    n_bins: int = 10,
) -> float:
    """
    Expected Calibration Error (ECE): mean absolute difference between
    predicted probability and observed fraction in each bin.
    """
    if len(y_true) != len(y_prob) or not y_true:
        return float("inf")
    bins: dict[int, list] = {i: [] for i in range(n_bins)}
    for y, p in zip(y_true, y_prob):
        bin_idx = min(int(p * n_bins), n_bins - 1)
        bins[bin_idx].append((y, p))

    ece = 0.0
    n = len(y_true)
    for bin_data in bins.values():
        if not bin_data:
            continue
        ys = [d[0] for d in bin_data]
        ps = [d[1] for d in bin_data]
        fraction_positive = sum(ys) / len(ys)
        mean_pred = sum(ps) / len(ps)
```

```
        ece += (len(bin_data) / n) * abs(fraction_positive - mean_pred)
    return round(ece, 6)


def evaluate_thompson_quality(
    y_true: list[int],
    y_prob: list[float],
) -> dict:
    """
    Full quality report for a Thompson Sampling arm's predicted CVR.
    y_true: 1 = converted, 0 = not converted
    y_prob: predicted conversion probability from alpha/(alpha+beta)
    """
    n = len(y_true)
    if n < MIN_SAMPLE_FOR_QUALITY:
        return {
            "status":  "INSUFFICIENT_DATA",
            "n":       n,
            "min_n":   MIN_SAMPLE_FOR_QUALITY,
        }

    ll  = compute_log_loss(y_true, y_prob)
    bs  = compute_brier_score(y_true, y_prob)
    ece = compute_calibration_error(y_true, y_prob)

    if ll < LOG_LOSS_GOOD and bs < BRIER_GOOD and ece < CALIBRATION_WARN:
        status = "GOOD"
    elif ll > LOG_LOSS_WARN or bs > BRIER_WARN:
        status = "DEGRADED"
    else:
        status = "ACCEPTABLE"

    return {
        "status":            status,
        "n":                 n,
        "log_loss":          round(ll, 6),
        "brier_score":       round(bs, 6),
        "calibration_error": ece,
        "thresholds": {
            "log_loss_good":     LOG_LOSS_GOOD,
            "log_loss_warn":     LOG_LOSS_WARN,
            "brier_good":        BRIER_GOOD,
            "brier_warn":        BRIER_WARN,
            "calibration_warn":  CALIBRATION_WARN,
        },
    }


def compute_rmse(y_true: list[float], y_pred: list[float]) -> float:
    """Root Mean Squared Error — for regression models (MMM, GLM)."""
    if len(y_true) != len(y_pred) or not y_true:
        return float("inf")
    mse = sum((a - b) ** 2 for a, b in zip(y_true, y_pred)) / len(y_true)
    return round(math.sqrt(mse), 6)


def assess_model_health(
    model_name: str,
    metrics: dict[str, float],
) -> dict:
    """
    Generic health assessment — returns status GOOD|ACCEPTABLE|DEGRADED
    based on model-specific thresholds.
    """
    if model_name in ("thompson", "cluster"):
```

```
        ll  = metrics.get("log_loss", float("inf"))
        bs  = metrics.get("brier_score", float("inf"))
        ece = metrics.get("calibration_error", float("inf"))
        if ll < LOG_LOSS_GOOD and bs < BRIER_GOOD and ece < CALIBRATION_WARN:
            status = "GOOD"
        elif ll > LOG_LOSS_WARN or bs > BRIER_WARN:
            status = "DEGRADED"
        else:
            status = "ACCEPTABLE"
    elif model_name in ("mmm", "glm"):
        rmse = metrics.get("rmse", float("inf"))
        # Heuristic: RMSE > 50% of mean target is degraded
        mean_target = metrics.get("mean_target", 1.0)
        ratio = rmse / max(mean_target, 1e-9)
        if ratio < 0.20:
            status = "GOOD"
        elif ratio > 0.50:
            status = "DEGRADED"
        else:
            status = "ACCEPTABLE"
    else:
        status = "UNKNOWN"

    return {"model_name": model_name, "status": status, "metrics": metrics}
```

4.9 `ml-service/app/services/segment_re_service.py`

Назначение: Байесовский частичный пулинг эффектов сегментов (аудитория × атом).

Математическая логика:
- **Shrinkage weight:** w = n / (n + 1/τ^2), TAU_SQUARED=4.0.
- **Posterior:** estimate = w × local_mean + (1−w) × global_mean.
- **SE:** √(estimate × (1−estimate) / n).
- **80% CI:** z=1.282.
- При n < 30 → полный shrinkage к global_mean.

Взаимодействие: `SegmentEffectController.java`; используется для ранжирования атомов по сегментам.

Потенциальные риски:
- TAU_SQUARED=4.0 означает очень слабый shrinkage (при n≥10 w→1.0). С τ^2=4, при n=10: w=10/(10+0.25)=0.976 → почти нет shrinkage. Эффективный shrinkage только при n<5.
- MIN_N=30 для shrinkage к prior противоречит: при n=10 shrinkage w=0.97 (почти нет), а при n=30 срабатывает полный режим.

Полный код

```
"""
Segment random effects — audience × atom interactions with partial pooling (vision §15.4 §3-1b).
Pure logic, zero FastAPI/Pydantic imports.
"""
from __future__ import annotations
import math

TAU_SQUARED = 0.25  # prior variance; precision=4 → 50% shrinkage at n≈4, meaningful pull below n≈20
# Previous value (4.0) gave w≈0.976 at n=10 — effectively disabled partial pooling
MIN_N = 30


def compute_shrinkage_weight(n_observations: int, tau_squared: float = TAU_SQUARED) -> float:
    """Weight for local estimate vs global prior. 0 = full shrinkage, 1 = no shrinkage."""
    return n_observations / (n_observations + 1.0 / tau_squared)


def compute_segment_effect(
    segment_conversions: int,
    segment_impressions: int,
    global_mean: float,
    min_n: int = MIN_N,
) -> dict:
    """
    Bayesian partial-pooling estimate for a segment's conversion rate.
    Falls back to global_mean with wide CI when n < min_n.
    """
    if segment_impressions < min_n:
        return {
            "estimate":       global_mean,
            "credible_lower": max(0.0, global_mean * 0.8),
            "credible_upper": min(1.0, global_mean * 1.2),
            "shrinkage":      1.0,
            "note":           f"n={segment_impressions} < {min_n}; using global prior",
        }

    local_mean = segment_conversions / segment_impressions
    w          = compute_shrinkage_weight(segment_impressions)
    estimate   = w * local_mean + (1.0 - w) * global_mean

    se = math.sqrt(estimate * (1.0 - estimate) / max(segment_impressions, 1))
    z  = 1.282  # 80% credible interval
    return {
        "estimate":       round(estimate, 6),
        "credible_lower": round(max(0.0, estimate - z * se), 6),
        "credible_upper": round(min(1.0, estimate + z * se), 6),
        "shrinkage":      round(1.0 - w, 4),
        "local_mean":     round(local_mean, 6),
        "n":              segment_impressions,
    }


def rank_atoms_by_segment(
    atom_segment_data: list[dict],
    global_mean: float,
) -> list[dict]:
    """
    Returns atoms ranked by shrinkage-adjusted posterior estimate per segment.
    Input: [{"atom_id", "segment", "conversions", "impressions"}]
    """
    results = []
    for row in atom_segment_data:
        effect = compute_segment_effect(
            int(row.get("conversions", 0)),
```

```
            int(row.get("impressions", 0)),
            global_mean,
        )
        results.append({
            "atom_id": row.get("atom_id"),
            "segment": row.get("segment"),
            **effect,
        })
    results.sort(key=lambda x: (-x["estimate"], str(x.get("segment", ""))))
    return results
```

СЕКЦИЯ 5 — ML и сигналы

5.1 `ml-service/app/services/cluster_service.py`

Назначение: Поведенческая кластеризация сессий (K-means) + персонализация порядка блоков.

Математическая логика:
- **Признаки (11):** session_duration, scroll_depth, scroll_velocity, block_price/proof/cta_time, form_started/submitted, is_mobile, is_repeat_visit, event_count.
- **K-means:** sklearn, n_clusters=7, random_state=42, n_init=10. StandardScaler перед кластеризацией.
- **Online assignment:** Евклидово расстояние, нормализованное по |centroid| (относительные отклонения).
- **Rule-based label:** иерархия правил (casual_bounce → form_abandoner → engaged_returner → price_seeker → proof_seeker → ready_to_buy → researcher).
- **IPS weight:** w = 1/propensity; clip [0.1, 10.0]; ESS = $(\Sigma w)^2/\Sigma w^2$.
- **Блок-ордер:** CLUSTER_BLOCK_RULES — 7 паттернов ре-упорядочивания блоков по кластеру.

Взаимодействие: признаки передаются в `propensity_service.py`; cluster_id → `decide_service.py`.

Потенциальные риски:
- Евклидово расстояние с нормализацией по |centroid| нестандартно и не эквивалентно StandardScaler-based distance. Несогласованность между fit (стандартизация) и online assign (нормализация по centroid).
- n_clusters=7 фиксировано. Без Elbow/Silhouette анализа — произвольный выбор.
- MIN для K-means: требуется минимум 7 сессий, что нереалистично мало.

Полный код

```
"""
Behavioral Cluster Service (Phase 5-1, §7.3 Этап 1).

Pure-logic functions: feature extraction, K-means fitting, online assignment.
No FastAPI imports — safe to unit-test without Pydantic.

Archetypes (5–8 per §7.3):
  price_seeker      — скроллит прямо к прайсу
  proof_seeker      — первым делом к отзывам
  ready_to_buy      — быстро к CTA, мало времени тратит
  researcher        — долго читает всё
  casual_bounce     — уходит < 10 сек, почти нет взаимодействия
  form_abandoner    — начинает форму, но не отправляет
  engaged_returner  — повторный визит, высокое вовлечение
"""
import json
from typing import Any

import numpy as np


# ─────────────────────────────────────────────────────────────────────────────
# Feature names (order must be stable — centroid serialisation depends on it)
# ─────────────────────────────────────────────────────────────────────────────

FEATURE_NAMES = [
    "session_duration_sec",   # total session length
    "scroll_depth_max",       # 0-1 max scroll fraction
    "scroll_velocity",        # scroll_depth / session_duration (fast = price_seeker)
    "block_price_time",       # seconds hovering / viewing pricing block
    "block_proof_time",       # seconds hovering / viewing proof/testimonials block
    "block_cta_time",         # seconds on CTA block
    "form_started",           # 0/1 started a form field
    "form_submitted",         # 0/1 submitted form
    "is_mobile",              # 0/1 mobile device
    "is_repeat_visit",        # 0/1 has prior session
    "event_count",            # total number of events in session
    "ticket_bucket_ord",      # roadmap 9-9 / arch §16.19: 0..4 ordinal of sessions.ticket_bucket
    "self_accredited",        # 0/1 sessions.self_accredited
]
N_FEATURES = len(FEATURE_NAMES)

_TICKET_ORDINAL = {"LT25K": 0.0, "25_50K": 1.0, "50_100K": 2.0, "100_250K": 3.0, "GT250K": 4.0}


def ticket_bucket_ordinal(bucket) -> float:
    """Ordinal 0..4 for the five investor ticket buckets; unknown/None → 0."""
    return _TICKET_ORDINAL.get(str(bucket).upper(), 0.0) if bucket else 0.0


def _investor_features(session: dict) -> tuple[float, float]:
    accredited = session.get("self_accredited")
    acc = 1.0 if accredited is True or str(accredited).lower() in ("1", "true", "t", "yes") else 0.0
    return ticket_bucket_ordinal(session.get("ticket_bucket")), acc

# Rule-based archetype labels (used when k-means has no trained model yet)
RULE_ARCHETYPES = {
    "casual_bounce":   "Случайный — ушёл в первые 10 секунд",
    "price_seeker":    "Ищущий цену — сразу скроллит к прайсу",
    "proof_seeker":    "Ищущий доказательства — фокус на отзывах",
    "ready_to_buy":    "Готовый купить — быстро к CTA",
    "researcher":      "Изучающий — долго читает всё",
    "form_abandoner":  "Начал форму, но ушёл",
    "engaged_returner": "Повторный визит с высоким вовлечением",
```

```
}


# ─────────────────────────────────────────────────────────────────────────────
# Feature extraction from raw event list
# ─────────────────────────────────────────────────────────────────────────────

def extract_features(events: list[dict], session: dict) -> np.ndarray:
    """
    Extract a fixed-length feature vector from a session's events.

    Args:
        events: list of event dicts (event_type, event_data, event_time, block_event_id)
        session: session dict (device, started_at, last_event_at, converted)

    Returns:
        np.ndarray of shape (N_FEATURES,) — float32
    """
    feat: dict[str, float] = {name: 0.0 for name in FEATURE_NAMES}
    feat["ticket_bucket_ord"], feat["self_accredited"] = _investor_features(session or {})

    if not events:
        return np.array([feat[n] for n in FEATURE_NAMES], dtype=np.float32)

    # Session duration
    try:
        from datetime import datetime
        start = _parse_ts(session.get("started_at", ""))
        end   = _parse_ts(session.get("last_event_at", ""))
        if start and end:
            feat["session_duration_sec"] = max(0.0, (end - start).total_seconds())
    except Exception:
        pass

    # Scroll signals
    scroll_events = [e for e in events if e.get("event_type") == "SCROLL"]
    if scroll_events:
        depths = [float(e.get("event_data", {}).get("scroll_depth", 0)) for e in scroll_events]
        feat["scroll_depth_max"] = max(depths) if depths else 0.0
        dur = feat["session_duration_sec"]
        feat["scroll_velocity"] = feat["scroll_depth_max"] / dur if dur > 0 else 0.0

    # Block-time signals (BLOCK_VIEW event with duration_sec in event_data)
    for ev in events:
        if ev.get("event_type") != "BLOCK_VIEW":
            continue
        data     = ev.get("event_data", {}) or {}
        dur_sec  = float(data.get("duration_sec", 0))
        block_id = str(data.get("block_type", ev.get("block_event_id", ""))).lower()
        if any(k in block_id for k in ("pric", "plan")):
            feat["block_price_time"] += dur_sec
        elif any(k in block_id for k in ("proof", "test", "review", "social")):
            feat["block_proof_time"] += dur_sec
        elif "cta" in block_id:
            feat["block_cta_time"] += dur_sec

    # Form signals
    feat["form_started"]   = 1.0 if any(e.get("event_type") == "FORM_START"   for e in events) else 0.0
    feat["form_submitted"] = 1.0 if any(e.get("event_type") == "FORM_SUBMIT"  for e in events) else 0.0

    # Device
    device = str(session.get("device", "")).lower()
    feat["is_mobile"] = 1.0 if device == "mobile" else 0.0

    # Repeat visit (crm_user_id set = returning contact)
```

```
    feat["is_repeat_visit"] = 1.0 if session.get("crm_user_id") else 0.0

    # Event count
    feat["event_count"] = float(len(events))

    return np.array([feat[n] for n in FEATURE_NAMES], dtype=np.float32)


def _parse_ts(ts: str):
    """Parse ISO timestamp string, returns datetime or None."""
    if not ts:
        return None
    try:
        from datetime import datetime, timezone
        return datetime.fromisoformat(str(ts).replace("Z", "+00:00"))
    except Exception:
        return None


# ─────────────────────────────────────────────────────────────────────────────
# K-means fitting
# ─────────────────────────────────────────────────────────────────────────────

def fit_clusters(
    feature_matrix: np.ndarray,
    n_clusters: int = 7,
    random_state: int = 42,
) -> list[dict[str, Any]]:
    """
    Fit K-means on session feature matrix.

    Args:
        feature_matrix: shape (n_sessions, N_FEATURES)
        n_clusters: 5–8 per §7.3
        random_state: for reproducibility

    Returns:
        List of centroid dicts: [{feature_name: value, ...}, ...]
    """
    from sklearn.cluster import KMeans
    from sklearn.preprocessing import StandardScaler

    if len(feature_matrix) < n_clusters:
        raise ValueError(
            f"Need at least {n_clusters} sessions to fit {n_clusters} clusters, "
            f"got {len(feature_matrix)}"
        )

    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(feature_matrix)

    km = KMeans(n_clusters=n_clusters, random_state=random_state, n_init=10)
    labels = km.fit_predict(X_scaled)

    # Unscale centroids back to interpretable values (for human inspection)
    centroids_raw = scaler.inverse_transform(km.cluster_centers_)

    # Phase 12-5: store both scaled centroids and scaler params so assign_cluster
    # can compute distances in the same (scaled) space used during fit.
    scaler_mean = scaler.mean_.tolist()
    scaler_std  = scaler.scale_.tolist()

    centroid_dicts = []
    for i, centroid in enumerate(centroids_raw):
        c = {name: float(val) for name, val in zip(FEATURE_NAMES, centroid)}
```

```
        c["_cluster_idx"]     = i
        c["_session_count"]   = int(np.sum(labels == i))
        # Store scaled centroid (in the fitted StandardScaler space)
        c["_centroid_scaled"] = km.cluster_centers_[i].tolist()
        c["_scaler_mean"]     = scaler_mean
        c["_scaler_std"]      = scaler_std
        centroid_dicts.append(c)

    return centroid_dicts


# ─────────────────────────────────────────────────────────────────────────────
# Online cluster assignment
# ─────────────────────────────────────────────────────────────────────────────

def assign_cluster(centroid_dicts: list[dict], features: np.ndarray) -> int:
    """
    Assign a single session to nearest centroid (Euclidean distance).

    Args:
        centroid_dicts: list from fit_clusters()
        features: (N_FEATURES,) array for new session

    Returns:
        cluster index (0-based)
    """
    if not centroid_dicts:
        return 0

    best_idx = 0
    best_dist = float("inf")

    # Phase 12-5: use StandardScaler params from fit_clusters so distances are
    # computed in scaled space — the same space K-means was fitted in.
    first = centroid_dicts[0]
    if "_scaler_mean" in first and "_scaler_std" in first:
        scaler_mean = np.array(first["_scaler_mean"], dtype=np.float32)
        scaler_std  = np.array(first["_scaler_std"],  dtype=np.float32)
        # Guard against zero std (constant feature)
        scaler_std  = np.where(scaler_std == 0.0, 1.0, scaler_std)
        features_scaled = (features - scaler_mean) / scaler_std

        for cd in centroid_dicts:
            centroid_scaled = np.array(cd["_centroid_scaled"], dtype=np.float32)
            dist = float(np.sum((features_scaled - centroid_scaled) ** 2))
            if dist < best_dist:
                best_dist = dist
                best_idx  = cd.get("_cluster_idx", 0)
    else:
        # Legacy / manually constructed centroid dicts without scaler info:
        # fall back to plain Euclidean distance in raw feature space.
        for cd in centroid_dicts:
            centroid_vec = np.array(
                [cd.get(name, 0.0) for name in FEATURE_NAMES], dtype=np.float32
            )
            dist = float(np.sum((features - centroid_vec) ** 2))
            if dist < best_dist:
                best_dist = dist
                best_idx  = cd.get("_cluster_idx", 0)

    return int(best_idx)


# ─────────────────────────────────────────────────────────────────────────────
# Rule-based cluster label (fallback when no trained model)
```

```
# ──────────────────────────────────────────────────────────────────────────────

def rule_based_label(features: np.ndarray) -> str:
    """Assign archetype label from rules (used pre-model or for validation)."""
    f = dict(zip(FEATURE_NAMES, features.tolist()))

    dur       = f["session_duration_sec"]
    scroll    = f["scroll_depth_max"]
    price_t   = f["block_price_time"]
    proof_t   = f["block_proof_time"]
    cta_t     = f["block_cta_time"]
    form_s    = f["form_started"]
    form_sub  = f["form_submitted"]
    repeat    = f["is_repeat_visit"]

    if dur < 10 and scroll < 0.1:
        return "casual_bounce"
    if form_s > 0 and form_sub < 1:
        return "form_abandoner"
    if repeat > 0 and dur > 60:
        return "engaged_returner"
    if price_t > proof_t and price_t > cta_t:
        return "price_seeker"
    if proof_t > price_t and proof_t > cta_t:
        return "proof_seeker"
    if cta_t > 0 and dur < 30 and scroll > 0.5:
        return "ready_to_buy"
    return "researcher"


# ──────────────────────────────────────────────────────────────────────────────
# Block-order personalisation by cluster (§5.4.1 Process 3)
# ──────────────────────────────────────────────────────────────────────────────

CLUSTER_BLOCK_RULES: dict[str, list[str]] = {
    "price_seeker":     ["pricing", "proof", "hero", "faq", "cta"],
    "proof_seeker":     ["proof", "hero", "pricing", "faq", "cta"],
    "ready_to_buy":     ["hero", "cta", "pricing", "proof", "faq"],
    "researcher":       ["hero", "problem", "proof", "pricing", "faq", "cta"],
    "casual_bounce":    ["hero", "proof", "pricing", "faq", "cta"],
    "form_abandoner":   ["hero", "proof", "pricing", "cta", "faq"],
    "engaged_returner": ["cta", "proof", "pricing", "hero", "faq"],
}


def personalise_block_order(
    default_order: list[str],
    cluster_label: str,
    allowed_block_ids: list[str] | None = None,
) -> list[str]:
    """
    Return a personalised block order based on cluster label.

    Constraints (§5.4.1):
    - Only re-orders blocks already in default_order (no new blocks generated)
    - If allowed_block_ids is provided, respects that set
    - Unknown cluster label → return default_order unchanged

    Args:
        default_order: the variant's default block_order list (event_ids OR type names)
        cluster_label: e.g. "price_seeker"
        allowed_block_ids: optional whitelist of block IDs that may be reordered

    Returns:
        Reordered list of block IDs (same length as default_order)
```

```
    """
    if not default_order or cluster_label not in CLUSTER_BLOCK_RULES:
        return default_order

    preferred_types = CLUSTER_BLOCK_RULES[cluster_label]
    blocks = list(default_order)
    if allowed_block_ids:
        blocks = [b for b in blocks if b in allowed_block_ids]
        fixed  = [b for b in default_order if b not in allowed_block_ids]
    else:
        fixed = []

    # Sort blocks by preferred type order; blocks whose type isn't in preferred list stay at end
    def sort_key(block_id: str) -> int:
        block_lower = block_id.lower()
        for i, ptype in enumerate(preferred_types):
            if ptype in block_lower:
                return i
        return len(preferred_types)

    reordered = sorted(blocks, key=sort_key)

    # Merge fixed (non-reorderable) blocks back at their original positions
    if fixed:
        result = list(default_order)
        reorder_positions = [i for i, b in enumerate(default_order) if b not in fixed]
        for pos, block in zip(reorder_positions, reordered):
            result[pos] = block
        return result

    return reordered


# ─────────────────────────────────────────────────────────────────────────────
# IPS weights for off-policy evaluation (§12.5, §7.3 Этап 3)
# ─────────────────────────────────────────────────────────────────────────────

def compute_ips_weight(
    chosen_propensity: float,
    clipping_max: float = 10.0,
    clipping_min: float = 0.1,
) -> float:
    """
    Compute clipped Inverse Propensity Score weight for off-policy evaluation.

    Formula: w = 1 / propensity, clipped to [min, max] to reduce variance.
    Required before contextual bandit goes live with real budget (§12.5).

    Args:
        chosen_propensity: P(chosen arm selected) at decision time (stored in decision_log)
        clipping_max: upper clip bound (default 10× baseline)
        clipping_min: lower clip bound (prevents division by near-zero)

    Returns:
        Clipped IPS weight
    """
    if chosen_propensity <= 0:
        return clipping_max
    raw = 1.0 / max(chosen_propensity, clipping_min)
    return float(min(raw, clipping_max))


def validate_ips_coverage(
    propensities: list[float],
    min_effective_n: int = 100,
```

```
) -> dict[str, Any]:
    """
    Validate that off-policy evaluation has sufficient coverage.

    Returns dict with: effective_n, max_weight, mean_weight, ready (bool)
    """
    if not propensities:
        return {"effective_n": 0, "max_weight": 0.0, "mean_weight": 0.0, "ready": False}

    weights = [compute_ips_weight(p) for p in propensities]
    w_arr = np.array(weights, dtype=np.float64)

    # Effective sample size: ESS = (Σw)² / Σw²
    ess = float(w_arr.sum() ** 2 / (w_arr ** 2).sum()) if (w_arr ** 2).sum() > 0 else 0.0

    return {
        "effective_n":  round(ess),
        "max_weight":   float(w_arr.max()),
        "mean_weight":  float(w_arr.mean()),
        "ready":        ess >= min_effective_n,
    }
```

5.2 `ml-service/app/services/hierarchy_prior_service.py`

Назначение: Трёхуровневая иерархия Байесовских приоров: universal → niche → client.

Математическая логика:
- **Universal priors:** CTR~N(0.02, 0.01), CVR~N(0.03, 0.02), bounce~N(0.55, 0.15).
- **Niche prior:** StudentT(ν=4, μ_niche, σ_niche) — тяжёлые хвосты для устойчивости к выбросам.
- μ_niche = μ_universal + effect_size × σ_universal
- σ_niche = max(CI_half/1.96, σ_universal/2)
- **Client posterior (Normal-Normal conjugate):**
- $\tau_{prior} = 1/\sigma_{prior}^2$, $\tau_{obs} = n/\sigma_{obs}^2$
- $\mu_{post} = (\tau_{prior} \times \mu_{prior} + \tau_{obs} \times \mu_{obs}) / (\tau_{prior} + \tau_{obs})$
- $\sigma_{post} = 1/\sqrt{(\tau_{prior} + \tau_{obs})}$
- При n < 30 → niche prior (без обновления).

Взаимодействие: используется для инициализации приоров в diagnostics, benchmark, GLM.

Потенциальные риски:
- Normal-Normal conjugate корректен только если likelihood — нормальная. Для CTR/CVR (бинарные пропорции) это приближение.
- StudentT niche prior → нормальный client posterior — смешивание семейств распределений. Формально некорректно, но на практике приемлемо как приближение.

Полный код

```
"""
Three-level prior hierarchy — pure logic, no FastAPI/Pydantic imports.
universal → niche → client.
Niche level uses StudentT(nu=4) robust priors (vision §6.4).
"""
from __future__ import annotations
import math
from typing import Optional

UNIVERSAL_PRIORS = {
    "ctr":          {"mu": 0.02,  "sigma": 0.01},
    "cvr":          {"mu": 0.03,  "sigma": 0.02},
    "bounce":       {"mu": 0.55,  "sigma": 0.15},
    "scroll_depth": {"mu": 0.45,  "sigma": 0.20},
}


def studentt_to_normal_moments(mu: float, sigma: float, nu: float = 4.0) -> tuple[float, float]:
    """
    Moment-match StudentT(nu, mu, sigma) to Normal for conjugate update approximation.

    Mean  = mu       (defined for nu > 1; we accept nu=4 default)
    Var   = sigma^2 * nu/(nu-2)  (defined for nu > 2)

    Returns (matched_mu, matched_sigma) suitable for Normal-Normal conjugate update.
    """
    # Approximation: StudentT moments matched to Normal (Phase 12-3).
    # Replace with PyMC sampling in Phase 7 MMM.
    if nu <= 2:
        # Undefined variance — inflate sigma as conservative fallback
        return mu, sigma * 3.0
    variance = sigma ** 2 * nu / (nu - 2)
    return mu, math.sqrt(variance)


def student_t_params(nu: float, mu: float, sigma: float) -> dict:
    """StudentT(ν, μ, σ) — robust prior for niche level."""
    return {"distribution": "StudentT", "nu": nu, "mu": mu, "sigma": sigma}


def build_niche_prior(
    metric: str,
    niche_effect_size: Optional[float] = None,
    niche_ci_lower: Optional[float] = None,
    niche_ci_upper: Optional[float] = None,
    nu: float = 4.0,
) -> dict:
    """
    Build StudentT(ν=4) niche prior from universal prior + niche effect size.
    Falls back to universal if no niche data (n_tenants < 5).
    """
    universal = UNIVERSAL_PRIORS.get(metric, {"mu": 0.0, "sigma": 0.1})

    if niche_effect_size is None:
        return {
            "source": "UNIVERSAL",
            "prior":  student_t_params(nu, universal["mu"], universal["sigma"]),
        }

    niche_mu = universal["mu"] + niche_effect_size * universal["sigma"]
    if niche_ci_lower is not None and niche_ci_upper is not None:
        ci_half = (niche_ci_upper - niche_ci_lower) / 2
        niche_sigma = max(ci_half / 1.96, universal["sigma"] * 0.5)
    else:
```

```
        niche_sigma = universal["sigma"]

    return {
        "source": "NICHE",
        "prior":  student_t_params(nu, niche_mu, niche_sigma),
    }


def build_client_prior(
    metric: str,
    niche_prior: dict,
    client_observations: int,
    client_mean: Optional[float] = None,
    client_std: Optional[float] = None,
) -> dict:
    """
    Build client-level posterior by updating niche prior with client data.
    Normal-Normal conjugate update. Full PyMC in Phase 7 MMM.
    """
    if client_observations < 30 or client_mean is None:
        return {
            "source":              "NICHE_PRIOR",
            "prior":               niche_prior["prior"],
            "client_observations": client_observations,
        }

    niche_mu    = niche_prior["prior"]["mu"]
    niche_sigma = niche_prior["prior"]["sigma"]

    # Approximation: StudentT moments matched to Normal (Phase 12-3).
    # Replace with PyMC sampling in Phase 7 MMM.
    if niche_prior["prior"].get("distribution") == "StudentT":
        nu = float(niche_prior["prior"].get("nu", 4.0))
        niche_mu, niche_sigma = studentt_to_normal_moments(niche_mu, niche_sigma, nu=nu)

    obs_sigma   = client_std if client_std and client_std > 0 else niche_sigma

    precision_prior = 1.0 / (niche_sigma ** 2)
    precision_obs   = client_observations / (obs_sigma ** 2)
    precision_post  = precision_prior + precision_obs
    mu_post         = (precision_prior * niche_mu + precision_obs * client_mean) / precision_post
    sigma_post      = math.sqrt(1.0 / precision_post)

    return {
        "source": "CLIENT_POSTERIOR",
        "prior": {"distribution": "Normal", "mu": round(mu_post, 6), "sigma": round(sigma_post, 6)},
        "client_observations": client_observations,
        "niche_weight":  round(precision_prior / precision_post, 3),
        "client_weight": round(precision_obs   / precision_post, 3),
    }


def get_effective_prior(
    metric: str,
    niche_benchmark: Optional[dict] = None,
    client_data: Optional[dict] = None,
    nu: float = 4.0,
) -> dict:
    """
    Entry point. Falls through: client → niche → universal.
    """
    niche_prior = build_niche_prior(
        metric,
        niche_benchmark.get("effect_size") if niche_benchmark else None,
        niche_benchmark.get("ci_lower")    if niche_benchmark else None,
```

```
        niche_benchmark.get("ci_upper")    if niche_benchmark else None,
        nu=nu,
    )
    if client_data and client_data.get("n_observations", 0) >= 30:
        return build_client_prior(
            metric, niche_prior,
            client_data["n_observations"],
            client_data.get("mean"),
            client_data.get("std"),
        )
    return niche_prior
```

5.3 `ml-service/app/services/signals_service.py`

Назначение: Классификация аномалий (MARKET/CREATIVE/TECHNICAL/FRAUD/NONE) и маршрутизация внешних сигналов.

Математическая логика:
- **Приоритет классификации:** TECHNICAL > FRAUD > MARKET > CREATIVE > NONE.
- **CPM z-score:** z = (CPM_current − μ_14d) / σ_14d; порог = 2.5.
- **Fraud threshold:** 15% подозрительных сессий.
- **Budget multiplier (DEMAND воронки):** drought_severe=1.6, drought_moderate=1.3, heat_stress=1.2; +10% за каждый urgent calendar event ≤7 дней.
- NARRATIVE воронки: бюджет не меняется (multiplier=1.0).

Взаимодействие: `AnchorController.java` передаёт anchor_verdict; результат используется `DiagnosticsController.java`.

Потенциальные риски:
- Только 5 точек минимально для CPM z-score. При 5 точках дисперсия крайне нестабильна.

Полный код

```
"""
External Signals + Baseline Anomaly Service — no FastAPI/Pydantic imports.
Vision §11.3, §8.2; arch §15.1.

Anomaly labels: MARKET / CREATIVE / TECHNICAL / FRAUD / NONE
"""
from __future__ import annotations

import math
from typing import Optional


# ── Anomaly Attribution ──────────────────────────────────────────────

def classify_anomaly(
    anchor_verdict: str,                # MARKET / CREATIVE / NONE from anchors_service
    site_health_open: bool,             # True if a site_health_incident is open
    fraud_share_pct: float,             # fraction of suspicious sessions [0, 100]
    offer_problem: bool,                # True if mmf_diagnoses shows OFFER_PROBLEM
    cpm_z_score: Optional[float],       # z-score of current CPM vs 14-day rolling
    calendar_tags: list[str],           # e.g. ["black_friday", "drought_moderate"]
    fraud_threshold_pct: float = 15.0,
    cpm_z_threshold: float = 2.5,
    news_label: str = "NEUTRAL",       # NEGATIVE / NEUTRAL / POSITIVE from /signals/news-sentiment (3-
12)
) -> dict:
    """
    Returns {verdict, evidence} per §8.2 three-row table.

    Priority: TECHNICAL > FRAUD > MARKET > CREATIVE > NONE
    """
    evidence = []

    if site_health_open:
        evidence.append("site_health_incident_open")
        return {"verdict": "TECHNICAL", "evidence": evidence}

    if fraud_share_pct >= fraud_threshold_pct:
        evidence.append(f"fraud_share={fraud_share_pct:.1f}%")
        return {"verdict": "FRAUD", "evidence": evidence}

    if anchor_verdict == "MARKET":
        evidence.append("anchors_dropped_vs_baseline")
        if cpm_z_score is not None and cpm_z_score > cpm_z_threshold:
            evidence.append(f"cpm_z={cpm_z_score:.2f}")
        if calendar_tags:
            evidence.append(f"calendar={calendar_tags}")
        if news_label == "NEGATIVE":
            evidence.append("news_sentiment=NEGATIVE")
        return {"verdict": "MARKET", "evidence": evidence}

    if anchor_verdict == "NONE" and news_label == "NEGATIVE" and cpm_z_score is not None and cpm_z_score
> cpm_z_threshold:
        # no anchor signal yet, but negative news + CPM spike is a market explanation (3-12)
        return {"verdict": "MARKET", "evidence": [f"cpm_z={cpm_z_score:.2f}",
"news_sentiment=NEGATIVE"]}

    if anchor_verdict == "CREATIVE":
        evidence.append("tests_dropped_anchors_stable")
        return {"verdict": "CREATIVE", "evidence": evidence}

    return {"verdict": "NONE", "evidence": []}
```

```
def compute_cpm_z_score(current_cpm: float, history: list[float]) -> Optional[float]:
    """Z-score of current_cpm vs 14-day rolling history."""
    if len(history) < 5:
        return None
    mean = sum(history) / len(history)
    var = sum((x - mean) ** 2 for x in history) / len(history)
    std = math.sqrt(var)
    if std < 1e-9:
        return 0.0
    return (current_cpm - mean) / std


def summarise_weather_signals(signals: list[dict]) -> dict:
    """
    Extract weather bucket and drought flag from external_signals rows.
    Returns {weather_bucket, is_drought, growing_degree_days, heat_stress}.
    """
    weather = {"weather_bucket": "normal", "is_drought": False,
               "growing_degree_days": None, "heat_stress": False}
    for s in signals:
        tags = s.get("value_tags", []) or []
        if "drought_severe" in tags or "drought_extreme" in tags:
            weather["is_drought"] = True
            weather["weather_bucket"] = "drought_severe"
        elif "drought_moderate" in tags and weather["weather_bucket"] == "normal":
            weather["is_drought"] = True
            weather["weather_bucket"] = "drought_moderate"
        if "heat_stress" in tags:
            weather["heat_stress"] = True
        if "season_start" in tags:
            weather["weather_bucket"] = weather.get("weather_bucket", "normal") + "_season_start"
        val = s.get("value_num")
        if val is not None and "growing_degree_days" in tags:
            weather["growing_degree_days"] = val
    return weather


# ─── Funnel-Aware Signal Routing (vision §19.13; arch §16.10) ──────────────

def route_signal_for_funnel(
    funnel_key: str,
    weather_role: str,          # 'NARRATIVE' | 'DEMAND'  from funnel_signal_config
    weather_summary: dict,      # output of summarise_weather_signals()
    news_tags: list[str],       # matched from tenant news_tags list
    calendar_events: list[dict],  # upcoming {event_type, event_name, days_until}
) -> dict:
    """
    Routes external signals to the correct funnel use-case.

    DEMAND  funnels (grower webinar / field events): weather_bucket drives
            a budget_multiplier: drought_severe → 1.6, drought_moderate → 1.3,
            heat_stress → 1.2, normal → 1.0.  Calendar urgency bump if
            event ≤7 days out.

    NARRATIVE funnels (investor): weather and news are context for the brief
            only — budget is unaffected.  Returns urgency_tags list for the
            brief generator.
    """
    result: dict = {"funnel_key": funnel_key, "weather_role": weather_role}
    bucket = weather_summary.get("weather_bucket", "normal")

    if weather_role == "DEMAND":
        multiplier = 1.0
        if "drought_severe" in bucket:
            multiplier = 1.6
```

```
        elif "drought_moderate" in bucket:
            multiplier = 1.3
        elif weather_summary.get("heat_stress"):
            multiplier = 1.2

        # Calendar urgency: nearest upcoming event ≤7 days adds 10 % per event
        urgent = [e for e in calendar_events if e.get("days_until", 99) <= 7]
        if urgent:
            multiplier = round(multiplier * (1.0 + 0.1 * len(urgent)), 3)

        result["budget_multiplier"] = multiplier
        result["urgent_events"]     = [e.get("event_name") for e in urgent]

    else:  # NARRATIVE
        urgency_tags: list[str] = []
        if weather_summary.get("is_drought"):
            urgency_tags.append("water_scarcity_context")
        for tag in news_tags:
            if tag in ("rate_change", "sec_reg", "agtech_funding"):
                urgency_tags.append(tag)
        close_events = [e for e in calendar_events
                        if e.get("event_type") == "round_close"
                        and e.get("days_until", 99) <= 30]
        if close_events:
            urgency_tags.append("round_close_approaching")

        result["urgency_tags"]    = urgency_tags
        result["budget_multiplier"] = 1.0   # NARRATIVE never touches budget

    return result


def extract_narrative_context(
    weather_summary: dict,
    news_tags: list[str],
    calendar_events: list[dict],
) -> dict:
    """
    Produces a brief context block for NARRATIVE (investor) funnels.
    Returns a dict the brief generator can inject into 'market_context'.

    Context lines are factual, offer-centric: they describe external conditions
    relevant to the investment opportunity, never describe the viewer.
    (Complies with META_PERSONAL_ATTR rule.)
    """
    lines: list[str] = []

    bucket = weather_summary.get("weather_bucket", "normal")
    if "drought_severe" in bucket:
        lines.append("Severe drought conditions are increasing AgTech investment urgency.")
    elif "drought_moderate" in bucket:
        lines.append("Moderate drought conditions are driving interest in water-efficient solutions.")

    for tag in news_tags:
        if tag == "agtech_funding":
            lines.append("AgTech funding activity has been elevated in recent weeks.")
        elif tag == "water_policy":
            lines.append("Recent water-policy developments are shaping the regulatory landscape.")
        elif tag == "rate_change":
            lines.append("Recent rate movements have shifted the risk-return calculus for private
placements.")
        elif tag == "sec_reg":
            lines.append("Regulatory activity in the securities space warrants monitoring.")

    close_events = sorted(
```

```
        [e for e in calendar_events
         if e.get("event_type") == "round_close" and e.get("days_until", 99) <= 30],
        key=lambda e: e.get("days_until", 99),
    )
    if close_events:
        nearest = close_events[0]
        days = nearest.get("days_until", "?")
        lines.append(f"The offering's round closes in approximately {days} days.")

    return {
        "context_lines": lines,
        "weather_bucket": bucket,
        "news_tags_matched": news_tags,
        "has_round_close_urgency": bool(close_events),
    }
```

5.4 `ml-service/app/services/burnout_service.py`

Назначение: Быстрое (без выживаемости) вычисление burnout score креатива.

Математическая логика:

```
score = 0.6 × ctr_drop_pct + 0.4 × frequency_penalty
frequency_penalty = clip((frequency − 1.5) / 6.5, [0, 1])
```

- critical: score > 0.7; warning: score > 0.35; none: иначе.
- **Estimate days remaining:** 0.35 / ctr_drop_rate_per_day; clip [1, 90].

Взаимодействие: вызывается из `burnout_survival_service.py` как fallback при N<30.

Потенциальные риски:
- frequency_penalty: базовая frequency=1.5 — если реклама показывается 2 раза в день, penalty уже 0.077. Это агрессивно для brand awareness кампаний.
- estimate_days_remaining: линейная экстраполяция CTR drop rate. CTR обычно падает нелинейно.

Полный код

```
"""Creative Burnout Detection Service (Phase 6-1, §8.1).

Pure-logic functions: burnout score, classification, days-remaining estimate.
No FastAPI imports — safe to unit-test without Pydantic.
"""
from typing import Optional


def compute_burnout_score(
    frequency: float,
    ctr_current: float,
    ctr_baseline: float,
    spend_current: float = 0.0,
    spend_prev: float = 0.0,
) -> dict:
    """Compute creative burnout score [0-1] from frequency and CTR drop."""
    ctr_drop_pct = (ctr_baseline - ctr_current) / ctr_baseline if ctr_baseline > 0 else 0.0
    ctr_drop_pct = max(0.0, ctr_drop_pct)
    frequency_penalty = min(1.0, max(0.0, (frequency - 1.5) / 6.5))
    score = min(1.0, max(0.0, 0.6 * ctr_drop_pct + 0.4 * frequency_penalty))
    stage = classify_burnout(score)
    signals = []
    if ctr_drop_pct > 0.3:
        signals.append(f"CTR dropped {ctr_drop_pct * 100:.0f}% from baseline")
    if frequency > 4.0:
        signals.append(f"High frequency: {frequency:.1f}")
    if frequency > 2.0 and ctr_drop_pct > 0.15:
        signals.append("Frequency rising while CTR declining")
    return {"score": round(score, 4), "stage": stage, "signals": signals}


def classify_burnout(score: float) -> str:
    """Classify burnout stage from score."""
    if score > 0.7:
        return "critical"
    if score > 0.35:
        return "warning"
    return "none"


def estimate_days_remaining(frequency: float, ctr_drop_rate_per_day: float) -> int:
    """Estimate days until creative reaches warning-stage burnout (score > 0.35)."""
    if ctr_drop_rate_per_day <= 0:
        return 90
    days = int(0.35 / ctr_drop_rate_per_day)
    return max(1, min(90, days))
```

5.5 `ml-service/app/services/saturation_service.py`

Назначение: Детекция насыщения аудитории vs. усталости от креатива vs. сдвига рынка.

Математическая логика:

```
eff_score = min(1, (1 − reach_efficiency) / baseline_efficiency)
freq_score = min(1, (frequency − frequency_prev) / 5.0)
score = 0.5 × eff_score + 0.5 × freq_score
```

- critical: score > 0.65; warning: score > 0.30.
- **Диагностика:** creative_fatigue если CTR упал без падения reach; audience_saturation если reach упал + high_freq; market_shift если reach упал без high_freq.

Взаимодействие: `SaturationController.java`; входные данные — метрики из Meta API.

Потенциальные риски:
- freq_score нормализован на 5.0 (подъём частоты). При исторически более высоком подъёме score будет занижен.
- Разделение на усталость/насыщение/сдвиг рынка — пороги CTR < −15% и reach < 0.70 произвольны.

Полный код

```
"""Audience Saturation Detection Service (Phase 6-2, §8.2).

Detects d(Reach)/d(Spend) decline and differentiates creative fatigue
from true audience saturation.
"""
from typing import Optional


def compute_reach_efficiency(
    reach_current: float,
    reach_prev: float,
    spend_current: float,
    spend_prev: float,
) -> float:
    """Compute current reach-per-dollar relative to previous period."""
    if spend_current <= 0 or spend_prev <= 0:
        return 1.0
    eff_current = reach_current / spend_current
    eff_prev    = reach_prev / spend_prev
    if eff_prev <= 0:
        return 1.0
    return eff_current / eff_prev


def classify_saturation(
    reach_efficiency: float,
    frequency: float,
    frequency_prev: float,
    baseline_efficiency: float = 1.0,
) -> dict:
    """Classify saturation stage using efficiency drop and frequency rise."""
    eff_drop    = max(0.0, baseline_efficiency - reach_efficiency)
    eff_score   = min(1.0, eff_drop / baseline_efficiency) if baseline_efficiency > 0 else 0.0

    freq_rise   = max(0.0, frequency - frequency_prev)
    freq_score  = min(1.0, freq_rise / 5.0)

    score = 0.5 * eff_score + 0.5 * freq_score

    if score > 0.65:
        stage = "critical"
    elif score > 0.30:
        stage = "warning"
    else:
        stage = "healthy"

    return {
        "score":            round(score, 4),
        "stage":            stage,
        "reach_efficiency": round(reach_efficiency, 4),
        "eff_score":        round(eff_score, 4),
        "freq_score":       round(freq_score, 4),
    }


def differentiate_diagnosis(
    ctr_change_pct: float,
    reach_efficiency: float,
    frequency: float,
) -> str:
    """Distinguish creative fatigue from audience saturation or market shift.

    Returns one of: 'creative_fatigue' | 'audience_saturation' | 'market_shift' | 'healthy'
    """
```

```
    ctr_dropped   = ctr_change_pct < -0.15
    reach_dropped = reach_efficiency < 0.70
    high_freq     = frequency > 4.0

    if ctr_dropped and not reach_dropped:
        return "creative_fatigue"
    if reach_dropped and high_freq:
        return "audience_saturation"
    if reach_dropped and not high_freq:
        return "market_shift"
    return "healthy"
```

5.6 `ml-service/app/services/entropy_service.py`

Назначение: Оценка разнообразия контента через косинусное подобие эмбеддингов.

Математическая логика:

```
entropy = 1 - mean_cosine_similarity(всех пар хуков)
```

- Порог: entropy < 0.3 при ≥5 тенантах → флаг LOW_DIVERSITY.
- AGGREGATE_FLOOR = 5 тенантов (чтобы не раскрывать данные одного тенанта).

Взаимодействие: `EntropyController.java`; эмбеддинги генерируются llm_client.

Потенциальные риски:
- 1 − mean_cosine_similarity не является информационной энтропией. Это мера разнообразия, но назвать "entropy" может вводить в заблуждение математиков.
- При 5 тенантах mean similarity из 10 пар — достаточно? Математически: нет для стабильной оценки.

Полный код

```
"""
Niche Creative Entropy Index service — no FastAPI/Pydantic imports.
Prevents "tragedy of the commons" when all tenants in a niche run similar hooks.
Uses cosine similarity from atom_registry embeddings (1-6).
Architecture §15.10, roadmap 6-8.
"""
from __future__ import annotations

import math
from typing import Optional

_ENTROPY_THRESHOLD = 0.3   # below this → diversity_flag = True
_MIN_TENANTS       = 5     # need at least 5 tenants per niche before scoring


def _cosine(v1: list[float], v2: list[float]) -> float:
    """Cosine similarity between two vectors."""
    dot = sum(a * b for a, b in zip(v1, v2))
    norm1 = math.sqrt(sum(a * a for a in v1))
    norm2 = math.sqrt(sum(b * b for b in v2))
    if norm1 == 0 or norm2 == 0:
        return 0.0
    return dot / (norm1 * norm2)


def compute_pairwise_similarities(embeddings: list[list[float]]) -> list[float]:
    """
    Compute all unique pairwise cosine similarities for N embeddings.
    Returns flat list of N*(N-1)/2 similarity values.
    """
    n = len(embeddings)
    sims = []
    for i in range(n):
        for j in range(i + 1, n):
            sims.append(_cosine(embeddings[i], embeddings[j]))
    return sims


def compute_entropy_from_similarities(similarities: list[float]) -> float:
    """
    Entropy = 1 - mean_cosine_similarity.
    Range [0, 1]: 1.0 = fully diverse, 0.0 = all identical hooks.
    """
    if not similarities:
        return 1.0  # no data → assume full diversity
    mean_sim = sum(similarities) / len(similarities)
    return round(1.0 - mean_sim, 4)


def check_diversity_penalty(
    entropy: float,
    tenant_count: int,
    niche: str,
) -> dict:
    """
    Determine if a diversity penalty should be applied.
    Requirements: >= MIN_TENANTS per niche; entropy < ENTROPY_THRESHOLD.
    Returns {diversity_flag, penalty_reason, recommendation}.
    """
    if tenant_count < _MIN_TENANTS:
        return {
            "diversity_flag": False,
            "diversity_insufficient_tenants": True,
            "tenant_count": tenant_count,
```

```
            "min_tenants_required": _MIN_TENANTS,
            "recommendation": f"Need >= {_MIN_TENANTS} tenants in niche '{niche}' before scoring",
        }

    diversity_flag = entropy < _ENTROPY_THRESHOLD

    result: dict = {
        "diversity_flag":   diversity_flag,
        "entropy":          entropy,
        "threshold":        _ENTROPY_THRESHOLD,
        "tenant_count":     tenant_count,
        "niche":            niche,
    }

    if diversity_flag:
        result["penalty_reason"] = (
            f"Entropy {entropy:.3f} < threshold {_ENTROPY_THRESHOLD}: "
            "too many tenants in this niche are using semantically-similar hooks"
        )
        result["recommendation"] = (
            "Rotate hooks: prioritise candidates with low cosine similarity to current niche centroid; "
            "see 6-4 Evolutionary Algorithm diversity ranking"
        )
    else:
        result["recommendation"] = "Diversity is healthy — no rotation required"

    return result


def build_entropy_report(
    niche: str,
    embeddings: list[list[float]],
    tenant_count: int,
) -> dict:
    """
    Full entropy report for a niche.
    embeddings: one representative hook embedding per active tenant.
    """
    if tenant_count < _MIN_TENANTS:
        return {
            "niche":          niche,
            "tenant_count":   tenant_count,
            "entropy":        None,
            "mean_similarity": None,
            "diversity_flag": False,
            "skipped_reason": f"Need >= {_MIN_TENANTS} tenants (have {tenant_count})",
        }

    sims = compute_pairwise_similarities(embeddings)
    entropy = compute_entropy_from_similarities(sims)
    mean_sim = round(1.0 - entropy, 4)
    penalty = check_diversity_penalty(entropy, tenant_count, niche)

    return {
        "niche":            niche,
        "tenant_count":     tenant_count,
        "entropy":          entropy,
        "mean_similarity":  mean_sim,
        "diversity_flag":   penalty["diversity_flag"],
        "recommendation":   penalty.get("recommendation"),
        "pair_count":       len(sims),
    }
```

5.7 `ml-service/app/services/niche_benchmark_service.py`

Назначение: Первичные бенчмарки по нише (p50, p75 метрик).

Математическая логика:
- AGGREGATE_FLOOR = 5 тенантов на ячейку (ниша × размещение × устройство).
- Процентили: numpy percentile по реализованным метрикам.
- Сравнение: above (>p50+0.02), below (<p50−0.02), at_median.
- ТОЛЬКО первичные события (first-party) — Meta API данные запрещены до юридической проверки.

Взаимодействие: `NicheBenchmarkController.java`; threshold для red_flags берётся отсюда; niche prior → `hierarchy_prior_service.py`.

Полный код

```
"""
Niche Benchmarks — first-party only (vision §8; arch §16.10).
Aggregate floor: ≥5 tenants per niche cell (vision §25 p.19).
Pure logic — zero FastAPI/Pydantic imports.
"""
from __future__ import annotations
from collections import defaultdict


AGGREGATE_FLOOR = 5


def compute_niche_benchmarks(
    tenant_funnel_data: list[dict],
    data_window_days: int = 30,
) -> list[dict]:
    """
    Computes benchmark percentiles per niche.
    Returns list of benchmark rows (value=None when tenant_count < AGGREGATE_FLOOR).

    Metrics: show_rate_p50, show_rate_p75, form_to_booked_p50, booking_distance_h_p50.
    Source: ONLY first-party events — never Meta Platform data (§25 p.19).
    """
    niche_data: dict[str, list[dict]] = defaultdict(list)
    for row in tenant_funnel_data:
        niche = row.get("niche_key") or "generic"
        niche_data[niche].append(row)

    benchmarks = []
    for niche_key, rows in niche_data.items():
        n = len(rows)
        if n < AGGREGATE_FLOOR:
            benchmarks.append({
                "niche_key":        niche_key,
                "metric_key":       "show_rate_p50",
                "value":            None,
                "tenant_count":     n,
                "data_window_days": data_window_days,
                "reason":           f"Below aggregate floor ({n} < {AGGREGATE_FLOOR})",
            })
            continue

        # Show rates
        show_rates = []
        for r in rows:
            booked = r.get("booked") or 0
            held   = r.get("held")   or 0
            if booked > 0:
                show_rates.append(held / booked)

        if show_rates:
            show_rates.sort()
            p50 = show_rates[min(int(len(show_rates) * 0.50), len(show_rates) - 1)]
            p75 = show_rates[min(int(len(show_rates) * 0.75), len(show_rates) - 1)]
            benchmarks.append({
                "niche_key": niche_key, "metric_key": "show_rate_p50",
                "value": round(p50, 4), "tenant_count": n, "data_window_days": data_window_days,
            })
            benchmarks.append({
                "niche_key": niche_key, "metric_key": "show_rate_p75",
                "value": round(p75, 4), "tenant_count": n, "data_window_days": data_window_days,
            })

        # Form-to-booked
```

```
        f2b_rates = []
        for r in rows:
            forms  = r.get("form_submits") or 0
            booked = r.get("booked")       or 0
            if forms > 0:
                f2b_rates.append(booked / forms)

        if f2b_rates:
            f2b_rates.sort()
            p50 = f2b_rates[min(int(len(f2b_rates) * 0.50), len(f2b_rates) - 1)]
            benchmarks.append({
                "niche_key": niche_key, "metric_key": "form_to_booked_p50",
                "value": round(p50, 4), "tenant_count": n, "data_window_days": data_window_days,
            })

        # Booking distance
        distances = [r.get("avg_booking_distance_h") for r in rows if r.get("avg_booking_distance_h") is
not None]
        if distances:
            distances.sort()
            p50 = distances[min(int(len(distances) * 0.50), len(distances) - 1)]
            benchmarks.append({
                "niche_key": niche_key, "metric_key": "booking_distance_h_p50",
                "value": round(p50, 2), "tenant_count": n, "data_window_days": data_window_days,
            })

    return benchmarks


def compare_tenant_to_niche(
    tenant_metrics: dict,
    benchmarks: list[dict],
) -> dict:
    """
    Returns relative performance: above/below/at benchmark per metric.
    Only for visible benchmarks (value is not None).
    """
    result = {}
    bm_map = {b["metric_key"]: b["value"] for b in benchmarks if b.get("value") is not None}

    metric_map = {
        "show_rate_p50":          "show_rate",
        "form_to_booked_p50":     "form_to_booked",
        "booking_distance_h_p50": "avg_booking_distance_h",
    }

    for metric_key, bm_value in bm_map.items():
        tenant_field = metric_map.get(metric_key)
        if not tenant_field:
            continue
        tenant_value = tenant_metrics.get(tenant_field)
        if tenant_value is None:
            continue

        diff = float(tenant_value) - float(bm_value)
        result[metric_key] = {
            "tenant_value":    round(float(tenant_value), 4),
            "benchmark_value": round(float(bm_value), 4),
            "diff":            round(diff, 4),
            "performance":     "above" if diff > 0.02 else ("below" if diff < -0.02 else "at"),
        }

    return result
```

5.8 `ml-service/app/services/evolution_service.py`

Назначение: Эволюционная генерация креативов (crossover + mutation + MMR diversification).

Математическая логика:
- **Crossover:** 50/50 — для каждого типа атома случайно берётся от parent_a или parent_b.
- **Mutation:** каждый атом независимо заменяется с P=0.15 на случайный из pool того же типа.
- **Coherence check:** cosine(hook_emb, body_emb) ≥ 0.65.
- **Graveyard similarity:** cosine(candidate, failed) ≥ 0.85 → флаг (по аудитории).
- **GLM posterior ranking:** семплы из N(μ_posterior, σ_posterior); ranking по E[sample].
- **MMR diversification:** λ=0.3; score = (1−λ)×relevance + λ×(1−max_sim_to_selected).

Взаимодействие: `EvolveController.java`; atom_pool берётся из approved_atoms; graveyard из `GraveyardController.java`.

Потенциальные риски:
- При одном parent_a и parent_b (len(ranked)==1) crossover вернёт копию одного родителя → все кандидаты идентичны до мутации.
- MMR: если у кандидатов нет поля "embedding", _cosine возвращает 0 (деление на 0 обработано), но MMR score будет некорректным.

Полный код

```
"""Evolutionary Creative Generation Service (Phase 6-4 + v1.1).

Crossover + mutation of creative atoms from top-performing arms.
v1.1 additions: coherence check, graveyard similarity, GLM posterior ranking,
graveyard re-review (vision §14.2, §13.2, §15.8 task 1).
No FastAPI/Pydantic imports — pure logic, safe to unit-test.
"""
from __future__ import annotations
import math
import random
from typing import Dict, List, Optional


def rank_creatives_by_cvr(arms: List[Dict]) -> List[Dict]:
    """Sort arms by expected CVR = alpha / (alpha + beta), descending."""
    def expected_cvr(arm: Dict) -> float:
        a = arm.get("alpha", 1.0)
        b = arm.get("beta", 1.0)
        return a / (a + b) if (a + b) > 0 else 0.0

    return sorted(arms, key=expected_cvr, reverse=True)


def crossover_atoms(
    parent_a: Dict,
    parent_b: Dict,
    rng_seed: Optional[int] = None,
) -> List[Dict]:
    """50/50 crossover: for each unique atom_type, pick from parent_a or parent_b.

    parent_a / parent_b: {"atoms": [{"atom_type": ..., ...}, ...]}
    Returns list of child atoms.
    """
    rng = random.Random(rng_seed)
    atoms_a: List[Dict] = parent_a.get("atoms", [])
    atoms_b: List[Dict] = parent_b.get("atoms", [])

    by_type_a = {a["atom_type"]: a for a in atoms_a}
    by_type_b = {a["atom_type"]: a for a in atoms_b}
    all_types = set(by_type_a) | set(by_type_b)

    child = []
    for atom_type in sorted(all_types):
        candidates = [
            x for x in [by_type_a.get(atom_type), by_type_b.get(atom_type)]
            if x is not None
        ]
        if candidates:
            child.append(rng.choice(candidates))
    return child


def mutate_atoms(
    atoms: List[Dict],
    atom_pool: List[Dict],
    mutation_rate: float = 0.15,
    rng_seed: Optional[int] = None,
) -> List[Dict]:
    """Replace each atom independently with probability mutation_rate,
    drawing replacement from atom_pool of the same atom_type.

    Atoms with no pool entry are kept as-is.
    """
    rng = random.Random(rng_seed)
```

```
    pool_by_type: Dict[str, List[Dict]] = {}
    for p in atom_pool:
        pool_by_type.setdefault(p["atom_type"], []).append(p)

    mutated = []
    for atom in atoms:
        t = atom["atom_type"]
        if rng.random() < mutation_rate and t in pool_by_type:
            mutated.append(rng.choice(pool_by_type[t]))
        else:
            mutated.append(atom)
    return mutated


def generate_candidates(
    top_arms: List[Dict],
    atom_pool: List[Dict],
    n_candidates: int = 5,
    mutation_rate: float = 0.15,
) -> List[Dict]:
    """Generate n_candidates new creative proposals from top-2 parents.

    Each candidate: crossover of top-2, then mutate.
    Falls back to single-parent mutation if only one arm is available.
    """
    ranked = rank_creatives_by_cvr(top_arms)
    if not ranked:
        return []

    parent_a = ranked[0]
    single_parent = len(ranked) < 2
    parent_b = ranked[1] if not single_parent else ranked[0]

    candidates = []
    for i in range(n_candidates):
        if single_parent:
            # Crossover between identical parents produces an exact copy — skip it, mutate only
            child_atoms = list(parent_a.get("atoms", []))
        else:
            child_atoms = crossover_atoms(parent_a, parent_b, rng_seed=i)
        mutated = mutate_atoms(child_atoms, atom_pool, mutation_rate, rng_seed=i + 1000)
        candidates.append({
            "candidate_index": i,
            "atoms":           mutated,
            "parent_ids":      [parent_a.get("id"), parent_b.get("id") if not single_parent else None],
        })
    return candidates


# ── v1.1 additions ──────────────────────────────────────────────────

def _cosine(a: list, b: list) -> float:
    dot    = sum(x * y for x, y in zip(a, b))
    norm_a = math.sqrt(sum(x * x for x in a))
    norm_b = math.sqrt(sum(x * x for x in b))
    denom  = norm_a * norm_b
    return dot / denom if denom > 0 else 0.0


def check_hook_body_coherence(
    hook_embedding: list,
    body_embedding: list,
    threshold: float = 0.65,
) -> dict:
    """
```

```
    Semantic coherence between hook and body via cosine similarity.
    Below threshold → DROP_OR_EDIT (vision §14.2 step 2).
    """
    sim = _cosine(hook_embedding, body_embedding)
    coherent = sim >= threshold
    return {
        "coherent":   coherent,
        "similarity": round(sim, 4),
        "threshold":  threshold,
        "action":     "PASS" if coherent else "DROP_OR_EDIT",
        "reason":     f"Hook-body cosine={sim:.3f} {'≥' if coherent else '<'} {threshold}",
    }


def check_graveyard_similarity(
    candidate_embedding: list,
    graveyard_embeddings: list,
    candidate_audience: str,
    theta: float = 0.85,
) -> dict:
    """
    Flag candidate if cosine similarity ≥ theta with any failed atom combination
    for the same audience (vision §13.2).
    """
    if not graveyard_embeddings:
        return {"flagged": False, "reason": "Empty graveyard", "matches": []}

    matches = []
    for entry in graveyard_embeddings:
        if entry.get("audience", "").upper() != candidate_audience.upper():
            continue
        emb = entry.get("embedding", [])
        if not emb:
            continue
        sim = _cosine(candidate_embedding, emb)
        if sim >= theta:
            matches.append({"atom_combo": entry.get("atom_combo", "UNKNOWN"), "similarity": round(sim,
4)})

    flagged = len(matches) > 0
    return {
        "flagged":  flagged,
        "theta":    theta,
        "matches":  matches,
        "reason": (f"Similar to {len(matches)} failed combo(s) at θ≥{theta}"
                   if flagged else f"No graveyard matches at θ≥{theta}"),
    }


def rank_by_glm_posterior(
    candidates: list,
    n_samples: int = 1000,
) -> list:
    """
    Rank candidates by GLM posterior predictive CVR, not raw CVR alone.
    vision §14.2 step 5.
    """
    if not candidates:
        return []

    rng = random.Random(42)
    scored = []
    for c in candidates:
        mu    = float(c.get("posterior_mean", 0.0))
        sigma = float(c.get("posterior_std", 0.01))
```

```
        samples  = [rng.gauss(mu, sigma) for _ in range(n_samples)]
        expected = sum(samples) / n_samples
        scored.append({**c, "ranking_score": round(expected, 6)})

    return sorted(scored, key=lambda x: x["ranking_score"], reverse=True)


def should_retry_graveyard_entry(failure_context: dict) -> dict:
    """
    Flag graveyard entry as retry candidate if failure was external:
    MARKET verdict, fraud, or site health incident (vision §13.2).
    """
    anchor = failure_context.get("anchor_verdict", "NONE")
    fraud  = failure_context.get("fraud_flag", False)
    health = failure_context.get("site_health_incident", False)

    external = anchor in ("MARKET", "MARKET_SHIFT") or fraud or health
    return {
        "retry_candidate": external,
        "reason": (
            f"Failure attributed to external cause: anchor={anchor}, fraud={fraud}, site_incident=
{health}"
            if external
            else "Failure appears intrinsic — no retry recommended"
        ),
    }


# ── DIVERSITY PENALTY — Maximum Marginal Relevance (vision §22) ──────────────

DIVERSITY_LAMBDA = 0.3   # 0 = pure relevance, 1 = pure diversity


def apply_diversity_penalty(
    candidates: list[dict],
    lambda_: float = DIVERSITY_LAMBDA,
) -> list[dict]:
    """
    MMR re-ranking: balances GLM posterior score with embedding diversity.
    Prevents top-5 list with identical hooks (vision §22).
    Each candidate: {"score": float, "embedding": list[float], ...}
    """
    if not candidates:
        return []

    selected: list[dict] = []
    remaining = list(candidates)

    while remaining:
        best: dict | None = None
        best_score = -1e9

        for cand in remaining:
            relevance = float(cand.get("score", cand.get("ranking_score", 0.0)))
            if not selected:
                diversity_bonus = 1.0
            else:
                sims = [
                    _cosine(cand.get("embedding", []), s.get("embedding", []))
                    for s in selected
                ]
                diversity_bonus = 1.0 - max(sims)

            mmr = (1.0 - lambda_) * relevance + lambda_ * diversity_bonus
            if mmr > best_score:
```

```
                best_score = mmr
                best       = cand

        if best is None:
            break
        selected.append(dict(best, mmr_score=round(best_score, 6)))
        remaining.remove(best)

    return selected
```

5.9 `ml-service/app/services/budget_curve_service.py`

Назначение: Монте-Карло предсказание CPA corridor по уровням бюджета.

Математическая логика:

```
cvr_sample ~ Beta(α, β)
sessions = budget × sessions_per_dollar
conversions = cvr_sample × sessions
CPA = budget / conversions  (∞ если conversions=0)
```

- p10, p50, p90 из distribution of CPA, исключая Inf.
- **Diminishing returns:** первый бюджетный уровень, где CPA_p50 растёт >20% от предыдущего.
- n_mc=2000 per budget point; rng_seed=i для воспроизводимости.

Взаимодействие: `PredictiveController.java`; α, β берутся из Thompson arms для основного варианта.

Потенциальные риски:
- sessions = budget × sessions_per_dollar предполагает линейную зависимость трафика от бюджета — некорректно при насыщении аудитории. Нужна Hill-кривая.
- Исключение Inf из CPA — если 90%+ семплов имеют CVR≈0, p50 тоже Inf → вернёт None. Нет обработки этого случая в caller.

Полный код

```
"""Predictive Budget Curve Service (Phase 6-3, §8.4).

Monte Carlo simulation over Beta(alpha, beta) CVR distribution to produce
CPA corridors at each budget level.
"""
import warnings
from typing import List, Optional
import numpy as np


def _hill_sessions(budget: float, sessions_max: float, ec50: float, k: float = 1.0) -> float:
    """Hill curve: sessions = sessions_max * budget^k / (ec50^k + budget^k)"""
    if budget <= 0:
        return 0.0
    bk = budget ** k
    return sessions_max * bk / (ec50 ** k + bk)


def simulate_budget_step(
    alpha: float,
    beta: float,
    sessions_per_dollar: float,
    budget: float,
    n_mc: int = 2000,
    rng_seed: Optional[int] = None,
    max_budget: Optional[float] = None,
) -> dict:
    """Sample CVR from Beta(alpha, beta) n_mc times, compute CPA distribution.

    Returns p10, p50, p90 of CPA at the given budget.
    sessions_per_dollar: expected traffic volume per $ spent.
    max_budget: when provided alongside sessions_per_dollar, enables Hill saturation
    model (ec50 = max_budget * 0.3, sessions_max = sessions_per_dollar * max_budget * 0.8).
    Falls back to linear with saturation_model='linear_fallback' when data is insufficient.
    """
    rng = np.random.default_rng(rng_seed)
    if alpha <= 0 or beta <= 0:
        alpha, beta = 1.0, 1.0

    saturation_model = "linear"
    if sessions_per_dollar is not None and max_budget is not None and max_budget > 0:
        ec50 = max_budget * 0.3
        sessions_max = sessions_per_dollar * max_budget * 0.8
        sessions = _hill_sessions(budget, sessions_max, ec50)
        saturation_model = "hill"
    else:
        sessions = budget * sessions_per_dollar
        saturation_model = "linear_fallback"

    cvr_samples    = rng.beta(alpha, beta, size=n_mc)
    conversions    = cvr_samples * sessions

    with np.errstate(divide="ignore", invalid="ignore"):
        cpa_samples = np.where(conversions > 0, budget / conversions, np.inf)

    finite_mask = np.isfinite(cpa_samples)
    if finite_mask.sum() == 0:
        return {"budget": budget, "cpa_p10": None, "cpa_p50": None, "cpa_p90": None,
                "saturation_model": saturation_model}

    cpa_finite = cpa_samples[finite_mask]
    return {
        "budget":  round(float(budget), 2),
        "cpa_p10": round(float(np.percentile(cpa_finite, 10)), 4),
```

```
        "cpa_p50": round(float(np.percentile(cpa_finite, 50)), 4),
        "cpa_p90": round(float(np.percentile(cpa_finite, 90)), 4),
        "saturation_model": saturation_model,
    }


def compute_budget_curve(
    alpha: float,
    beta: float,
    sessions_per_dollar: float,
    budget_points: List[float],
    n_mc: int = 2000,
) -> List[dict]:
    """Compute budget curve across multiple budget points."""
    return [
        simulate_budget_step(alpha, beta, sessions_per_dollar, b, n_mc, rng_seed=i)
        for i, b in enumerate(budget_points)
    ]


def find_diminishing_returns_point(curve: List[dict]) -> Optional[float]:
    """Return the first budget where CPA p50 increases >20% vs the last valid step.

    Skips None entries rather than resetting the comparison baseline.
    Returns None if no such point found (curve is still efficient).
    """
    prev_cpa = None
    for step in curve:
        curr = step.get("cpa_p50")
        if curr is None:
            continue
        if prev_cpa is not None and prev_cpa > 0 and (curr - prev_cpa) / prev_cpa > 0.20:
            return step["budget"]
        prev_cpa = curr
    return None
```

СЕКЦИЯ 6 — Compliance Math

6.1 `ml-service/app/services/compliance_service.py`

Назначение: Проверка текстов на соответствие Meta Advertising Policies и правилам SEC.

Математическая логика:
- Regex-based правила: каждое правило имеет pattern, severity (FAIL/WARN), suggestion.
- BIC-like приоритизация: FAIL > WARN > PASS.
- **META_PERSONAL_ATTR:** 6 паттернов (PA001–PA005) — описание предложения, а не зрителя.
- Trademark: точное совпадение (case-insensitive).
- LLM проверка (claude-haiku-5-5) для META_POLICY и CLAIMS — асинхронная, с фолбеком на WARN при ошибке.

Взаимодействие: `ModerationController.java`; rulebases хранятся в compliance_rulebases; LLM вызывается через `llm_client.py`.

Потенциальные риски:
- Regex для юридических правил — ненадёжен. Paraphrase нарушений не детектируется.
- LLM-based проверка: модель меняется со временем → результаты могут измениться при обновлении

модели.
- При llm_client=None возвращается WARN, а не FAIL → систематический downgrade severity.

Полный код

```
"""
Compliance check service — no FastAPI/Pydantic imports.
Checks landing page text against Meta policies, product facts, trademark watchlist,
and securities-specific rulebases (vision §19.5 / arch §16.5).
"""
from __future__ import annotations

import json
import re
from typing import Any


def check_meta_policy(text: str, llm_client: Any, model: str = "claude-haiku-5-5") -> dict:
    """
    Check text against Meta Advertising Policies.
    Returns {result: PASS|WARN|FAIL, findings: [{rule, excerpt, severity, suggestion}]}
    """
    if llm_client is None:
        return {
            "result": "FAIL",
            "findings": [{"rule": "ML_UNAVAILABLE", "excerpt": "", "severity": "FAIL",
                          "suggestion": "Manual policy review required — LLM unavailable, cannot confirm
compliance"}],
        }

    prompt = (
        "Review this marketing text for Meta Advertising Policy violations.\n\n"
        f"Text:\n{text}\n\n"
        "Check for these violations:\n"
        "1. Before/after claims (prohibited): 'lost 20 lbs', 'made $10k'\n"
        "2. Health outcome guarantees: 'guaranteed to cure/treat/heal'\n"
        "3. Financial outcome guarantees: 'guaranteed returns/profits'\n"
        "4. Personal attribute targeting: implies the reader has a negative characteristic\n"
        "5. Misleading claims: false urgency, hidden fees, bait-and-switch\n"
        "6. Prohibited content: weapons, drugs, discrimination\n\n"
        "Respond ONLY with JSON:\n"
        '{"result": "PASS", "findings": []}\n'
        "or with violations:\n"
        '{"result": "FAIL", "findings": [{"rule": "before_after_claims", '
        '"excerpt": "exact text", "severity": "FAIL", "suggestion": "Rephrase as..."}]}'
    )
    try:
        response = llm_client.complete(prompt, model=model)
        data = json.loads(response)
        return {"result": data.get("result", "PASS"), "findings": data.get("findings", [])}
    except Exception:
        return {
            "result": "WARN",
            "findings": [{"rule": "PARSE_ERROR", "excerpt": "", "severity": "WARN",
                          "suggestion": "Manual review required"}],
        }


def check_claims_vs_facts(text: str, product_facts: list[str], llm_client: Any, model: str = "claude-
haiku-5-5") -> dict:
    """
    Check factual claims in text against the product fact base.
    Unsupported → WARN, contradicted → FAIL.
    """
    if not product_facts:
        return {
            "result": "PASS",
            "findings": [{"rule": "NO_FACTS", "excerpt": "", "severity": "WARN",
                          "suggestion": "Add product facts to enable claims verification"}],
```

```
        }

    if llm_client is None:
        return {
            "result": "FAIL",
            "findings": [{"rule": "ML_UNAVAILABLE", "excerpt": "", "severity": "FAIL",
                          "suggestion": "Manual claims verification required — LLM unavailable, cannot
confirm compliance"}],
        }

    facts_str = "\n".join(f"- {f}" for f in product_facts)

    prompt = (
        "Compare the claims in this marketing text against the verified product facts.\n\n"
        f"Marketing text:\n{text}\n\n"
        f"Verified product facts:\n{facts_str}\n\n"
        "Rules:\n"
        "- Claim directly supported by facts → OK\n"
        "- Claim not mentioned in facts → WARN (UNSUPPORTED_CLAIM)\n"
        "- Claim contradicts facts → FAIL (CONTRADICTED_CLAIM)\n\n"
        "Respond ONLY with JSON:\n"
        '{"result": "PASS", "findings": []}\n'
        "or with issues:\n"
        '{"result": "WARN", "findings": [{"rule": "UNSUPPORTED_CLAIM", '
        '"excerpt": "exact claim", "severity": "WARN", '
        '"suggestion": "Add supporting fact or rephrase"}]}'
    )
    try:
        response = llm_client.complete(prompt, model=model)
        data = json.loads(response)
        return {"result": data.get("result", "PASS"), "findings": data.get("findings", [])}
    except Exception:
        return {"result": "WARN", "findings": []}


def check_trademarks(text: str, trademark_terms: list[str]) -> dict:
    """Exact + case-insensitive match of trademark terms in text."""
    if not trademark_terms:
        return {"result": "PASS", "findings": []}

    findings = []
    text_lower = text.lower()
    for term in trademark_terms:
        if term.lower() in text_lower:
            findings.append({
                "rule": "TRADEMARK_DETECTED",
                "excerpt": term,
                "severity": "WARN",
                "suggestion": f"Remove or obtain licence for '{term}'",
            })

    return {"result": "WARN" if findings else "PASS", "findings": findings}


def check_text_against_rules(text: str, rules: list[dict]) -> dict:
    """
    Regex-based rule matching against a list of rulebase rule objects.
    Each rule: {rule_id, severity, description, pattern, suggestion}.
    Returns {result: PASS|WARN|FAIL, findings: [...]}.
    """
    findings = []
    text_lower = text.lower()

    for rule in rules:
        pattern = rule.get("pattern", "")
```

```
        if not pattern:
            continue
        try:
            if re.search(pattern, text_lower, re.IGNORECASE):
                findings.append({
                    "rule_id": rule.get("rule_id", "UNKNOWN"),
                    "severity": rule.get("severity", "WARN"),
                    "description": rule.get("description", ""),
                    "suggestion": rule.get("suggestion", ""),
                })
        except re.error:
            # Fallback to plain string match if regex is malformed
            if pattern.lower() in text_lower:
                findings.append({
                    "rule_id": rule.get("rule_id", "UNKNOWN"),
                    "severity": rule.get("severity", "WARN"),
                    "description": rule.get("description", ""),
                    "suggestion": rule.get("suggestion", ""),
                })

    severities = [f["severity"] for f in findings]
    if "FAIL" in severities:
        result = "FAIL"
    elif "WARN" in severities:
        result = "WARN"
    else:
        result = "PASS"

    return {"result": result, "findings": findings}


def run_compliance_check(
    text: str,
    rulebases: dict[str, list[dict]],
) -> dict:
    """
    Run text against multiple named rulebases.
    rulebases: {rulebase_key: [rule, ...]}
    Returns {overall: PASS|WARN|FAIL, per_rulebase: {key: {result, findings}}}.
    """
    per_rulebase: dict[str, dict] = {}
    overall_severities: list[str] = []

    for key, rules in rulebases.items():
        check = check_text_against_rules(text, rules)
        per_rulebase[key] = check
        overall_severities.append(check["result"])

    if "FAIL" in overall_severities:
        overall = "FAIL"
    elif "WARN" in overall_severities:
        overall = "WARN"
    else:
        overall = "PASS"

    return {"overall": overall, "per_rulebase": per_rulebase}


# ─── Personal attribute check (vision §19.5: describe offer, not viewer) ─────

_PERSONAL_ATTR_PATTERNS = [
    (r"are you.{0,40}accredited", "FAIL", "PA001", "Replace with: 'Open to accredited investors'"),
    (r"you (are|must be).{0,40}accredited", "FAIL", "PA001b", "Replace with offer-description
language"),
    (r"(if you|you have|struggling with).{0,60}(debt|financial|money problem)", "FAIL", "PA002",
```

```
     "Describe solution benefits, not viewer hardship"),
    (r"for (high.net.worth|wealthy|rich)", "FAIL", "PA003", "State eligibility criteria without
targeting"),
    (r"do you (qualify|earn|make|have \$)", "WARN", "PA004", "Rephrase to describe offer eligibility"),
    (r"your (portfolio|investments).{0,60}(underperform|failing|losing)", "FAIL", "PA005",
     "Do not imply negative viewer situation"),
]


def check_meta_personal_attr(text: str) -> dict:
    """
    Check text for personal attribute targeting violations.
    Returns {result: PASS|WARN|FAIL, findings: [...]}.
    """
    text_lower = text.lower()
    findings = []

    for pattern, severity, rule_id, suggestion in _PERSONAL_ATTR_PATTERNS:
        try:
            if re.search(pattern, text_lower, re.IGNORECASE):
                findings.append({
                    "rule_id": rule_id,
                    "severity": severity,
                    "description": "Personal attribute targeting violation (§19.5)",
                    "suggestion": suggestion,
                })
        except re.error:
            pass

    severities = [f["severity"] for f in findings]
    result = "FAIL" if "FAIL" in severities else ("WARN" if "WARN" in severities else "PASS")
    return {"result": result, "findings": findings}


def is_from_approved_phrases(text: str, approved_phrases: list[str]) -> dict:
    """
    Check whether text is contained within or matches an approved phrase.
    Returns {approved: bool, matched_phrase: str|None}.
    """
    text_stripped = text.strip().lower()
    for phrase in approved_phrases:
        if phrase.strip().lower() in text_stripped or text_stripped in phrase.strip().lower():
            return {"approved": True, "matched_phrase": phrase}
    return {"approved": False, "matched_phrase": None}


def run_all_text_checks(
    text: str,
    product_facts: list[str],
    trademark_terms: list[str],
    llm_client: Any,
    model: str = "claude-haiku-5-5",
) -> dict:
    """
    Run META_POLICY + CLAIMS + TRADEMARK checks and return combined result.
    Severity: FAIL overrides WARN overrides PASS.
    """
    policy = check_meta_policy(text, llm_client, model)
    claims = check_claims_vs_facts(text, product_facts, llm_client, model)
    tm = check_trademarks(text, trademark_terms)

    all_findings = (
        [{"check_type": "META_POLICY", **f} for f in policy["findings"]]
        + [{"check_type": "CLAIMS", **f} for f in claims["findings"]]
        + [{"check_type": "TRADEMARK", **f} for f in tm["findings"]]
```

```
    )

    results = [policy["result"], claims["result"], tm["result"]]
    if "FAIL" in results:
        overall = "FAIL"
    elif "WARN" in results:
        overall = "WARN"
    else:
        overall = "PASS"

    return {
        "result": overall,
        "findings": all_findings,
        "checks": {
            "meta_policy": policy["result"],
            "claims": claims["result"],
            "trademark": tm["result"],
        },
    }
```

6.2 `ml-service/app/services/moderation_service.py`

Назначение: Compliance by Construction + META_REVIEWER persona score.

Математическая логика:
- **Сборка из одобренных фраз:** только approved_phrases → если совпадение не найдено → new_phrases (требует юридического ревью).
- **Coverage:** 1 − (len(new_phrases) / max(total_chars, 1) × 10) — эвристика, не формула покрытия.
- **P(rejection):** `p_reject = 1 - (1-0.85)^fail_count × (1-0.20)^warn_count`
- Предполагает независимость правил; каждый FAIL добавляет 85% вероятность отклонения.
- ADVISORY ONLY — никогда не блокирует запуск.

Взаимодействие: `ModerationController.java`; persona META_REVIEWER сеется при инициализации.

Потенциальные риски:
- P(rejection) formula предполагает независимые нарушения — нереалистично. Два связанных FAIL считаются как $1-0.15^2 = 0.977$, но это тот же контент.
- coverage = 1 − (len(new_phrases)/chars × 10) — делитель 10 произвольный.

Полный код

```
"""
Compliance by Construction + Moderation Probe + Reviewer Persona (vision §19.18.4; arch §16.17).
Pure logic — zero FastAPI/Pydantic imports.
"""
from __future__ import annotations
import re


def compose_from_approved(
    slots: dict,
    approved_phrases: list[dict],
    context_filter: str | None = None,
) -> dict:
    """
    Fill content slots from approved phrases only.
    Unmatched text → new_phrases[] needing legal review.
    """
    approved_by_context: dict = {}
    for ap in approved_phrases:
        ctx = ap.get("context")
        approved_by_context.setdefault(ctx, []).append(ap.get("phrase", ""))
        approved_by_context.setdefault(None, []).append(ap.get("phrase", ""))

    available       = approved_by_context.get(context_filter, []) + approved_by_context.get(None, [])
    available_lower = {p.lower(): p for p in available if p}

    filled_slots = {}
    new_phrases: list[str] = []

    for slot_key, desired_text in slots.items():
        if not desired_text:
            filled_slots[slot_key] = ""
            continue

        sentences   = [s.strip() for s in re.split(r"[.!?]+", desired_text) if s.strip()]
        slot_parts  = []
        needs_review = []

        for sentence in sentences:
            matched = next(
                (ap for ap_lower, ap in available_lower.items()
                 if ap_lower in sentence.lower() or sentence.lower() in ap_lower),
                None,
            )
            if matched:
                slot_parts.append(matched)
            else:
                slot_parts.append(sentence)
                needs_review.append(sentence)

        filled_slots[slot_key] = ". ".join(slot_parts)
        new_phrases.extend(needs_review)

    total_chars = sum(len(s) for s in slots.values())
    coverage = max(0.0, min(1.0, 1.0 - (len(new_phrases) / max(total_chars, 1) * 10)))

    return {
        "filled_slots": filled_slots,
        "new_phrases":  new_phrases,
        "needs_legal":  len(new_phrases) > 0,
        "coverage":     round(coverage, 3),
    }
```

```
def score_with_persona(
    text: str,
    persona: dict,
    compliance_results: dict,
) -> dict:
    """
    META_REVIEWER persona: pre-verdict P(Meta rejection).
    Advisory only — never blocks a launch.
    """
    persona_key = persona.get("persona_key", "UNKNOWN")
    findings    = compliance_results.get("findings", [])

    fail_count = sum(1 for f in findings if f.get("action") == "FAIL")
    warn_count = sum(1 for f in findings if f.get("action") == "WARN")

    p_reject = 1.0 - (1.0 - 0.85) ** fail_count * (1.0 - 0.20) ** warn_count
    p_reject = min(0.99, p_reject)

    verdict = (
        "LIKELY_REJECTED" if p_reject > 0.7
        else ("NEEDS_REVIEW" if p_reject > 0.3 else "LIKELY_APPROVED")
    )

    return {
        "persona_key": persona_key,
        "p_reject":    round(p_reject, 4),
        "verdict":     verdict,
        "fail_count":  fail_count,
        "warn_count":  warn_count,
        "note":        "Advisory only — never blocks a launch; compare weekly vs moderation_results",
    }


def check_moderation_probe_needed(
    is_new_creative: bool,
    budget_usd_day:  float,
    skip_probe:      bool = False,
) -> dict:
    """Recommend $5-10/day probe before real budget for new creatives."""
    if skip_probe:
        return {"probe_recommended": False, "reason": "probe skipped by caller"}
    if is_new_creative and budget_usd_day > 10:
        return {
            "probe_recommended": True,
            "probe_budget_day":  min(10.0, budget_usd_day * 0.1),
            "reason":            "New creative + real budget → probe first",
        }
    return {"probe_recommended": False, "reason": "Existing creative or low budget"}
```

6.3 `ml-service/app/services/geo_lift_service.py`

Назначение: Синтетический контроль для измерения инкрементального лифта по географии.

Математическая логика:

```
scale = mean(test_pre) / mean(control_pre)
synthetic_post = scale × mean(control_post)
lift = mean(test_post) − synthetic_post
lift_pct = lift / synthetic_post
```

- **Bootstrap CI (80%):** n=5000 бутстреп по временным рядам; α = (1−0.80)/2 = 0.10.

- **P(lift > 0):** доля bootstrap итераций с lift > 0.
- seed=42, MC_SAMPLES=5000.
- ТОЛЬКО на клиентских данных продаж, НЕ pixel данных (vision §17.3).

Взаимодействие: `GeoLiftController.java`; данные о продажах по геозонам.

Потенциальные риски:
- Простой synthetic control через scale ratio — слабая модель. Классический DiD или SDID был бы более точным.
- Требует ≥2 control geos. При 2 control geos bootstrap крайне нестабилен.
- Bootstrap семплирует независимо pre и post — нарушает временну́ю зависимость внутри рядов.

Полный код

```
"""
Geo-Lift service — no FastAPI/Pydantic imports.
Synthetic-control incrementality on client sales data (NOT pixel).
Credible intervals on all figures (vision §17.3).
Architecture §15.21, roadmap 5-7.
"""
from __future__ import annotations

import math
import numpy as np

_RNG = np.random.default_rng(seed=42)
_MC_SAMPLES = 5_000
_CI_LEVEL = 0.80  # 80% credible interval


def _block_bootstrap_sample(arr: np.ndarray, block_size: int = None) -> np.ndarray:
    """Block bootstrap preserving temporal autocorrelation."""
    n = len(arr)
    if block_size is None:
        block_size = max(7, int(np.sqrt(n)))  # weekly blocks by default
    n_blocks = int(np.ceil(n / block_size))
    starts = _RNG.integers(0, n, size=n_blocks)
    indices = np.concatenate([np.arange(s, min(s + block_size, n)) for s in starts])[:n]
    return arr[indices]


def validate_geo_split(
    control_geos: list[str],
    test_geos: list[str],
) -> dict:
    """
    Validate that geo lists are non-empty, non-overlapping, and have at least 2 geos each.
    Returns {valid: bool, errors: list[str]}.
    """
    errors = []

    if not control_geos:
        errors.append("control_geos must not be empty")
    if not test_geos:
        errors.append("test_geos must not be empty")

    overlap = set(control_geos) & set(test_geos)
    if overlap:
        errors.append(f"Overlapping geos: {sorted(overlap)}")

    if len(control_geos) < 2:
        errors.append("At least 2 control geos are needed for a reliable synthetic control")
    if len(test_geos) < 1:
        errors.append("At least 1 test geo is required")

    return {"valid": len(errors) == 0, "errors": errors}


def compute_synthetic_control_simple(
    pre_period_control: list[float],
    pre_period_test: list[float],
    post_period_control: list[float],
    post_period_test: list[float],
) -> dict:
    """
    Simple synthetic control: scale ratio from pre-period.
    scale = mean(test_pre) / mean(control_pre)
    synthetic_test_post = scale * mean(control_post)
```

```
    lift = mean(test_post) - synthetic_test_post
    lift_pct = lift / synthetic_test_post

    Credible interval via bootstrap on the pre-period scaling residuals.
    Returns {lift_estimate, lift_pct, ci_lower, ci_upper, ci_level,
             p_positive, synthetic_counterfactual, method}.
    """
    if not pre_period_control or not pre_period_test or not post_period_control or not post_period_test:
        return {
            "lift_estimate": None,
            "error": "Empty input series",
            "method": "synthetic_control_simple",
        }

    mean_control_pre  = float(np.mean(pre_period_control))
    mean_test_pre     = float(np.mean(pre_period_test))
    mean_control_post = float(np.mean(post_period_control))
    mean_test_post    = float(np.mean(post_period_test))

    if mean_control_pre == 0:
        return {
            "lift_estimate": None,
            "error": "control pre-period mean is zero; cannot compute scale ratio",
            "method": "synthetic_control_simple",
        }

    scale = mean_test_pre / mean_control_pre
    synthetic_post = scale * mean_control_post
    lift = mean_test_post - synthetic_post
    lift_pct = lift / synthetic_post if synthetic_post != 0 else 0.0

    # Bootstrap credible interval on the lift estimate
    control_pre_arr  = np.array(pre_period_control, dtype=float)
    test_pre_arr     = np.array(pre_period_test, dtype=float)
    control_post_arr = np.array(post_period_control, dtype=float)
    test_post_arr    = np.array(post_period_test, dtype=float)

    n_ctrl_pre  = len(control_pre_arr)
    n_test_pre  = len(test_pre_arr)
    n_ctrl_post = len(control_post_arr)
    n_test_post = len(test_post_arr)

    bootstrap_lifts = []
    for _ in range(_MC_SAMPLES):
        s_ctrl_pre  = _block_bootstrap_sample(control_pre_arr)
        s_test_pre  = _block_bootstrap_sample(test_pre_arr)
        s_ctrl_post = _block_bootstrap_sample(control_post_arr)
        s_test_post = _block_bootstrap_sample(test_post_arr)

        m_ctrl_pre  = s_ctrl_pre.mean()
        if m_ctrl_pre == 0:
            continue
        s_scale     = s_test_pre.mean() / m_ctrl_pre
        s_synthetic = s_scale * s_ctrl_post.mean()
        bootstrap_lifts.append(float(s_test_post.mean() - s_synthetic))

    if not bootstrap_lifts:
        ci_lower = ci_upper = lift
        p_positive = 1.0 if lift > 0 else 0.0
    else:
        alpha = (1 - _CI_LEVEL) / 2
        ci_lower  = float(np.quantile(bootstrap_lifts, alpha))
        ci_upper  = float(np.quantile(bootstrap_lifts, 1 - alpha))
        p_positive = float(np.mean(np.array(bootstrap_lifts) > 0))
```

```
    return {
        "lift_estimate":          round(lift, 4),
        "lift_pct":               round(lift_pct * 100, 2),
        "ci_lower":               round(ci_lower, 4),
        "ci_upper":               round(ci_upper, 4),
        "ci_level":               _CI_LEVEL,
        "p_positive":             round(p_positive, 4),
        "synthetic_counterfactual": round(synthetic_post, 4),
        "scale_ratio":            round(scale, 4),
        "method":                 "synthetic_control_simple",
        "note":                   "Uses client sales data, not pixel. Honest noise per §17.3.",
    }
```

СЕКЦИЯ 7 — Agency & Calibration

7.1 `ml-service/app/services/agency_service.py`

Назначение: Агрегация KPI по тенантам агентства + попарное сравнение источников креативов.

Математическая логика:
- **Beta CI для источников:** α = held+1; β = (lpv−held)+1; mean = α/(α+β); SE = √(mean×(1−mean)/max(lpv,1)).
- **80% CI:** ±1.282 × SE.
- **P(A>B):** нормальное приближение: z = (mean_A − mean_B) / √(se_A^2+se_B^2); Φ(z) через math.erf.

Взаимодействие: `AgencyController.java`; данные из agency_kpi_snapshots.

Потенциальные риски:
- SE используется из CI (ci_80_upper − mean ≈ 1.282×SE) — восстановление SE из округлённого CI вносит ошибку округления.
- Нормальное приближение для P(A>B) некорректно при малых lpv или extreme hold rate.

Полный код

```
"""
Agency Measurability (vision §19.18.7; arch §16.20).
Pure logic — zero FastAPI/Pydantic imports.

Aggregates KPIs and creative-source performance across an agency's tenants.
No tenant data is mixed — only aggregate totals flow to agency layer.
"""
from __future__ import annotations
import math


def aggregate_agency_kpis(
    tenant_snapshots: list[dict],
) -> dict:
    """
    Aggregate KPIs across tenants for agency dashboard.
    """
    totals: dict[str, int] = {"lpv": 0, "held_calls": 0, "committed": 0}
    cph_values: list[float] = []

    for snap in tenant_snapshots:
        totals["lpv"]        += int(snap.get("lpv") or 0)
        totals["held_calls"] += int(snap.get("held_calls") or 0)
        totals["committed"]  += int(snap.get("committed") or 0)
        if snap.get("cph_usd"):
            cph_values.append(float(snap["cph_usd"]))

    n = len(tenant_snapshots) or 1
    avg_cph = sum(cph_values) / len(cph_values) if cph_values else None
    held = totals["held_calls"] or 1
    commit_rate = round(totals["committed"] / held, 4)

    return {
        "tenant_count":     n,
        "total_lpv":        totals["lpv"],
        "total_held_calls": totals["held_calls"],
        "total_committed":  totals["committed"],
        "avg_cph_usd":      round(avg_cph, 2) if avg_cph is not None else None,
        "commit_rate":      commit_rate,
    }


def rank_tenants_by_performance(
    tenant_snapshots: list[dict],
    metric: str = "held_calls",
) -> list[dict]:
    """
    Rank agency tenants by a given metric (descending).
    """
    ranked = sorted(
        tenant_snapshots,
        key=lambda x: float(x.get(metric) or 0),
        reverse=True,
    )
    for i, snap in enumerate(ranked):
        snap["rank"] = i + 1
    return ranked


def compare_creative_sources(
    source_rows: list[dict],
) -> dict:
    """
    P(source A better than source B) on cost per held call.
```

```
    Uses Beta posterior on held-call rate: alpha = held+1, beta = (lpv-held)+1.
    Returns pairwise comparison for the agency contractor report.
    """
    results: list[dict] = []
    for row in source_rows:
        lpv   = int(row.get("lpv") or 0)
        held  = int(row.get("held_calls") or 0)
        alpha = held + 1
        beta  = (lpv - held) + 1
        mean  = alpha / (alpha + beta)
        se    = math.sqrt(mean * (1 - mean) / max(lpv, 1))
        results.append({
            "source_id":    row.get("source_id"),
            "source_name":  row.get("source_name"),
            "kind":         row.get("kind"),
            "lpv":          lpv,
            "held_calls":   held,
            "hold_rate":    round(mean, 4),
            "ci_80_lower":  round(max(0, mean - 1.282 * se), 4),
            "ci_80_upper":  round(min(1, mean + 1.282 * se), 4),
        })
    results.sort(key=lambda x: -x["hold_rate"])

    # Pairwise P(A > B) using Beta mean difference approximation
    comparisons: list[dict] = []
    for i in range(len(results)):
        for j in range(i + 1, len(results)):
            a, b = results[i], results[j]
            # Approximate P(A > B) as Φ((mean_A - mean_B) / sqrt(se_A^2 + se_B^2))
            diff = a["hold_rate"] - b["hold_rate"]
            se_a = (a["ci_80_upper"] - a["hold_rate"]) / 1.282  # recover se: CI_upper = mean + 1.282·se
            se_b = (b["ci_80_upper"] - b["hold_rate"]) / 1.282
            pooled_se = math.sqrt(max(se_a**2 + se_b**2, 1e-9))
            z = diff / pooled_se
            # Φ(z) approximation
            p_a_better = 0.5 * (1 + math.erf(z / math.sqrt(2)))
            comparisons.append({
                "source_a": a["source_name"],
                "source_b": b["source_name"],
                "p_a_better_than_b": round(p_a_better, 4),
            })

    return {"sources": results, "pairwise": comparisons}
```

7.2 `ml-service/app/services/archetype_service.py`

Назначение: Классификация инвесторов по архетипам (6 типов).

Математическая логика:
- Rule-based scoring: суммирование очков по признакам для каждого архетипа.
- DISQUALIFIED: +0.7 если не аккредитован, +0.3 если LT25K.
- COMMITTED_QUICK: +0.8 при COMMITTED outcome, +0.3 при fast form (<3h), +0.3 при high ticket.
- RESEARCHER: +0.4 при ≥3 material views, +0.3 dataroom, +0.2 slow form (>24h), +0.2 high touch.
- FENCE_SITTER: +0.4×matched_objections, +0.3 repeated touches, +0.2 HIGH noshow risk.
- HIGH_VALUE_QUIET: +0.6 при high ticket + 0 material views + ≤2 touches, +0.3 no objections + GT250K.
- confidence = min(1.0, best_score); UNKNOWN если best_score < 0.3.

Взаимодействие: `InvestorArchetypeController.java`; вызывается на CALL_HELD событие.

Потенциальные риски:
- Очки не нормированы между архетипами → DISQUALIFIED (max=1.0) vs HIGH_VALUE_QUIET (max=0.9) —

близкие значения, но семантически разные.
- Возможно одновременное высокое score у DISQUALIFIED и COMMITTED_QUICK (non-accredited + committed call). DISQUALIFIED выиграет только если score 1.0 vs 0.8.

Полный код

```
"""
Investor Archetype Classifier (vision §19.18.6; arch §16.19).
Pure logic — zero FastAPI/Pydantic imports.

Rule-based until sufficient labelled data for ML.
Seeded labels: COMMITTED_QUICK / RESEARCHER / FENCE_SITTER / HIGH_VALUE_QUIET / DISQUALIFIED / UNKNOWN
"""
from __future__ import annotations


def classify_archetype(
    ticket_bucket:     str | None,
    self_accredited:   bool | None,
    time_to_form_h:    float | None,
    material_views:    int,
    dataroom_opens:    int,
    call_outcome:      str | None,
    objection_tags:    list[str],
    noshow_risk_tier:  str | None,
    touch_count:       int,
) -> dict:
    """
    Classify lead into investor archetype.
    Returns: {archetype, confidence, scores, signals}
    """
    signals: list[str] = []
    scores: dict[str, float] = {
        "COMMITTED_QUICK":  0.0,
        "RESEARCHER":       0.0,
        "FENCE_SITTER":     0.0,
        "HIGH_VALUE_QUIET": 0.0,
        "DISQUALIFIED":     0.0,
    }

    # DISQUALIFIED
    if self_accredited is False:
        scores["DISQUALIFIED"] += 0.7
        signals.append("NOT_ACCREDITED")
    if ticket_bucket == "LT25K":
        scores["DISQUALIFIED"] += 0.3
        signals.append("LOW_TICKET")

    # COMMITTED_QUICK
    if call_outcome == "COMMITTED":
        scores["COMMITTED_QUICK"] += 0.8
        signals.append("COMMITTED_ON_CALL")
    if time_to_form_h is not None and time_to_form_h < 3:
        scores["COMMITTED_QUICK"] += 0.3
        signals.append("FAST_FORM_SUBMIT")
    if ticket_bucket in ("GT250K", "100_250K") and self_accredited:
        scores["COMMITTED_QUICK"] += 0.3
        signals.append("HIGH_TICKET_ACCREDITED")

    # RESEARCHER
    if material_views >= 3:
        scores["RESEARCHER"] += 0.4
        signals.append("MULTI_MATERIAL_VIEW")
    if dataroom_opens >= 1:
        scores["RESEARCHER"] += 0.3
        signals.append("DATAROOM_OPENED")
    if time_to_form_h is not None and time_to_form_h > 24:
        scores["RESEARCHER"] += 0.2
        signals.append("SLOW_FORM_SUBMIT")
    if touch_count >= 5:
```

```
        scores["RESEARCHER"] += 0.2
        signals.append("HIGH_TOUCH_COUNT")

    # FENCE_SITTER
    fence_objections = {"TIMING_NOT_RIGHT", "PRICE_CONCERN", "SPOUSE_APPROVAL"}
    matched_fence = fence_objections & set(objection_tags)
    if matched_fence:
        scores["FENCE_SITTER"] += 0.4 * len(matched_fence)
        signals.extend(sorted(matched_fence))
    if touch_count >= 3 and call_outcome in (None, "INTERESTED_FOLLOW"):
        scores["FENCE_SITTER"] += 0.3
        signals.append("REPEATED_TOUCHES_NO_COMMIT")
    if noshow_risk_tier == "HIGH":
        scores["FENCE_SITTER"] += 0.2
        signals.append("HIGH_NOSHOW_RISK")

    # HIGH_VALUE_QUIET
    if ticket_bucket in ("GT250K", "100_250K") and material_views == 0 and touch_count <= 2:
        scores["HIGH_VALUE_QUIET"] += 0.6
        signals.append("HIGH_TICKET_LOW_ENGAGEMENT")
    if ticket_bucket == "GT250K" and not objection_tags:
        scores["HIGH_VALUE_QUIET"] += 0.3
        signals.append("NO_OBJECTIONS_HIGH_TICKET")

    best = max(scores, key=lambda k: scores[k])
    best_score = scores[best]

    if best_score < 0.3:
        archetype, confidence = "UNKNOWN", 0.3
    else:
        archetype  = best
        confidence = min(1.0, best_score)

    return {
        "archetype":  archetype,
        "confidence": round(confidence, 4),
        "scores":     {k: round(v, 4) for k, v in scores.items()},
        "signals":    list(dict.fromkeys(signals)),  # dedup, preserve first order
    }


def archetype_follow_up_strategy(archetype: str) -> dict:
    """
    Recommended follow-up strategy per archetype.
    """
    strategies: dict[str, dict] = {
        "COMMITTED_QUICK": {
            "priority":   "URGENT",
            "action":     "Send wire instructions + schedule confirmation call",
            "ghl_tag":    "archetype_committed_quick",
            "sla_hours":  2,
        },
        "RESEARCHER": {
            "priority":   "HIGH",
            "action":     "Send full data room + schedule Q&A call",
            "ghl_tag":    "archetype_researcher",
            "sla_hours":  12,
        },
        "FENCE_SITTER": {
            "priority":   "NORMAL",
            "action":     "Targeted objection-handling sequence",
            "ghl_tag":    "archetype_fence_sitter",
            "sla_hours":  24,
        },
        "HIGH_VALUE_QUIET": {
```

```
            "priority":   "HIGH",
            "action":     "Founder personal outreach",
            "ghl_tag":    "archetype_high_value_quiet",
            "sla_hours":  6,
        },
        "DISQUALIFIED": {
            "priority":   "LOW",
            "action":     "Remove from active sequence; add to waitlist if Reg CF opens",
            "ghl_tag":    "archetype_disqualified",
            "sla_hours":  None,
        },
        "UNKNOWN": {
            "priority":   "NORMAL",
            "action":     "Standard nurture sequence",
            "ghl_tag":    "archetype_unknown",
            "sla_hours":  48,
        },
    }
    return strategies.get(archetype, strategies["UNKNOWN"])
```

7.3 `ml-service/app/services/post_call_service.py`

Назначение: Маршрутизация follow-up рукава по итогам звонка + статистика возражений.

Математическая логика:
- OBJECTION_ARM_MAP: 10 возражений → рукав (детерминированный mapping).
- First match wins — первое найденное возражение в списке определяет рукав.
- Conversion rate по возражению: count_converted / count_total.
- Commit rate: COMMITTED / total (all outcomes).

Взаимодействие: `PostCallController.java`; objection_tags сохраняются в call_notes.

Потенциальные риски:
- First match wins — порядок тегов в objection_tags влияет на выбор рукава. Не документировано.

Полный код

```
"""
Post-Call Intelligence (vision §19.18.5; arch §16.18).
Pure logic — zero FastAPI/Pydantic imports.

Routes follow-up arm based on call outcome + objection tags.
Computes per-objection conversion rates for brief and coordinator dashboard.
"""
from __future__ import annotations
import json
from collections import Counter

OBJECTION_ARM_MAP: dict[str, str] = {
    "ACCRED_UNCLEAR":       "arm_accred_education",
    "PRICE_CONCERN":        "arm_roi_calculator",
    "TIMING_NOT_RIGHT":     "arm_nurture_60d",
    "COMPETITOR_MENTION":   "arm_differentiator",
    "SPOUSE_APPROVAL":      "arm_couple_deck",
    "RISK_CONCERN":         "arm_risk_brief",
    "LIQUIDITY_CONCERN":    "arm_exit_options",
    "WANTS_MORE_INFO":      "arm_full_deck",
    "PORTFOLIO_ALLOCATION": "arm_allocation_model",
    "GEOGRAPHIC_CONCERN":   "arm_geo_compliance",
}

DEFAULT_ARM = "arm_standard_followup"


def route_followup_arm(objection_tags: list[str], call_outcome: str) -> dict:
    """
    Returns recommended follow-up arm for a completed call.
    NO_SHOW → recovery sequence (high priority).
    COLD → no arm.
    COMMITTED → wire instructions (urgent).
    Otherwise: first matched objection tag wins.
    """
    if call_outcome == "NO_SHOW":
        return {
            "arm":      "arm_noshow_recovery",
            "reason":   "No-show recovery sequence",
            "ghl_tag":  "post_call_noshow",
            "priority": "HIGH",
        }
    if call_outcome == "COLD":
        return {
            "arm":      None,
            "reason":   "Cold lead — no active follow-up",
            "ghl_tag":  None,
            "priority": "NONE",
        }
    if call_outcome == "COMMITTED":
        return {
            "arm":      "arm_wire_instructions",
            "reason":   "Committed — send wire instructions",
            "ghl_tag":  "post_call_committed",
            "priority": "URGENT",
        }

    for tag in objection_tags:
        if tag in OBJECTION_ARM_MAP:
            return {
                "arm":      OBJECTION_ARM_MAP[tag],
                "reason":   f"Objection: {tag}",
                "ghl_tag":  f"post_call_{tag.lower()}",
                "priority": "NORMAL",
```

```
            }

    return {
        "arm":      DEFAULT_ARM,
        "reason":   "No specific objection matched",
        "ghl_tag":  "post_call_standard",
        "priority": "NORMAL",
    }


def _parse_tags(raw) -> list[str]:
    if isinstance(raw, list):
        return raw
    if isinstance(raw, str):
        try:
            return json.loads(raw)
        except Exception:
            return []
    return []


def compute_objection_rates(call_notes: list[dict]) -> list[dict]:
    """
    Per-objection conversion rate (COMMITTED / total with that objection).
    Sorted by frequency desc.
    """
    tag_total: Counter = Counter()
    tag_converted: Counter = Counter()

    for note in call_notes:
        tags = _parse_tags(note.get("objection_tags") or [])
        converted = note.get("call_outcome") == "COMMITTED"
        for tag in tags:
            tag_total[tag] += 1
            if converted:
                tag_converted[tag] += 1

    return [
        {
            "objection_tag":   tag,
            "count_total":     total,
            "count_converted": tag_converted.get(tag, 0),
            "conversion_rate": round(tag_converted.get(tag, 0) / total, 4),
        }
        for tag, total in tag_total.most_common()
    ]


def summarise_call_batch(call_notes: list[dict], days: int = 30) -> dict:
    """
    Summary of call outcomes and top objections for coordinator dashboard.
    """
    outcomes: Counter = Counter(n.get("call_outcome") for n in call_notes)
    total = len(call_notes) or 1
    all_tags: list[str] = []
    for n in call_notes:
        all_tags.extend(_parse_tags(n.get("objection_tags") or []))

    top_objections = Counter(all_tags).most_common(5)
    return {
        "total_calls":    len(call_notes),
        "days":           days,
        "outcomes":       dict(outcomes),
        "commit_rate":    round(outcomes.get("COMMITTED", 0) / total, 4),
        "noshow_rate":    round(outcomes.get("NO_SHOW", 0) / total, 4),
```

```
        "top_objections": [{"tag": t, "count": c} for t, c in top_objections],
    }
```

7.4 `ml-service/app/services/form_qualification_service.py`

Назначение: Квалификация лида по форме — lead score + валидация + калибровка коэффициентов.

Математическая логика:
- **Lead score:** k_accr × k_ticket / 3.0
- k_accr: SELF_ACCREDITED=1.0, PENDING=0.6, NO=0.2, None=0.4
- k_ticket: LT25K=0.3, 25_50K=0.6, 50_100K=1.0, 100_250K=1.8, GT250K=3.0
- **Нормировка /3.0** — максимальный score = (1.0 × 3.0)/3.0 = 1.0.
- **Калибровка:** медианный ratio метода (observed_conversion_rate / predicted_lead_score).
- Принимается если ratio ∈[0.5, 2.0].

Взаимодействие: `FormQualificationController.java`; score влияет на TIERED override в funnel_path_service.

Потенциальные риски:
- k_accr×k_ticket — мультипликативная комбинация. При NO accreditation (0.2) и GT250K (3.0): score=0.2, что кажется низким для высокобюджетного инвестора.
- Медианный ratio для калибровки нестабилен при малой выборке.

Полный код

```
"""
Form Self-Qualification + Value Calibration (vision §19.5).
Pure logic — zero FastAPI/Pydantic imports.
"""
from __future__ import annotations
import re
from collections import defaultdict


TICKET_BUCKET_LABELS = {
    "LT25K":    "Under $25K",
    "25_50K":   "$25K – $50K",
    "50_100K":  "$50K – $100K",
    "100_250K": "$100K – $250K",
    "GT250K":   "Over $250K",
}

TICKET_BUCKET_ORDER = ["LT25K", "25_50K", "50_100K", "100_250K", "GT250K"]

K_TICKET_DEFAULT = {
    "LT25K":    0.3,
    "25_50K":   0.6,
    "50_100K":  1.0,
    "100_250K": 1.8,
    "GT250K":   3.0,
}


def classify_ticket_bucket(amount_usd: float | None) -> str | None:
    if amount_usd is None:
        return None
    if amount_usd < 25_000:
        return "LT25K"
    elif amount_usd < 50_000:
        return "25_50K"
    elif amount_usd < 100_000:
        return "50_100K"
    elif amount_usd < 250_000:
        return "100_250K"
    else:
        return "GT250K"


def validate_form_payload(payload: dict) -> dict:
    """
    Validates and normalizes a form submission payload.
    Returns {valid, errors, normalized}.
    """
    errors     = []
    normalized = {}

    email = payload.get("email", "").strip().lower()
    if not email:
        errors.append("email is required")
    elif not re.match(r"^[^@\s]+@[^@\s]+\.[^@\s]+$", email):
        errors.append("email format invalid")
    else:
        normalized["email"] = email

    phone = payload.get("phone", "").strip()
    if phone:
        digits = re.sub(r"\D", "", phone)
        if len(digits) < 7:
            errors.append("phone too short")
```

```
        else:
            normalized["phone"] = phone

    first = payload.get("first_name", "").strip()
    last  = payload.get("last_name", "").strip()
    if not first:
        errors.append("first_name is required")
    else:
        normalized["first_name"] = first[:100]
    if last:
        normalized["last_name"] = last[:100]

    sa = payload.get("self_accredited")
    if sa is not None:
        normalized["self_accredited"] = bool(sa)

    tb = payload.get("ticket_bucket")
    if tb is not None:
        if tb in TICKET_BUCKET_ORDER:
            normalized["ticket_bucket"] = tb
        else:
            matched = False
            for k, label in TICKET_BUCKET_LABELS.items():
                if tb.lower() in label.lower():
                    normalized["ticket_bucket"] = k
                    matched = True
                    break
            if not matched:
                errors.append(f"ticket_bucket '{tb}' not recognized")

    return {
        "valid":      len(errors) == 0,
        "errors":     errors,
        "normalized": normalized,
    }


def compute_lead_score(
    self_accredited: bool | None,
    ticket_bucket: str | None,
    k_ticket: dict | None = None,
) -> dict:
    """
    Lead quality score [0,1] from self-qualification data.
    Used for coordinator routing priority — NOT for CAPI value.
    """
    kt     = k_ticket or K_TICKET_DEFAULT
    k_accr = 1.0 if self_accredited else (0.3 if self_accredited is None else 0.0)
    k_tick = kt.get(ticket_bucket, 1.0) if ticket_bucket else 0.5
    raw    = k_accr * k_tick
    score  = min(raw / 3.0, 1.0)
    tier   = "HIGH" if score >= 0.5 else ("MED" if score >= 0.2 else "LOW")
    return {
        "score":    round(score, 3),
        "tier":     tier,
        "k_accr":   k_accr,
        "k_ticket": k_tick,
    }


def calibrate_value_coefficients(
    calibration_log: list[dict],
    min_samples: int = 20,
) -> dict:
    """
```

```
    Calibrates k_ticket coefficients by comparing predicted vs actual value.
    Returns suggested coefficient updates per bucket.
    """
    import statistics

    bucket_data: dict[str, list[float]] = defaultdict(list)
    for entry in calibration_log:
        predicted = entry.get("predicted_value")
        actual    = entry.get("actual_value")
        bucket    = entry.get("ticket_bucket")
        if predicted is not None and actual is not None and bucket and float(predicted) > 0:
            bucket_data[bucket].append(float(actual) / float(predicted))

    suggestions = {}
    for bucket, ratios in bucket_data.items():
        if len(ratios) < min_samples:
            suggestions[bucket] = {
                "current":   K_TICKET_DEFAULT.get(bucket, 1.0),
                "suggested": None,
                "n":         len(ratios),
                "reason":    f"Insufficient data (need {min_samples}, have {len(ratios)})",
            }
        else:
            median_ratio = statistics.median(ratios)
            current      = K_TICKET_DEFAULT.get(bucket, 1.0)
            suggestions[bucket] = {
                "current":   current,
                "suggested": round(current * median_ratio, 2),
                "ratio":     round(median_ratio, 3),
                "n":         len(ratios),
                "reason":    "median predicted/actual ratio",
            }

    return {
        "suggestions":      suggestions,
        "total_samples":    len(calibration_log),
        "buckets_analyzed": list(bucket_data.keys()),
    }
```

7.5 `ml-service/app/services/mmm_service.py`

Назначение: Media Mix Modeling — adstock + Hill saturation + оптимизация бюджета.

Математическая логика:
- **Geometric adstock:** adstock(t) = spend(t) + decay × adstock(t−1); decay ∈[0,1].
- **Hill saturation:** f(x) = x^s / (hs^s + x^s); s = shape, hs = half-saturation.
- **Greedy marginal allocation:** увеличиваем бюджет на Δ для канала с максимальным marginal return; итерируем до распределения всего бюджета.
- Оптимизация только для каналов без CBO (CBO excluded).

Взаимодействие: `MmmController.java`; данные о spend и conversions по каналам и периодам.

Потенциальные риски:
- Greedy allocation не гарантирует глобальный оптимум. При неконвексной Hill-кривой может застрять в локальном оптимуме.
- Greedy шаг Δ не определён явно в service — зависит от caller. Слишком большой Δ → грубая аппроксимация.
- Adstock decay фиксирован как входной параметр — не оптимизируется, хотя это ключевой гиперпараметр.

Полный код

```
"""
Bayesian Media Mix Modeling (MMM) — Phase 7, §17.3.
Adstock + Hill saturation. PyMC imported lazily (not at module level).
"""
from __future__ import annotations


def apply_adstock(spends: list[float], decay_rate: float) -> list[float]:
    """Geometric adstock: adstock[t] = spend[t] + decay_rate * adstock[t-1]."""
    if not spends:
        return []
    result: list[float] = []
    prev = 0.0
    for s in spends:
        val = s + decay_rate * prev
        result.append(val)
        prev = val
    return result


def apply_saturation(
    adstocked: list[float],
    half_saturation: float,
    slope: float = 1.0,
) -> list[float]:
    """Hill saturation: f(x) = x^slope / (half_saturation^slope + x^slope)."""
    if not adstocked:
        return []
    hs = half_saturation ** slope if half_saturation > 0 else 1.0
    result: list[float] = []
    for x in adstocked:
        if x <= 0.0:
            result.append(0.0)
        else:
            xs = x ** slope
            result.append(xs / (hs + xs))
    return result


def compute_channel_contributions(
    spend_data: dict[str, list[float]],
    contributions: dict[str, float],
) -> dict[str, list[float]]:
    """Per-channel contribution time series (not normalised across channels)."""
    result: dict[str, list[float]] = {}
    for channel, spends in spend_data.items():
        weight = contributions.get(channel, 0.0)
        result[channel] = [s * weight for s in spends]
    return result


def _expected_roi(
    saturation_params: dict[str, dict],
    allocation: dict[str, float],
) -> float:
    """Compute expected total conversions given a budget split."""
    total = 0.0
    for channel, budget in allocation.items():
        p = saturation_params.get(channel, {})
        hs = p.get("half_saturation", 5000.0)
        slope = p.get("slope", 1.0)
        decay = p.get("decay_rate", 0.5)
        roi_scale = p.get("roi_scale", 1.0)
        # Single-period adstock for a lump budget
```

```
        adstocked = [budget / (1.0 - decay)] if decay < 1.0 else [budget]
        saturated = apply_saturation(adstocked, hs, slope)
        total += saturated[0] * roi_scale
    return total


def optimize_budget_allocation(
    saturation_params: dict[str, dict],
    total_budget: float,
    n_steps: int = 20,
) -> dict:
    """
    Grid search over budget splits to maximise expected conversions.
    Returns {"allocation": {channel: budget}, "expected_roi": float, "curve": [...]}
    """
    channels = list(saturation_params.keys())
    if not channels:
        return {"allocation": {}, "expected_roi": 0.0, "curve": []}

    if len(channels) == 1:
        ch = channels[0]
        roi = _expected_roi(saturation_params, {ch: total_budget})
        return {
            "allocation": {ch: total_budget},
            "expected_roi": round(roi, 4),
            "curve": [{"budget": total_budget, "expected_roi": round(roi, 4)}],
        }

    # For 2 channels: grid search; generalises to N via greedy marginal increments
    if len(channels) == 2:
        ch_a, ch_b = channels[0], channels[1]
        best_roi = -1.0
        best_alloc: dict[str, float] = {}
        curve = []
        for i in range(n_steps + 1):
            frac = i / n_steps
            alloc = {ch_a: frac * total_budget, ch_b: (1 - frac) * total_budget}
            roi = _expected_roi(saturation_params, alloc)
            curve.append({"budget_frac_a": round(frac, 3), "expected_roi": round(roi, 4)})
            if roi > best_roi:
                best_roi = roi
                best_alloc = {k: round(v, 2) for k, v in alloc.items()}
        return {"allocation": best_alloc, "expected_roi": round(best_roi, 4), "curve": curve}

    # N channels: try SLSQP (global optimum under Hill saturation); fall back to greedy
    try:
        from scipy.optimize import minimize  # type: ignore[import]

        def _neg_roi(x: list) -> float:
            return -_expected_roi(saturation_params, {ch: max(0.0, float(b)) for ch, b in zip(channels,
x)})

        x0 = [total_budget / len(channels)] * len(channels)
        result = minimize(
            _neg_roi,
            x0,
            method="SLSQP",
            bounds=[(0.0, total_budget)] * len(channels),
            constraints=[{"type": "eq", "fun": lambda x: float(sum(x)) - total_budget}],
            options={"ftol": 1e-9, "maxiter": 1000},
        )
        if result.success:
            best_alloc = {ch: round(max(0.0, float(b)), 2) for ch, b in zip(channels, result.x)}
            # re-normalise to exact total_budget
            alloc_sum = sum(best_alloc.values())
```

```
            if alloc_sum > 0:
                best_alloc = {k: round(v / alloc_sum * total_budget, 2) for k, v in best_alloc.items()}
            roi = _expected_roi(saturation_params, best_alloc)
            return {"allocation": best_alloc, "expected_roi": round(roi, 4), "curve": [], "method":
"slsqp"}
    except Exception:
        pass  # scipy unavailable or failed → greedy fallback below

    # Greedy marginal increment fallback (may find local optima on non-convex Hill curves)
    step_size = total_budget / n_steps
    allocation = {ch: 0.0 for ch in channels}
    for _ in range(n_steps):
        best_channel = None
        best_marginal = -1.0
        for ch in channels:
            test = dict(allocation)
            test[ch] += step_size
            marginal = _expected_roi(saturation_params, test) - _expected_roi(saturation_params,
allocation)
            if marginal > best_marginal:
                best_marginal = marginal
                best_channel = ch
        if best_channel:
            allocation[best_channel] += step_size

    roi = _expected_roi(saturation_params, allocation)
    return {
        "allocation": {k: round(v, 2) for k, v in allocation.items()},
        "expected_roi": round(roi, 4),
        "curve": [],
        "method": "greedy",
    }


def build_mmm_summary(
    spend_data: dict[str, list[float]],
    contributions: dict[str, float],
    saturation_params: dict[str, dict],
    date_range: tuple[str, str],
) -> dict:
    """Build dict for storage in mmm_results table."""
    channels = list(spend_data.keys())
    total_budget = sum(sum(v) for v in spend_data.values()) / max(len(next(iter(spend_data.values())),
[1])), 1)

    saturation_curves: dict[str, list[dict]] = {}
    for ch in channels:
        p = saturation_params.get(ch, {})
        hs = p.get("half_saturation", 5000.0)
        slope = p.get("slope", 1.0)
        roi_scale = p.get("roi_scale", 1.0)
        pts = []
        for k in range(1, 11):
            spend = hs * k * 0.5
            sat = apply_saturation([spend], hs, slope)
            marginal_roi = sat[0] * roi_scale / spend if spend > 0 else 0.0
            pts.append({"spend": round(spend, 2), "marginal_roi": round(marginal_roi, 4)})
        saturation_curves[ch] = pts

    alloc_result = optimize_budget_allocation(saturation_params, total_budget)
    n_obs = max(len(v) for v in spend_data.values()) if spend_data else 0

    return {
        "channels": channels,
        "contributions": contributions,
```

```
        "saturation_curves": saturation_curves,
        "budget_allocation": alloc_result["allocation"],
        "total_budget": round(total_budget, 2),
        "date_range_start": date_range[0],
        "date_range_end": date_range[1],
        "n_obs": n_obs,
        "fit_metrics": {"expected_roi": alloc_result["expected_roi"]},
    }
```

7.6 `ml-service/app/services/portfolio_service.py`

Назначение: Оптимизация портфеля рекламных кампаний по Sharpe ratio.

Математическая логика:

```
Sharpe = (mean_r − rf) × 365 / (std_r × √365)
```

- rf = risk-free rate = 0.
- **Mean-variance optimisation:** минимизация дисперсии при заданном Sharpe; с 5% floor для каждой кампании.
- CBO кампании полностью исключены.
- **РЕКОМЕНДАЦИЯ ТОЛЬКО** — не автоматическое распределение.

Взаимодействие: `PortfolioController.java`; входные данные — исторические daily returns по кампаниям.

Потенциальные риски:
- Sharpe × 365 предполагает daily компаундирование. При месячных данных нужно ×12 (не ×365). Нужно уточнить временной масштаб return.
- std_r из исторических данных может занижать хвостовой риск (fat tails в рекламе).

Полный код

```
"""
Portfolio Budget Optimisation service — no FastAPI/Pydantic imports.
Sharpe-ratio mean-variance allocation across campaigns/channels.
NEVER inside a CBO campaign (conflict with Meta).
Output = recommendation ONLY, not action (vision §8.5).
Architecture §15.21, roadmap 6-7.
"""
from __future__ import annotations

import math
from datetime import date, timedelta
from typing import Optional

_MIN_HISTORY_MONTHS = 6
_MIN_WEIGHT_FLOOR = 0.05     # 5% floor per asset
_RISK_FREE_RATE   = 0.0      # daily risk-free return
_ANNUALISE        = 365.0


def check_sufficient_history(
    earliest_date_str: str,
    latest_date_str: str,
) -> dict:
    """
    Verify at least 6 months of ad-set return history.
    Dates as ISO strings (YYYY-MM-DD).
    """
    try:
        d0 = date.fromisoformat(earliest_date_str)
        d1 = date.fromisoformat(latest_date_str)
    except ValueError as exc:
        return {"sufficient": False, "error": str(exc)}

    days = (d1 - d0).days
    months = days / 30.44  # average month length
    required_days = _MIN_HISTORY_MONTHS * 30.44

    return {
        "sufficient": days >= required_days,
        "history_days": days,
        "history_months": round(months, 1),
        "required_months": _MIN_HISTORY_MONTHS,
    }


def _sharpe_weight(returns: list[float], risk_free: float = _RISK_FREE_RATE,
                   periods_per_year: float = _ANNUALISE) -> float:
    """Annualised Sharpe ratio. periods_per_year: 365 daily, 52 weekly, 12 monthly."""
    if len(returns) < 2:
        return 0.0
    import math as _math
    n = len(returns)
    mean_r = sum(returns) / n
    variance = sum((r - mean_r) ** 2 for r in returns) / (n - 1)
    std_r = _math.sqrt(variance) if variance > 0 else 1e-9
    sharpe = (mean_r - risk_free) * periods_per_year / (std_r * _math.sqrt(periods_per_year))
    return max(sharpe, 0.0)  # floor at 0 — negative Sharpe gets zero allocation


def compute_portfolio_allocation(
    campaigns: list[dict],
    total_budget: float,
) -> dict:
    """
```

```
    Compute Sharpe-ratio portfolio allocation.
    Each campaign dict: {id, name, is_cbo: bool, daily_returns: list[float]}.
    CBO campaigns excluded (conflict with Meta's own optimisation).
    5% minimum weight floor applied to each non-CBO campaign.
    Output is recommendation only — caller must confirm before any spend change.

    Returns {allocations: [{id, name, weight, budget}], total_budget, is_recommendation_only,
             excluded_cbo: [...], warnings: [...]}
    """
    warnings = []
    excluded_cbo = []
    eligible = []

    for c in campaigns:
        if c.get("is_cbo"):
            excluded_cbo.append(c.get("id") or c.get("name", "unknown"))
            warnings.append(f"CBO campaign '{c.get('name', c.get('id'))}' excluded "
                            "(Portfolio allocation conflicts with Meta CBO — §8.5)")
        else:
            eligible.append(c)

    if not eligible:
        return {
            "allocations": [],
            "total_budget": total_budget,
            "is_recommendation_only": True,
            "excluded_cbo": excluded_cbo,
            "warnings": warnings + ["No eligible campaigns after CBO exclusion"],
        }

    # Compute Sharpe per campaign
    sharpes = []
    for c in eligible:
        returns = c.get("daily_returns") or []
        sharpes.append(_sharpe_weight(returns))

    total_sharpe = sum(sharpes)

    # If all Sharpe are zero, use equal weights
    if total_sharpe == 0:
        raw_weights = [1.0 / len(eligible)] * len(eligible)
        warnings.append("All campaigns have zero or negative Sharpe; using equal weights")
    else:
        raw_weights = [s / total_sharpe for s in sharpes]

    # Apply 5% floor and renormalize
    weights = _apply_floor(raw_weights, _MIN_WEIGHT_FLOOR)

    allocations = []
    for c, w in zip(eligible, weights):
        allocations.append({
            "id":     c.get("id"),
            "name":   c.get("name", ""),
            "weight": round(w, 4),
            "budget": round(w * total_budget, 2),
            "sharpe": round(_sharpe_weight(c.get("daily_returns") or []), 3),
        })

    # Sort by budget descending
    allocations.sort(key=lambda x: x["budget"], reverse=True)

    return {
        "allocations":          allocations,
        "total_budget":         total_budget,
        "is_recommendation_only": True,  # never an action — must be confirmed by marketer
```

```
        "excluded_cbo":         excluded_cbo,
        "warnings":             warnings,
        "note":                 "Output is recommendation only. Requires ≥6 months history per §8.5.",
    }


def _apply_floor(weights: list[float], floor: float) -> list[float]:
    """Iteratively apply minimum weight floor, renormalising remaining budget."""
    w = list(weights)
    n = len(w)
    if n == 0:
        return w

    max_iter = n * 2
    for _ in range(max_iter):
        clamped = [max(x, floor) for x in w]
        total = sum(clamped)
        if total == 0:
            return [1.0 / n] * n
        w = [c / total for c in clamped]
        if all(x >= floor - 1e-9 for x in w):
            break

    return w


# ─── Markowitz mean-variance (roadmap 9-12; vision §8.5) ──────────────────────

def _roas_samples(asset: dict, n_samples: int, rng) -> "np.ndarray":
    """Posterior ROAS samples for one asset.

    Accepts either raw samples {roas_samples: [...]} or a Beta-scaled posterior
    {roas_alpha, roas_beta, roas_scale?}: ROAS = scale × Beta(α, β) (scale defaults to the
    tenant's max plausible ROAS, 10). Raw samples are resampled with replacement to n_samples.
    """
    import numpy as np
    samples = asset.get("roas_samples")
    if samples:
        arr = np.asarray(samples, dtype=float)
        if arr.size == 0:
            return np.zeros(n_samples)
        return rng.choice(arr, size=n_samples, replace=True) if arr.size != n_samples else arr
    a = float(asset.get("roas_alpha", 1.0))
    b = float(asset.get("roas_beta", 1.0) or 1.0)
    scale = float(asset.get("roas_scale", 10.0))
    return scale * rng.beta(max(a, 1e-6), max(b, 1e-6), size=n_samples)


def compute_markowitz(
    assets: list[dict],
    risk_aversion: float = 3.0,
    floor: float = _MIN_WEIGHT_FLOOR,
    n_samples: int = 2000,
    rng=None,
    n_iter: int = 2000,
) -> dict:
    """Long-only mean-variance allocation with a per-asset floor.

    maximise  μᵀw − λ·wᵀΣw   s.t. Σw = 1, w ≥ floor
    solved by projected gradient ascent (projection onto the floored simplex). μ and Σ come from
    posterior ROAS samples, so the covariance carries the *uncertainty* of each campaign, not only
    its historical volatility. Output is a recommendation only (never inside CBO).
    """
    import numpy as np
    g = rng if rng is not None else np.random.default_rng()
```

```
    assets = [a for a in assets if not a.get("is_cbo")]
    n = len(assets)
    if n == 0:
        return {"weights": {}, "expected_roas": 0.0, "variance": 0.0, "method": "markowitz",
                "is_recommendation_only": True, "warnings": ["no eligible assets"]}
    if n == 1:
        return {"weights": {assets[0]["id"]: 1.0}, "expected_roas":
float(np.mean(_roas_samples(assets[0], n_samples, g))),
                "variance": 0.0, "method": "markowitz", "is_recommendation_only": True, "warnings": []}

    S = np.column_stack([_roas_samples(a, n_samples, g) for a in assets])   # n_samples × n
    mu = S.mean(axis=0)
    cov = np.cov(S, rowvar=False) + np.eye(n) * 1e-9
    floor = min(max(float(floor), 0.0), 1.0 / n)

    def project(w):
        """Project onto {w ≥ floor, Σw = 1}: shift by the floor, project the rest onto the simplex."""
        v = w - floor
        budget = 1.0 - floor * n
        u = np.sort(v)[::-1]
        css = np.cumsum(u)
        k = np.nonzero(u * np.arange(1, n + 1) > (css - budget))[0]
        rho = k[-1] if k.size else 0
        theta = (css[rho] - budget) / (rho + 1)
        return np.maximum(v - theta, 0.0) + floor

    w = np.full(n, 1.0 / n)
    lam = float(risk_aversion)
    step = 1.0 / (2.0 * lam * float(np.linalg.eigvalsh(cov).max()) + float(np.abs(mu).max()) + 1e-9)
    for _ in range(n_iter):
        grad = mu - 2.0 * lam * cov @ w
        w_new = project(w + step * grad)
        if np.abs(w_new - w).max() < 1e-9:
            w = w_new
            break
        w = w_new

    weights = {a["id"]: round(float(x), 4) for a, x in zip(assets, w)}
    return {
        "weights": weights,
        "expected_roas": round(float(mu @ w), 6),
        "variance": round(float(w @ cov @ w), 6),
        "mu": {a["id"]: round(float(m), 6) for a, m in zip(assets, mu)},
        "risk_aversion": lam,
        "floor": floor,
        "method": "markowitz",
        "is_recommendation_only": True,
        "warnings": [],
    }


def allocate(
    assets: list[dict],
    history_ok: bool,
    total_budget: float = 0.0,
    risk_aversion: float = 3.0,
    rng=None,
) -> dict:
    """Markowitz when ≥ 6 months of history (`history_ok`), otherwise the Sharpe allocation."""
    if history_ok:
        res = compute_markowitz(assets, risk_aversion=risk_aversion, rng=rng)
        res["allocations"] = [
            {"id": k, "weight": v, "budget": round(v * total_budget, 2)} for k, v in
res["weights"].items()
        ]
```

```
        return res
    res = compute_portfolio_allocation(assets, total_budget)
    res["method"] = "sharpe"
    res["weights"] = {a["id"]: a["weight"] for a in res["allocations"]}
    return res
```

СЕКЦИЯ 8 — Java Math (Spring Boot)

8.1 `platform-api/.../service/GateEvaluationService.java`

Назначение: Байесовские ворота (Gate 1/2/3) для ad sets — kill/advance решения.

Математическая логика:
- **Gate 1 (Hook):** ≥5000 impressions AND ≥2 дней → P(CTR_adset > CTR_group_mean) ≥ 10%.
- **Gate 2 (Click+Landing):** ≥300 clicks → P(CTR > group_mean) ≥ 10%.
- **Gate 3 (Conversion):** ≥50 conversions → P(CVR > anchor_CVR) ≥ 10%.
- **Kill threshold:** P < 0.10 → KILLED.

P(θ1 > θ2) — Marsaglia-Tsang Beta sampler (MC=10000):

```
// Marsaglia & Tsang (2000): Gamma(shape) via accept-reject
// Beta(a,b) = Gamma(a) / (Gamma(a) + Gamma(b))
// При shape < 1: Gamma(a) = Gamma(a+1) × U^(1/a)
```

- d = shape − 1/3; c = 1/√(9d)
- Accept if: u < 1 − 0.0331 × x^4 (быстрая проверка)
- Accept if: log(u) < $0.5x^2$ + d(1 − v + log(v)) (точная проверка)

Взаимодействие: вызывается `GateScanScheduler.java` (каждый час); `GateController.java` (вручную). Пишет статус в meta_ad_sets.gate_status.

Потенциальные риски:
- `new Random()` без seed → каждый вызов даёт разные результаты. При граничных P=0.10 возможна нестабильность.
- Group mean включает сам ad set → slight self-comparison bias.
- KILL_THRESHOLD = 0.10 — это очень мягкий порог. P < 10% означает "почти наверняка хуже среднего".

Полный код

```
package com.marketingai.service;

import java.util.Map;
import org.springframework.jdbc.core.JdbcTemplate;
import com.marketingai.entity.MetaAdSet;
import com.marketingai.entity.MetaCampaign;
import com.marketingai.repository.MetaAdSetRepository;
import com.marketingai.repository.MetaCampaignRepository;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.time.Instant;
import java.time.temporal.ChronoUnit;
import java.util.List;
import java.util.Optional;
import java.util.Random;
import java.util.UUID;
import java.util.concurrent.ThreadLocalRandom;

/**
 * Bayesian Staged Gate evaluation per vision §12.3.
 *
 * Three gates with Bayesian stopping rules:
 *   Gate 1 (Hook): ≥5K impressions + ≥2 days → P(CTR > group mean) ≥ 10%
 *   Gate 2 (Click+Landing): ≥300 clicks → P(CTR > group mean) ≥ 10%
 *   Gate 3 (Conversion): ≥50 conversions → P(CVR > anchor CVR) ≥ 10%
 *
 * Kill rule: if P(metric_i > baseline) < KILL_THRESHOLD (0.10) → pause ad set.
 * NOT hard thresholds — Bayesian posterior probabilities (§12.3).
 *
 * L1 circuit breaker: if spend > max_spend_per_adset → kill regardless of gate.
 */
@Slf4j
@Service
@RequiredArgsConstructor
public class GateEvaluationService {

    private static final double KILL_THRESHOLD = 0.10; // P < 10% → kill
    private static final int GATE_1_MIN_IMPRESSIONS = 5_000;
    private static final int GATE_2_MIN_CLICKS = 300;
    private static final int GATE_3_MIN_CONVERSIONS = 50;
    private static final long GATE_1_MIN_DAYS = 2;
    private static final int MC_SAMPLES = 10_000;

    private final MetaAdSetRepository adSetRepo;
    private final MetaCampaignRepository campaignRepo;
    private final MetaApiService metaApi;
    private final JdbcTemplate jdbc;
    private final FunnelService funnelService;

    // ─── Held-call funnel counts (roadmap 8-3; vision §19.5) ──────────────

    /** Booking + held stage keys for the tenant, or null when the funnel has no appointment quality
stage. */
    record HeldStages(String bookingStage, String heldStage) {}

    private HeldStages heldStages(UUID tenantId) {
        String held = funnelService.getQualityStageKey(tenantId).orElse(null);
        if (held == null) return null;
        List<String> booking = jdbc.queryForList(
                "SELECT stage_key FROM funnel_stages WHERE tenant_id = ? " +
                "AND (stage_key = 'BOOKED' OR maps_to_ghl_status LIKE 'AppointmentCreate%') " +
```

```
                "ORDER BY ordinal DESC LIMIT 1", String.class, tenantId);
        if (booking.isEmpty()) return null;
        // the quality stage must be an appointment outcome, otherwise the legacy conversion gate
applies
        List<String> ghl = jdbc.queryForList(
                "SELECT maps_to_ghl_status FROM funnel_stages WHERE tenant_id = ? AND stage_key = ?",
                String.class, tenantId, held);
        boolean appointment = !ghl.isEmpty() && ghl.get(0) != null &&
ghl.get(0).startsWith("AppointmentUpdate");
        return appointment ? new HeldStages(booking.get(0), held) : null;
    }

    /** reward_events of sessions attributed to the ad set's creative, for the batch window. */
    private HeldCallGate.Counts heldCounts(MetaAdSet adSet, HeldStages stages) {
        if (adSet.getCreativeId() == null) return new HeldCallGate.Counts(safe(adSet.getClicksTotal()),
0, 0, spend(adSet));
        Map<String, Object> row = jdbc.queryForMap("""
                SELECT COUNT(DISTINCT re.session_id) FILTER (WHERE re.stage_key = ?) AS bookings,
                       COUNT(DISTINCT re.session_id) FILTER (WHERE re.stage_key = ?) AS held
                FROM reward_events re
                JOIN sessions s ON s.id = re.session_id AND s.tenant_id = re.tenant_id
                WHERE re.tenant_id = ? AND s.creative_id = ?
                  AND re.rewarded_at >= COALESCE(?, now() - INTERVAL '90 days')
                """, stages.bookingStage(), stages.heldStage(), adSet.getTenantId(),
adSet.getCreativeId(),
                adSet.getBatchLaunchedAt() != null ? java.sql.Timestamp.from(adSet.getBatchLaunchedAt())
: null);
        return new HeldCallGate.Counts(safe(adSet.getClicksTotal()),
                ((Number) row.get("bookings")).longValue(), ((Number) row.get("held")).longValue(),
spend(adSet));
    }

    private static HeldCallGate.Counts sum(List<HeldCallGate.Counts> list) {
        long c = 0, b = 0, h = 0; double sp = 0;
        for (HeldCallGate.Counts x : list) { c += x.clicks(); b += x.bookings(); h += x.held(); sp +=
x.spend(); }
        return new HeldCallGate.Counts(c, b, h, sp);
    }

    private static double spend(MetaAdSet a) {
        return a.getSpendTotal() == null ? 0.0 : a.getSpendTotal();
    }

    /**
     * Evaluate all eligible TEST campaign ad sets for the given tenant.
     * Called by GateScanScheduler (hourly) and manually via GateController.
     *
     * @return number of ad sets evaluated (killed + advanced)
     */
    @Transactional
    public int evaluateAll(UUID tenantId) {
        int count = 0;
        for (MetaCampaign campaign : campaignRepo.findAllByTenantId(tenantId)) {
            if (!"TEST".equals(campaign.getContour())) continue;
            count += evaluateCampaign(campaign);
        }
        return count;
    }

    /**
     * Evaluate all ad sets in a single campaign.
     */
    @Transactional
    public int evaluateCampaign(MetaCampaign campaign) {
        List<MetaAdSet> adSets = adSetRepo.findAllByCampaignId(campaign.getId())
```

```
                .stream()
                .filter(a -> !"KILLED".equals(a.getGateStatus()) && !"PASSED".equals(a.getGateStatus()))
                .toList();

        if (adSets.isEmpty()) return 0;

        // Find anchor for Gate 3 comparison
        Optional<MetaAdSet> anchorOpt = adSets.stream().filter(MetaAdSet::isAnchor).findFirst();

        int count = 0;
        for (MetaAdSet adSet : adSets) {
            boolean changed = evaluateAdSet(adSet, campaign, adSets, anchorOpt.orElse(null));
            if (changed) count++;
        }
        return count;
    }

    private boolean evaluateAdSet(MetaAdSet adSet, MetaCampaign campaign,
                                  List<MetaAdSet> batchPeers, MetaAdSet anchor) {
        // L1 circuit breaker: spend cap
        if (campaign.getMaxSpendPerAdset() != null
                && adSet.getSpendTotal() != null
                && adSet.getSpendTotal() >= campaign.getMaxSpendPerAdset()) {
            return kill(adSet, "L1: spend $" + adSet.getSpendTotal()
                    + " exceeded max $" + campaign.getMaxSpendPerAdset());
        }

        return switch (adSet.getGateStatus()) {
            case "GATE_1" -> evaluateGate1(adSet, batchPeers);
            case "GATE_2" -> evaluateGate2(adSet, batchPeers);
            case "GATE_3" -> evaluateGate3(adSet, batchPeers, anchor);
            default -> false;
        };
    }

    // ── Gate 1: Hook (impressions) ──────────────────────────────

    private boolean evaluateGate1(MetaAdSet adSet, List<MetaAdSet> peers) {
        long impressions = safe(adSet.getImpressionsTotal());

        // Entry condition: ≥5K impressions AND ≥2 days since launch
        if (impressions < GATE_1_MIN_IMPRESSIONS) return false;
        if (adSet.getBatchLaunchedAt() != null
                && ChronoUnit.DAYS.between(adSet.getBatchLaunchedAt(), Instant.now()) < GATE_1_MIN_DAYS)
{
            return false;
        }

        long clicks = safe(adSet.getClicksTotal());

        // Group mean CTR (all GATE_1 or higher peers)
        long groupClicks = peers.stream().mapToLong(a -> safe(a.getClicksTotal())).sum();
        long groupImpressions = peers.stream().mapToLong(a -> safe(a.getImpressionsTotal())).sum();
        double groupAlpha = groupClicks + 1.0;
        double groupBeta  = Math.max(groupImpressions - groupClicks, 0) + 1.0;

        // Peer posterior for this ad set
        double alpha = clicks + 1.0;
        double beta  = Math.max(impressions - clicks, 0) + 1.0;

        double pBetter = probBetter(alpha, beta, groupAlpha, groupBeta);

        adSet.setGate1EvaluatedAt(Instant.now());
        log.info("Gate 1 — adSet={} impressions={} clicks={} P(better)={}",
                adSet.getId(), impressions, clicks, pBetter);
```

```
        if (pBetter < KILL_THRESHOLD) {
            return kill(adSet, String.format("Gate 1: P(CTR>group)=%.2f%% < 10%% (impressions=%d)",
                    pBetter * 100, impressions));
        }

        // Advance to Gate 2
        adSet.setGateStatus("GATE_2");
        adSetRepo.save(adSet);
        return true;
    }

    // ── Gate 2: Click + Landing ──────────────────────────────────────

    private boolean evaluateGate2(MetaAdSet adSet, List<MetaAdSet> peers) {
        long clicks = safe(adSet.getClicksTotal());
        if (clicks < GATE_2_MIN_CLICKS) return false;

        long impressions = safe(adSet.getImpressionsTotal());
        long groupClicks = peers.stream().mapToLong(a -> safe(a.getClicksTotal())).sum();
        long groupImpressions = peers.stream().mapToLong(a -> safe(a.getImpressionsTotal())).sum();

        double alpha = clicks + 1.0;
        double beta  = Math.max(impressions - clicks, 0) + 1.0;
        double groupAlpha = groupClicks + 1.0;
        double groupBeta  = Math.max(groupImpressions - groupClicks, 0) + 1.0;

        double pBetter = probBetter(alpha, beta, groupAlpha, groupBeta);

        adSet.setGate2EvaluatedAt(Instant.now());
        log.info("Gate 2 — adSet={} clicks={} P(CTR>group)={}", adSet.getId(), clicks, pBetter);

        if (pBetter < KILL_THRESHOLD) {
            return kill(adSet, String.format("Gate 2: P(CTR>group)=%.2f%% < 10%% (clicks=%d)",
                    pBetter * 100, clicks));
        }

        // Roadmap 8-3: Gate 2 += booking_rate — clicks that never book are not a landing win
        HeldStages stages = heldStages(adSet.getTenantId());
        if (stages != null) {
            HeldCallGate.Counts mine = heldCounts(adSet, stages);
            HeldCallGate.Counts group = sum(peers.stream().map(p -> heldCounts(p, stages)).toList());
            Optional<Double> pBook = HeldCallGate.probBookingRateBetter(mine, group);
            if (pBook.isPresent() && pBook.get() < KILL_THRESHOLD) {
                return kill(adSet, String.format("Gate 2: P(booking_rate>group)=%.2f%% < 10%%
(bookings=%d of %d clicks)",
                        pBook.get() * 100, mine.bookings(), mine.clicks()));
            }
        }

        adSet.setGateStatus("GATE_3");
        adSetRepo.save(adSet);
        return true;
    }

    // ── Gate 3: Conversion ──────────────────────────────────────

    private boolean evaluateGate3(MetaAdSet adSet, List<MetaAdSet> peers, MetaAdSet anchor) {
        // Roadmap 8-3: for an appointment funnel Gate 3 is cost per CALL_HELD vs anchor + show-rate
kill
        HeldStages stages = heldStages(adSet.getTenantId());
        if (stages != null) {
            HeldCallGate.Counts mine = heldCounts(adSet, stages);
            if (!HeldCallGate.hasEnoughData(mine)) return false;
            HeldCallGate.Counts base = (anchor != null && !anchor.getId().equals(adSet.getId()))
```

```
                    ? heldCounts(anchor, stages)
                    : sum(peers.stream().filter(p -> !p.getId().equals(adSet.getId())).map(p ->
heldCounts(p, stages)).toList());
            adSet.setGate3EvaluatedAt(Instant.now());
            Optional<String> killReason = HeldCallGate.evaluateGate3(mine, base);
            log.info("Gate 3 (held calls) — adSet={} bookings={} held={} cost/held={} kill={}",
                    adSet.getId(), mine.bookings(), mine.held(), mine.costPerHeld(),
killReason.orElse("no"));
            if (killReason.isPresent()) return kill(adSet, killReason.get());
            if (mine.held() >= HeldCallGate.MIN_HELD_FOR_COST) {
                adSet.setGateStatus("PASSED");
                adSetRepo.save(adSet);
                log.info("Gate 3 PASSED on held calls: adSet={} is a winner candidate", adSet.getId());
                return true;
            }
            return false;
        }

        long conversions = safe(adSet.getConversionsTotal());
        if (conversions < GATE_3_MIN_CONVERSIONS) return false;

        long clicks = safe(adSet.getClicksTotal());

        // Baseline: anchor if available, otherwise group mean
        double baseAlpha, baseBeta;
        if (anchor != null && !anchor.getId().equals(adSet.getId())) {
            long anchorConv = safe(anchor.getConversionsTotal());
            long anchorClicks = safe(anchor.getClicksTotal());
            baseAlpha = anchorConv + 1.0;
            baseBeta  = Math.max(anchorClicks - anchorConv, 0) + 1.0;
        } else {
            long groupConv = peers.stream().mapToLong(a -> safe(a.getConversionsTotal())).sum();
            long groupClicks = peers.stream().mapToLong(a -> safe(a.getClicksTotal())).sum();
            baseAlpha = groupConv + 1.0;
            baseBeta  = Math.max(groupClicks - groupConv, 0) + 1.0;
        }

        double alpha = conversions + 1.0;
        double beta  = Math.max(clicks - conversions, 0) + 1.0;

        double pBetter = probBetter(alpha, beta, baseAlpha, baseBeta);

        adSet.setGate3EvaluatedAt(Instant.now());
        log.info("Gate 3 — adSet={} conversions={} P(CVR>baseline)={}",
                adSet.getId(), conversions, pBetter);

        if (pBetter < KILL_THRESHOLD) {
            return kill(adSet, String.format("Gate 3: P(CVR>baseline)=%.2f%% < 10%% (conversions=%d)",
                    pBetter * 100, conversions));
        }

        adSet.setGateStatus("PASSED");
        adSetRepo.save(adSet);
        log.info("Gate 3 PASSED: adSet={} is a winner candidate", adSet.getId());
        return true;
    }

    // ─── Bayesian Monte Carlo ──────────────────────────────────────────

    /**
     * P(θ1 > θ2) where θ1 ~ Beta(a1, b1) and θ2 ~ Beta(a2, b2).
     * Estimated via Monte Carlo (Marsaglia-Tsang Gamma sampler).
     */
    static double probBetter(double a1, double b1, double a2, double b2) {
        // Replace: Random rng = new Random();
```

```
        // With: ThreadLocalRandom (thread-safe, no shared state)
        Random rng = ThreadLocalRandom.current();
        int wins = 0;
        for (int i = 0; i < MC_SAMPLES; i++) {
            if (sampleBeta(a1, b1, rng) > sampleBeta(a2, b2, rng)) wins++;
        }
        return (double) wins / MC_SAMPLES;
    }

    /**
     * Sample from Beta(alpha, beta) using Marsaglia-Tsang Gamma sampler.
     * Accurate for all alpha, beta > 0.
     */
    static double sampleBeta(double alpha, double beta, Random rng) {
        double g1 = sampleGamma(alpha, rng);
        double g2 = sampleGamma(beta, rng);
        return g1 / (g1 + g2);
    }

    /**
     * Marsaglia & Tsang (2000) Gamma sampler.
     * Works for shape > 0 via the GS/squeeze algorithm.
     */
    static double sampleGamma(double shape, Random rng) {
        if (shape < 1.0) {
            // Reduction: Gamma(a) = Gamma(a+1) * U^(1/a)
            return sampleGamma(shape + 1.0, rng) * Math.pow(rng.nextDouble(), 1.0 / shape);
        }
        double d = shape - 1.0 / 3.0;
        double c = 1.0 / Math.sqrt(9.0 * d);
        while (true) {
            double x, v;
            do {
                x = rng.nextGaussian();
                v = 1.0 + c * x;
            } while (v <= 0);
            v = v * v * v;
            double u = rng.nextDouble();
            if (u < 1.0 - 0.0331 * (x * x) * (x * x)) return d * v;
            if (Math.log(u) < 0.5 * x * x + d * (1.0 - v + Math.log(v))) return d * v;
        }
    }

    // ── Helpers ──────────────────────────────────────────────────────

    private boolean kill(MetaAdSet adSet, String reason) {
        adSet.setGateStatus("KILLED");
        adSet.setGateKillReason(reason);
        adSet.setGateKillAt(Instant.now());
        adSet.setStatus("PAUSED");
        adSetRepo.save(adSet);
        log.info("KILLED adSet={} reason={}", adSet.getId(), reason);
        return true;
    }

    private static long safe(Long v) {
        return v == null ? 0L : v;
    }
}
```

8.2 `platform-api/.../service/ExperimentService.java`

Назначение: Жизненный цикл A/B экспериментов (DRAFT→CONCLUDED); делегация математики FastAPI.

Математическая логика:
- Spring Boot НЕ вычисляет α/β — только вызывает FastAPI `/decide/experiment/should-conclude` и `/decide/experiment/check-pause`.
- Параметры из БД: min_ess, auto_stop_p.
- Бизнес-правила: ≥2 варианта, ровно 1 контрол перед review.

Взаимодействие: `ExperimentMonitorScheduler.java` (регулярный мониторинг); `experiment_service.py` (математика).

Полный код

```java
package com.marketingai.service;

import com.marketingai.config.AppProperties;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.http.HttpStatus;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestTemplate;
import org.springframework.web.server.ResponseStatusException;

import java.util.*;

/**
 * A/B Experiment Lifecycle (vision §3; arch §16.9).
 * Lifecycle: DRAFT → PENDING_REVIEW → APPROVED → ACTIVE → PAUSED/CONCLUDED → RETIRED
 * FastAPI is the SOLE writer of thompson_arms alpha/beta.
 */
@Service
@RequiredArgsConstructor
@Slf4j
public class ExperimentService {

    private final JdbcTemplate jdbc;
    private final RestTemplate restTemplate;
    private final AppProperties appProperties;
    private final AlertService  alertService;

    public UUID createExperiment(UUID tenantId, UUID createdBy, Map<String, Object> body) {
        UUID id = UUID.randomUUID();
        jdbc.update("""
            INSERT INTO experiments
                (id, tenant_id, name, hypothesis, experiment_type, created_by)
            VALUES (?, ?, ?, ?, ?, ?)
            """, id, tenantId,
            body.get("name"), body.get("hypothesis"),
            body.getOrDefault("experiment_type", "landing"), createdBy);
        return id;
    }

    public void submitForReview(UUID tenantId, UUID experimentId) {
        Integer variantCount = jdbc.queryForObject("""
            SELECT COUNT(*) FROM landing_variants
            WHERE experiment_id = ? AND status NOT IN ('ARCHIVED','RETIRED')
            """, Integer.class, experimentId);
        if (variantCount == null || variantCount < 2) {
            throw new ResponseStatusException(HttpStatus.BAD_REQUEST,
                "Experiment needs ≥2 variants before review");
        }
        Integer controlCount = jdbc.queryForObject("""
            SELECT COUNT(*) FROM landing_variants
            WHERE experiment_id = ? AND is_control = TRUE
            """, Integer.class, experimentId);
        if (controlCount == null || controlCount != 1) {
            throw new ResponseStatusException(HttpStatus.BAD_REQUEST,
                "Experiment needs exactly 1 control variant");
        }
        jdbc.update("""
            UPDATE experiments SET status = 'PENDING_REVIEW', updated_at = NOW()
            WHERE id = ? AND tenant_id = ? AND status = 'DRAFT'
            """, experimentId, tenantId);
        logEvent(tenantId, experimentId, "SUBMITTED", null, null);
    }
```

```
    public void approve(UUID tenantId, UUID experimentId, UUID approvedBy) {
        jdbc.update("""
            UPDATE experiments SET status = 'APPROVED', approved_by = ?, updated_at = NOW()
            WHERE id = ? AND tenant_id = ? AND status = 'PENDING_REVIEW'
            """, approvedBy, experimentId, tenantId);
        logEvent(tenantId, experimentId, "APPROVED", approvedBy, null);
    }

    public void start(UUID tenantId, UUID experimentId) {
        jdbc.update("""
            UPDATE experiments SET status = 'ACTIVE', started_at = NOW(), updated_at = NOW()
            WHERE id = ? AND tenant_id = ? AND status = 'APPROVED'
            """, experimentId, tenantId);
        jdbc.update("""
            UPDATE landing_variants SET status = 'ACTIVE'
            WHERE experiment_id = ? AND status = 'DRAFT'
            """, experimentId);
        logEvent(tenantId, experimentId, "STARTED", null, null);
        log.info("Experiment {} started", experimentId);
    }

    public void checkExperiment(UUID tenantId, UUID experimentId) {
        Map<String, Object> exp;
        try {
            exp = jdbc.queryForMap("""
                SELECT id, min_ess, auto_stop_p FROM experiments
                WHERE id = ? AND status = 'ACTIVE'
                """, experimentId);
        } catch (Exception e) { return; }

        List<Map<String, Object>> arms = jdbc.queryForList("""
            SELECT lv.id AS variant_id, lv.is_control,
                   COALESCE(ta.alpha, 1.0) AS alpha,
                   COALESCE(ta.beta,  1.0) AS beta
            FROM landing_variants lv
            LEFT JOIN LATERAL (
                SELECT 1 + SUM(ta.alpha - 1) AS alpha, 1 + SUM(ta.beta - 1) AS beta
                FROM thompson_arms ta
                WHERE ta.variant_id = lv.id AND ta.tenant_id = lv.tenant_id
            ) ta ON TRUE
            WHERE lv.experiment_id = ? AND lv.status = 'ACTIVE'
            """, experimentId);

        if (arms.isEmpty()) return;

        int    minEss = ((Number) exp.get("min_ess")).intValue();
        double autoP  = ((Number) exp.get("auto_stop_p")).doubleValue();

        // Check auto-conclude via FastAPI
        try {
            Map<String, Object> req = Map.of("arms", arms, "min_ess", minEss, "auto_stop_p", autoP);
            @SuppressWarnings("unchecked")
            Map<String, Object> result = restTemplate.postForObject(
                appProperties.getMlServiceUrl() + "/decide/experiment/should-conclude", req, Map.class);
            if (result != null && Boolean.TRUE.equals(result.get("should_conclude"))) {
                String winnerId = (String) result.get("winner_variant_id");
                if (winnerId != null) {
                    concludeExperiment(tenantId, experimentId, UUID.fromString(winnerId),
                        (String) result.get("reason"));
                    return;
                }
            }
        } catch (Exception e) {
            log.warn("Conclude check failed for {}: {}", experimentId, e.getMessage());
        }
```

```
        // Check auto-pause arms via FastAPI
        try {
            Map<String, Object> req = Map.of("arms", arms, "min_ess", 50, "auto_stop_p", 0.95);
            @SuppressWarnings("unchecked")
            List<Map<String, Object>> pauses = (List<Map<String, Object>>)
                restTemplate.postForObject(
                    appProperties.getMlServiceUrl() + "/decide/experiment/check-pause", req,
List.class);
            if (pauses != null) {
                for (Map<String, Object> p : pauses) {
                    if (Boolean.TRUE.equals(p.get("should_pause"))) {
                        UUID variantId = UUID.fromString((String) p.get("variant_id"));
                        jdbc.update("""
                            UPDATE landing_variants SET status = 'PAUSED'
                            WHERE id = ? AND status = 'ACTIVE'
                            """, variantId);
                        alertService.sendAlert(
                            "Variant " + variantId + " auto-paused: " + p.get("reason"));
                    }
                }
            }
        } catch (Exception e) {
            log.warn("Pause check failed for {}: {}", experimentId, e.getMessage());
        }
    }

    private void concludeExperiment(UUID tenantId, UUID experimentId, UUID winnerId, String reason) {
        jdbc.update("""
            UPDATE experiments
            SET status = 'CONCLUDED', concluded_at = NOW(),
                winner_variant_id = ?, updated_at = NOW()
            WHERE id = ? AND tenant_id = ?
            """, winnerId, experimentId, tenantId);
        logEvent(tenantId, experimentId, "CONCLUDED", null, reason);
        alertService.sendAlert(
            "Experiment " + experimentId + " concluded — winner: " + winnerId);
    }

    private void logEvent(UUID tenantId, UUID expId, String type, UUID by, String note) {
        jdbc.update("""
            INSERT INTO experiment_events (experiment_id, tenant_id, event_type, triggered_by, note)
            VALUES (?, ?, ?, ?, ?)
            """, expId, tenantId, type, by, note);
    }
}
```

8.3 `platform-api/.../service/RewardService.java`

Назначение: Фиксация награды за прохождение воронки; триггер Thompson reward в FastAPI.

Математическая логика:
- Spring Boot пишет в reward_events и quality_gate_log.
- При is_quality_stage = TRUE → POST FastAPI `/thompson/reward` с payload `{reward: 1.0}`.
- **CRITICAL INVARIANT:** FastAPI — единственный писатель thompson_arms.alpha/beta. Spring Boot только уведомляет.

Взаимодействие: `RewardController.java`; `thompson_service.py` применяет reward.

Полный код

```
package com.marketingai.service;

import com.marketingai.config.AppProperties;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.http.HttpEntity;
import org.springframework.http.HttpHeaders;
import org.springframework.http.MediaType;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestTemplate;

import java.util.List;
import java.util.Map;
import java.util.UUID;

/**
 * Reward Ladder — records reward events and triggers Thompson arm updates (vision §19.3).
 * FastAPI is the SOLE writer of thompson_arms alpha/beta. Spring only fires the notification.
 */
@Service
@RequiredArgsConstructor
@Slf4j
public class RewardService {

    private final JdbcTemplate jdbc;
    private final RestTemplate restTemplate;
    private final AppProperties appProperties;
    private final AlertService  alertService;
    private final FunnelService funnelService;

    /**
     * tenants.settings.reward_levels (roadmap 8-3, Audit F1): {"L1": "SLOT_SELECTED", "L2":
"FORM_SUBMIT",
     * "corrector": "CALL_HELD"}. L1/L2 stages reward the landing arm; the corrector never touches
     * alpha/beta (it drives the weekly show-rate multiplier instead).
     */
    public record RewardLevels(String l1, String l2, String corrector) {
        static RewardLevels parse(Object json) {
            Map<String, Object> m = com.marketingai.util.Jsonb.asMap(json);
            if (m.isEmpty()) return null;
            return new RewardLevels(str(m.get("L1")), str(m.get("L2")), str(m.get("corrector")));
        }
        private static String str(Object o) { return o == null ? null : o.toString(); }
    }

    /**
     * Reward type to send to FastAPI for a reached stage, or null when the stage carries no
     * Thompson reward. With reward_levels configured, only L1/L2 reward (L1 = micro, L2 = secondary);
     * without it, the legacy rule applies: the quality stage rewards with its CRM-mapped type.
     */
    static String rewardTypeFor(String stageKey, RewardLevels levels, boolean isQualityStage) {
        if (stageKey == null) return null;
        if (levels != null) {
            if (stageKey.equalsIgnoreCase(levels.l1())) return "CTA_CLICK";
            if (stageKey.equalsIgnoreCase(levels.l2())) return "FORM_SUBMIT";
            return null;  // corrector and every other stage: no α/β increment
        }
        return isQualityStage ? rewardTypeForStage(stageKey) : null;
    }

    RewardLevels loadRewardLevels(UUID tenantId) {
        try {
            List<Map<String, Object>> rows = jdbc.queryForList(
```

```
                    "SELECT settings->'reward_levels' AS rl FROM tenants WHERE id = ?", tenantId);
            return rows.isEmpty() ? null : RewardLevels.parse(rows.get(0).get("rl"));
        } catch (Exception e) {
            return null;
        }
    }

    /**
     * First-party event (SLOT_SELECTED, FORM_SUBMIT, …) mapped to a funnel stage through
     * funnel_stages.maps_to_event_type — the L1/L2 rewards of an investor funnel arrive this way,
     * not from the CRM. Called by EventIngestionService.
     */
    public void onEventIngested(UUID tenantId, String sessionId, UUID variantId, String eventType) {
        funnelService.stageForEventType(tenantId, eventType)
                .ifPresent(stage -> onStageReached(tenantId, sessionId, variantId, stage));
    }

    /**
     * Map a funnel stage to the Thompson reward type understood by FastAPI /decide/reward
     * (thompson_service.REWARD_MAP). Quality stages are CRM-confirmed outcomes.
     */
    static String rewardTypeForStage(String stageKey) {
        if (stageKey == null) return "CONVERSION";
        String k = stageKey.toUpperCase();
        if (k.endsWith("_HELD") || k.contains("MEETING") || k.contains("ATTENDED")) return
"MEETING_ATTENDED";
        if (k.contains("ANSWERED")) return "CALL_ANSWERED";
        if (k.contains("FUNDS") || k.contains("PURCHASE") || k.contains("PAID") || k.contains("WON"))
return "PURCHASE";
        if (k.contains("FORM_SUBMIT")) return "FORM_SUBMIT";
        return "CONVERSION";
    }

    /**
     * Called when a funnel stage is reached for a session.
     * Writes reward_events. If quality stage, also writes quality_gate_log and
     * posts the reward to FastAPI /decide/reward (binary increment 1.0 — FastAPI sole writer).
     * The decision_id is looked up in decision_log by (tenant, session, variant); the Redis
     * decision key alone expires after 1 h while CRM rewards arrive days later (audit 2.1/2.2).
     */
    public void onStageReached(UUID tenantId, String sessionId, UUID variantId, String stageKey) {
        List<Boolean> qualityCheck = jdbc.queryForList("""
            SELECT is_quality_key FROM funnel_stages
            WHERE tenant_id = ? AND stage_key = ?
            """, Boolean.class, tenantId, stageKey);
        boolean isQuality = !qualityCheck.isEmpty() && Boolean.TRUE.equals(qualityCheck.get(0));

        List<Double> weightList = jdbc.queryForList("""
            SELECT COALESCE(reward_weight, 1.0) FROM funnel_stages
            WHERE tenant_id = ? AND stage_key = ?
            """, Double.class, tenantId, stageKey);
        double rewardWeight = weightList.isEmpty() ? 1.0 : weightList.get(0);

        jdbc.update("""
            INSERT INTO reward_events
                (tenant_id, session_id, variant_id, stage_key, is_quality_stage, reward_value)
            VALUES (?, ?, ?, ?, ?, ?)
            """, tenantId, sessionId, variantId, stageKey, isQuality, rewardWeight);

        if (isQuality) {
            jdbc.update("""
                INSERT INTO quality_gate_log
                    (tenant_id, session_id, variant_id, stage_key)
                VALUES (?, ?, ?, ?)
                ON CONFLICT (session_id, variant_id, stage_key) DO NOTHING
```

```
                """, tenantId, sessionId, variantId, stageKey);
        }

        RewardLevels levels = loadRewardLevels(tenantId);
        String rewardType = rewardTypeFor(stageKey, levels, isQuality);
        if (rewardType != null && variantId != null) {
            try {
                List<String> decisionIds = jdbc.queryForList("""
                    SELECT id::text FROM decision_log
                    WHERE tenant_id = ? AND session_id = ? AND chosen_variant_id = ?
                    ORDER BY decided_at DESC LIMIT 1
                    """, String.class, tenantId, sessionId, variantId);
                if (decisionIds.isEmpty()) {
                    log.info("No Thompson decision for session {} variant {} — reward not forwarded",
sessionId, variantId);
                } else {
                    Map<String, Object> req = Map.of(
                        "decision_id", decisionIds.get(0),
                        "reward_type", rewardType,
                        "tenant_configured", levels != null
                    );
                    HttpHeaders headers = new HttpHeaders();
                    headers.setContentType(MediaType.APPLICATION_JSON);
                    headers.set("X-Internal-API-Key", appProperties.getInternalApiKey());
                    restTemplate.postForObject(appProperties.getMlServiceUrl() + "/decide/reward",
                            new HttpEntity<>(req, headers), Map.class);
                }
            } catch (Exception e) {
                log.warn("Thompson reward notification failed for variant {}: {}", variantId,
e.getMessage());
            }
        }

        log.debug("Stage reward: tenant={} stage={} quality={}", tenantId, stageKey, isQuality);
    }

    public Map<String, Object> getRewardLadder(UUID tenantId) {
        List<Map<String, Object>> stages = jdbc.queryForList("""
            SELECT
                fs.stage_key,
                fs.label AS stage_label,
                fs.ordinal,
                fs.is_quality_key,
                fs.reward_weight,
                fs.micro_reward_weight,
                cem.meta_event_name                                 AS capi_event,
                COUNT(re.id)                                        AS total_rewards,
                COUNT(re.id) FILTER (WHERE re.is_quality_stage)     AS quality_rewards,
                COUNT(DISTINCT re.session_id)                       AS unique_sessions
            FROM funnel_stages fs
            LEFT JOIN reward_events re
                ON re.tenant_id = fs.tenant_id AND re.stage_key = fs.stage_key
            LEFT JOIN capi_event_map cem
                ON cem.tenant_id = fs.tenant_id AND cem.stage_key = fs.stage_key
            WHERE fs.tenant_id = ?
            GROUP BY fs.stage_key, fs.label, fs.ordinal,
                     fs.is_quality_key, fs.reward_weight, fs.micro_reward_weight,
                     cem.meta_event_name
            ORDER BY fs.ordinal
            """, tenantId);

        // Weekly counters: this week vs prev week
        List<Map<String, Object>> thisWeek = jdbc.queryForList("""
            SELECT stage_key, COUNT(*) AS cnt
            FROM reward_events
```

```
            WHERE tenant_id = ? AND rewarded_at >= date_trunc('week', NOW())
            GROUP BY stage_key
            """, tenantId);

        List<Map<String, Object>> prevWeek = jdbc.queryForList("""
            SELECT stage_key, COUNT(*) AS cnt
            FROM reward_events
            WHERE tenant_id = ?
              AND rewarded_at >= date_trunc('week', NOW()) - INTERVAL '7 days'
              AND rewarded_at <  date_trunc('week', NOW())
            GROUP BY stage_key
            """, tenantId);

        java.util.Map<String, Long> thisMap = thisWeek.stream().collect(
            java.util.stream.Collectors.toMap(
                r -> (String) r.get("stage_key"),
                r -> ((Number) r.get("cnt")).longValue()
            ));
        java.util.Map<String, Long> prevMap = prevWeek.stream().collect(
            java.util.stream.Collectors.toMap(
                r -> (String) r.get("stage_key"),
                r -> ((Number) r.get("cnt")).longValue()
            ));

        List<Map<String, Object>> weeklyCounters = stages.stream().map(s -> {
            String key  = (String) s.get("stage_key");
            long   curr = thisMap.getOrDefault(key, 0L);
            long   prev = prevMap.getOrDefault(key, 0L);
            Double delta = prev > 0 ? ((curr - prev) * 100.0 / prev) : null;
            return java.util.Map.<String, Object>of(
                "stage_key",       key,
                "count_this_week", curr,
                "count_prev_week", prev,
                "delta_pct",       delta != null ? Math.round(delta * 10.0) / 10.0 : null
            );
        }).collect(java.util.stream.Collectors.toList());

        return Map.of("stages", stages, "weekly_counters", weeklyCounters);
    }
}
```

8.4 `platform-api/.../scheduler/PosteriorRecalcScheduler.java`

Назначение: Мониторинг свежести posterior — логирование вариантов с новыми conversion events.

Математическая логика:
- Нет математики — только запрос к БД.
- Находит варианты с новыми CONVERSION событиями за последние 15 минут.
- **НЕ пишет** α/β — только логирует. Обновление posterior происходит при следующем вызове FastAPI `/decide`.

Полный код

```java
package com.marketingai.scheduler;

import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;

import java.util.List;
import java.util.Map;

/**
 * Logs variants that received new conversion events in the last 15 minutes.
 * Actual alpha/beta writes happen in FastAPI /decide (single writer rule).
 * This scheduler ensures the decide endpoint is called for active variants
 * and the posterior stays fresh (vision §2.x, §10.1 single-writer invariant).
 */
@Slf4j
@Component
@RequiredArgsConstructor
public class PosteriorRecalcScheduler {

    private final JdbcTemplate jdbc;

    @Scheduled(fixedDelay = 900_000)
    public void logStaleVariants() {
        // Find variants that have new conversion events but haven't had a /decide call
        // ONLY FastAPI writes alpha/beta — Spring Boot NEVER modifies thompson_arms
        List<Map<String, Object>> stale = jdbc.queryForList("""
                SELECT DISTINCT v.id  AS variant_id,
                                v.tenant_id,
                                COUNT(e.id) AS new_conversions
                FROM landing_variants v
                JOIN sessions s ON s.tenant_id = v.tenant_id AND s.variant_id = v.id
                JOIN events   e ON e.session_id = s.id AND e.tenant_id = s.tenant_id
                WHERE v.status = 'ACTIVE'
                  AND e.event_type IN ('CONVERSION', 'FORM_SUBMIT')
                  AND e.event_time > NOW() - INTERVAL '15 minutes'
                GROUP BY v.id, v.tenant_id
                LIMIT 200
                """);

        if (!stale.isEmpty()) {
            log.debug("PosteriorRecalcScheduler: {} variants have new conversion events — "
                    + "posteriors will refresh on next FastAPI /decide call", stale.size());
        }
    }
}
```

ИТОГ: МАТРИЦА РИСКОВ ДЛЯ АУДИТА

Файл	Критический риск
experiment_service.py	n=5000 Monte Carlo для P(best) — погрешность ~0.7% у границы 0.95
thompson_service.py	n=200 propensity — грубая оценка для OPE
noshow_service.py	Все параметры экспертные, нет калибровки на данных
lead_value_service.py	OLS slope [0.8,1.2]: если истинный >1.2 — постоянное занижение PLV

Файл	Критический риск
reward_service.py	apply_show_rate_to_gates: скорр. threshold может превысить 1.0
cluster_service.py	Несогласованность метрики расстояния (fit vs assign)
segment_re_service.py	$TAU^2=4$ → почти нет shrinkage при n≥10
hierarchy_prior_service.py	StudentT→Normal conjugate: смешивание семейств
attribution_service.py	Shapley n=200 коалиций — погрешность ±5%
geo_lift_service.py	Простой scale-ratio synthetic control; bootstrap нарушает время. зависимость
ope_service.py	w_max=10 обрезает редкие рукавы; propensity из n=200
portfolio_service.py	Sharpe ×365: зависит от временно́го масштаба return
evolution_service.py	Single-parent crossover возвращает клоны
GateEvaluationService.java	new Random() без seed → нестабильность при граничных значениях
moderation_service.py	P(reject): независимость нарушений предполагается
cb_bayesian_service.py	CPA=spend/CVR: корректно только если clicks=total exposures

---

Документ создан 2026-10-17. Версия платформы: Phase 8 complete, Client #1 onboarded.